

```
In [1]: # A1 – required packages (run once if not already installed)
#%pip install -q pandas numpy matplotlib seaborn scipy missingno scikit-learn
#%pip install --force-reinstall matplotlib
#%pip uninstall -y matplotlib
#%pip install matplotlib
```

```
In [2]: #!rmdir /s /q "c:\Users\sanda\AppData\Local\Programs\Python\Python311\Lib\site-pack
#!rmdir /s /q "c:\Users\sanda\AppData\Local\Programs\Python\Python311\Lib\site-pack
```

```
In [3]: import os
import warnings
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import missingno as msno
from scipy import stats
import math

def stratified_subsample(df, n, label_col='isFraud', random_state=42):
    """Stratified subsample; avoids groupby.apply (pandas 2.2+ drops grouping cols)
    if label_col not in df.columns:
        raise KeyError(
            f"Column '{label_col}' not found – use train_transaction.csv (not test)
            f"Columns (first 12): {list(df.columns[:12])}"
        )
    if len(df) <= n:
        return df.copy().reset_index(drop=True)
    parts = []
    for _, group in df.groupby(label_col, sort=False):
        k = max(1, int(round(n * len(group) / len(df))))
        k = min(k, len(group))
        parts.append(group.sample(n=k, random_state=random_state))
    return pd.concat(parts, ignore_index=True)

# 1. Block warnings at the system level before pandas fully initializes its internal
os.environ['PYTHONWARNINGS'] = 'ignore'

# 2. Force python and pandas to completely suppress formatting warnings
warnings.filterwarnings('ignore')
warnings.simplefilter(action='ignore', category=FutureWarning)

# Establish global reproducibility seed as mandated by the course guidelines
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

# Configure production-ready visualization standards
sns.set_theme(style="whitegrid")
plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.labelsize'] = 12
```

Setup — files & scripts (same folder as this notebook)

Required data files

- `train_transaction.csv`
- `train_identity.csv`

B0 profile exports (generated by Section B0 or scripts below)

- `feature_profile_434.html` — scroll + Ctrl+F to search any column
- `feature_profile_434.csv` — same table for Excel
- `b0_comparison_report.md` — B0 custom vs YData comparison

Optional: `ydata_profile_report.html` (~65 MB, interactive; needs `ydata-profiling`)

Standalone Python scripts (run from terminal in this folder; no Jupyter needed)

Script	Purpose
<code>generate_b0_profile.py</code>	Regenerate B0 custom CSV + HTML (~25 s, full dataset)
<code>generate_ydata_profile.py</code>	Regenerate YData HTML (~10 min, 50K sample; optional)
<code>compare_b0_profiles.py</code>	Run both profilers and write <code>b0_comparison_report.md</code>
<code>validate_a1_audit.py</code>	Smoke test: 434 rows, D5 pool, section structure

```
python generate_b0_profile.py
python compare_b0_profiles.py --report-only
python validate_a1_audit.py
```

Optional YData install (B0.5 only — skip if you use B0 custom HTML):

```
pip install ydata-profiling --user --no-deps
pip install phik visions dacite minify-html filetype wordcloud imagehash
multimethod typeguard puremagic --user
```

Recommendation: use **B0 custom** as the primary profile for A1; YData as an optional appendix.

A1 - Exploratory Data Analysis (EDA) for Feature Selection and Dimensionality Reduction

Section A: Dataset Identification & Justification

The IEEE-CIS Fraud Detection dataset was selected for this project. The dataset contains real-world online transactions and information about whether a transaction was fraudulent or legitimate. Because the dataset contains more than 500,000 records and hundreds of variables, it is suitable for exploratory data analysis, feature selection, and dimensionality reduction.

Citation

@misc{ieee-fraud-detection, author = {Addison Howard and Bernadette Bouchon-Meunier and IEEE CIS and inversion and John Lei and Lynn@Vesta and Marcus2010 and Prof. Hussein Abbass}, title = {IEEE-CIS Fraud Detection}, year = {2019}, howpublished = {\url{https://kaggle.com/competitions/ieee-fraud-detection}}, note = {Kaggle} }

Section B: Data Dictionary

The IEEE-CIS Fraud Detection dataset is structured around a relational schema composed of two main files linked by a unique transaction identifier: `train_transaction.csv` and `train_identity.csv`. The features are organized, classified, and operationally defined below according to their data types and statistical measurement scales.

```
In [4]: # 1. Stream files from disk into Pandas DataFrames
# Adjust the strings if your files are inside a specific 'data/' subfolder
train_transaction = pd.read_csv('train_transaction.csv')
train_identity = pd.read_csv('train_identity.csv')

print(f"Base transaction matrix loaded: {train_transaction.shape[0]} rows, {train_t
print(f"Base identity matrix loaded: {train_identity.shape[0]} rows, {train_identi

# 2. Execute relational merge using an asymmetric Left Join on the primary key
# This guarantees we don't drop transactions that lack corresponding identity logs
df_train = pd.merge(train_transaction, train_identity, on='TransactionID', how='lef

print("-" * 60)
print(f"Consolidated EDA Matrix Shape: {df_train.shape[0]} rows and {df_train.shape
```

Base transaction matrix loaded: 590540 rows, 394 features.
Base identity matrix loaded: 144233 rows, 41 features.

Consolidated EDA Matrix Shape: 590540 rows and 434 features.

B0 — Global Feature Profile (434 columns)

This section profiles **every column** in the merged dataset (YData Profiling style):

Logical type	What you see in the table
Categorical	Sample of unique values (top categories with counts)
Numerical / Discrete Numerical	mean, min, max

Logical type	What you see in the table
Binary	Value counts in table + histogram grid below
Identifier / Target	Type label only

The full **434-row** table is displayed below and exported to `feature_profile_434.csv` and `feature_profile_434.html` for scrolling/search.

```
In [5]: import re
from IPython.display import display, HTML

def infer_feature_family(col):
    if col == "TransactionID":
        return "Identifier"
    if col == "isFraud":
        return "Target"
    if col in ["TransactionDT", "TransactionAmt"]:
        return "Transaction"
    if col == "ProductCD":
        return "Product"
    if re.match(r"card[1-6]$", col):
        return "Card"
    if col in ["addr1", "addr2"]:
        return "Address"
    if col in ["P_emaildomain", "R_emaildomain"]:
        return "Email"
    if re.match(r"M[1-9]$", col):
        return "Match Flags"
    if re.match(r"C\d+$", col):
        return "Count Features"
    if re.match(r"D\d+$", col):
        return "Timedelta Features"
    if col in ["dist1", "dist2"]:
        return "Distance"
    if re.match(r"V\d+$", col):
        return "Vesta Engineered"
    if re.match(r"id_\d+$", col):
        return "Identity"
    if col in ["DeviceType", "DeviceInfo"]:
        return "Device"
    return "Other"

def infer_logical_type(series, col):
    if col == "TransactionID":
        return "Identifier"
    if col == "isFraud":
        return "Target"
    unique = series.nunique(dropna=True)
    dtype = series.dtype
    if unique == 2:
        return "Binary"
    if dtype == "object" or str(dtype) == "category":
        return "Categorical"
    if pd.api.types.is_numeric_dtype(series):
```

```

        if nunique < 20:
            return "Discrete Numerical"
        return "Numerical"
    return "Other"

MAX_CAT_SAMPLES = 12

profile_rows = []
for col in df_train.columns:
    s = df_train[col]
    lt = infer_logical_type(s, col)
    missing_pct = round(s.isna().mean() * 100, 2)
    n_unique = int(s.nunique(dropna=True))

    row = {
        "feature": col,
        "family": infer_feature_family(col),
        "logical_type": lt,
        "dtype": str(s.dtype),
        "missing_pct": missing_pct,
        "n_unique": n_unique,
        "mean": None,
        "min": None,
        "max": None,
        "sample_values": "",
    }

    if lt == "Categorical":
        vc = s.dropna().astype(str).value_counts().head(MAX_CAT_SAMPLES)
        row["sample_values"] = "; ".join(f"{k} ({v:},{})" for k, v in vc.items())
        if n_unique > MAX_CAT_SAMPLES:
            row["sample_values"] += f"; ... ({n_unique - MAX_CAT_SAMPLES} more)"
    elif lt == "Binary":
        vc = s.value_counts(dropna=False)
        row["sample_values"] = "; ".join(f"{k}: {v:},{}" for k, v in vc.items())
    elif lt in ("Numerical", "Discrete Numerical"):
        sn = pd.to_numeric(s, errors="coerce").dropna()
        if len(sn) > 0:
            row["mean"] = round(float(sn.mean()), 4)
            row["min"] = round(float(sn.min()), 4)
            row["max"] = round(float(sn.max()), 4)
    elif lt in ("Identifier", "Target"):
        row["sample_values"] = "(ID/target - no value sample)"

    profile_rows.append(row)

feature_profile = pd.DataFrame(profile_rows)
feature_audit = feature_profile.copy() # alias for downstream sections

print(f"Dataset shape: {df_train.shape}")
print(f"Profile rows: {len(feature_profile)} (expect 434)")
print("\nCounts by logical type:")
display(feature_profile["logical_type"].value_counts().reset_index(name="count"))

```

Dataset shape: (590540, 434)
Profile rows: 434 (expect 434)

Counts by logical type:

	logical_type	count
0	Numerical	262
1	Discrete Numerical	132
2	Binary	25
3	Categorical	13
4	Target	1
5	Identifier	1

```
In [6]: # --- Full 434-column profile table ---
cols_show = [
    "feature", "family", "logical_type", "dtype",
    "missing_pct", "n_unique", "mean", "min", "max", "sample_values",
]
profile_show = feature_profile[cols_show].copy()

# Export (scroll/search outside notebook)
profile_show.to_csv("feature_profile_434.csv", index=False)
profile_show.to_html("feature_profile_434.html", index=False, escape=False)
print("Exported: feature_profile_434.csv | feature_profile_434.html")

# Show ALL 434 rows in notebook (scrollable output)
pd.set_option("display.max_rows", 500)
pd.set_option("display.max_colwidth", 80)
display(profile_show.style.set_properties(
    subset=["sample_values"], **{"max-width": "400px", "text-align": "left"}
))
pd.reset_option("display.max_rows")
pd.reset_option("display.max_colwidth")
```

Exported: feature_profile_434.csv | feature_profile_434.html

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
0	TransactionID	Identifier	Identifier	int64	0.000000	590540	nan
1	isFraud	Target	Target	int64	0.000000	2	nan
2	TransactionDT	Transaction	Numerical	int64	0.000000	573349	7372311.310100
3	TransactionAmt	Transaction	Numerical	float64	0.000000	20902	135.027200
4	ProductCD	Product	Categorical	object	0.000000	5	nan
5	card1	Card	Numerical	int64	0.000000	13553	9898.734700
6	card2	Card	Numerical	float64	1.510000	500	362.555500
7	card3	Card	Numerical	float64	0.270000	114	153.194900
8	card4	Card	Categorical	object	0.270000	4	nan
9	card5	Card	Numerical	float64	0.720000	119	199.278900
10	card6	Card	Categorical	object	0.270000	4	nan
11	addr1	Address	Numerical	float64	11.130000	332	290.733800
12	addr2	Address	Numerical	float64	11.130000	74	86.800600
13	dist1	Distance	Numerical	float64	59.650000	2651	118.502200
14	dist2	Distance	Numerical	float64	93.630000	1751	231.855400
15	P_emaildomain	Email	Categorical	object	15.990000	59	nan
16	R_emaildomain	Email	Categorical	object	76.750000	60	nan

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
17	C1	Count Features	Numerical	float64	0.000000	1657	14.092500
18	C2	Count Features	Numerical	float64	0.000000	1216	15.269700
19	C3	Count Features	Numerical	float64	0.000000	27	0.005600
20	C4	Count Features	Numerical	float64	0.000000	1260	4.092200
21	C5	Count Features	Numerical	float64	0.000000	319	5.571500
22	C6	Count Features	Numerical	float64	0.000000	1328	9.071100
23	C7	Count Features	Numerical	float64	0.000000	1103	2.848500
24	C8	Count Features	Numerical	float64	0.000000	1253	5.144600
25	C9	Count Features	Numerical	float64	0.000000	205	4.480200
26	C10	Count Features	Numerical	float64	0.000000	1231	5.240300
27	C11	Count Features	Numerical	float64	0.000000	1476	10.241500
28	C12	Count Features	Numerical	float64	0.000000	1199	4.076200
29	C13	Count Features	Numerical	float64	0.000000	1597	32.539900
30	C14	Count Features	Numerical	float64	0.000000	1108	8.295200
31	D1	Timedelta Features	Numerical	float64	0.210000	641	94.347600
32	D2	Timedelta Features	Numerical	float64	47.550000	641	169.563200
33	D3	Timedelta Features	Numerical	float64	44.510000	649	28.343300

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
34	D4	Timedelta Features	Numerical	float64	28.600000	808	140.002400
35	D5	Timedelta Features	Numerical	float64	52.470000	688	42.336000
36	D6	Timedelta Features	Numerical	float64	87.610000	829	69.805700
37	D7	Timedelta Features	Numerical	float64	93.410000	597	41.638900
38	D8	Timedelta Features	Numerical	float64	87.310000	12353	146.058100
39	D9	Timedelta Features	Numerical	float64	87.310000	24	0.561100
40	D10	Timedelta Features	Numerical	float64	12.870000	818	123.982100
41	D11	Timedelta Features	Numerical	float64	47.290000	676	146.621500
42	D12	Timedelta Features	Numerical	float64	89.040000	635	54.037500
43	D13	Timedelta Features	Numerical	float64	89.510000	577	17.901300
44	D14	Timedelta Features	Numerical	float64	89.470000	802	57.724400
45	D15	Timedelta Features	Numerical	float64	15.090000	859	163.744600
46	M1	Match Flags	Binary	object	45.910000	2	nan
47	M2	Match Flags	Binary	object	45.910000	2	nan
48	M3	Match Flags	Binary	object	45.910000	2	nan
49	M4	Match Flags	Categorical	object	47.660000	3	nan
50	M5	Match Flags	Binary	object	59.350000	2	nan
51	M6	Match Flags	Binary	object	28.680000	2	nan
52	M7	Match Flags	Binary	object	58.640000	2	nan

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
53	M8	Match Flags	Binary	object	58.630000	2	nan
54	M9	Match Flags	Binary	object	58.630000	2	nan
55	V1	Vesta Engineered	Binary	float64	47.290000	2	nan
56	V2	Vesta Engineered	Discrete Numerical	float64	47.290000	9	1.045200
57	V3	Vesta Engineered	Discrete Numerical	float64	47.290000	10	1.078100
58	V4	Vesta Engineered	Discrete Numerical	float64	47.290000	7	0.846500
59	V5	Vesta Engineered	Discrete Numerical	float64	47.290000	7	0.877000
60	V6	Vesta Engineered	Discrete Numerical	float64	47.290000	10	1.045700
61	V7	Vesta Engineered	Discrete Numerical	float64	47.290000	10	1.072900
62	V8	Vesta Engineered	Discrete Numerical	float64	47.290000	9	1.027700
63	V9	Vesta Engineered	Discrete Numerical	float64	47.290000	9	1.041500
64	V10	Vesta Engineered	Discrete Numerical	float64	47.290000	5	0.463900
65	V11	Vesta Engineered	Discrete Numerical	float64	47.290000	6	0.479000
66	V12	Vesta Engineered	Discrete Numerical	float64	12.880000	4	0.559700
67	V13	Vesta Engineered	Discrete Numerical	float64	12.880000	7	0.599200
68	V14	Vesta Engineered	Binary	float64	12.880000	2	nan
69	V15	Vesta Engineered	Discrete Numerical	float64	12.880000	8	0.122300
70	V16	Vesta Engineered	Discrete Numerical	float64	12.880000	15	0.123500
71	V17	Vesta Engineered	Discrete Numerical	float64	12.880000	16	0.134000

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
72	V18	Vesta Engineered	Discrete Numerical	float64	12.880000	16	0.135400
73	V19	Vesta Engineered	Discrete Numerical	float64	12.880000	8	0.816400
74	V20	Vesta Engineered	Discrete Numerical	float64	12.880000	15	0.847800
75	V21	Vesta Engineered	Discrete Numerical	float64	12.880000	6	0.129700
76	V22	Vesta Engineered	Discrete Numerical	float64	12.880000	9	0.132300
77	V23	Vesta Engineered	Discrete Numerical	float64	12.880000	14	1.034800
78	V24	Vesta Engineered	Discrete Numerical	float64	12.880000	14	1.058100
79	V25	Vesta Engineered	Discrete Numerical	float64	12.880000	7	0.977700
80	V26	Vesta Engineered	Discrete Numerical	float64	12.880000	13	0.988000
81	V27	Vesta Engineered	Discrete Numerical	float64	12.880000	4	0.000800
82	V28	Vesta Engineered	Discrete Numerical	float64	12.880000	4	0.000800
83	V29	Vesta Engineered	Discrete Numerical	float64	12.880000	6	0.387800
84	V30	Vesta Engineered	Discrete Numerical	float64	12.880000	8	0.406400
85	V31	Vesta Engineered	Discrete Numerical	float64	12.880000	8	0.140800
86	V32	Vesta Engineered	Discrete Numerical	float64	12.880000	15	0.142400
87	V33	Vesta Engineered	Discrete Numerical	float64	12.880000	7	0.130700
88	V34	Vesta Engineered	Discrete Numerical	float64	12.880000	13	0.139200
89	V35	Vesta Engineered	Discrete Numerical	float64	28.610000	4	0.542600
90	V36	Vesta Engineered	Discrete Numerical	float64	28.610000	6	0.579200

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
91	V37	Vesta Engineered	Numerical	float64	28.610000	55	1.108100
92	V38	Vesta Engineered	Numerical	float64	28.610000	55	1.162400
93	V39	Vesta Engineered	Discrete Numerical	float64	28.610000	16	0.166100
94	V40	Vesta Engineered	Discrete Numerical	float64	28.610000	18	0.177100
95	V41	Vesta Engineered	Binary	float64	28.610000	2	nan
96	V42	Vesta Engineered	Discrete Numerical	float64	28.610000	9	0.156100
97	V43	Vesta Engineered	Discrete Numerical	float64	28.610000	9	0.168900
98	V44	Vesta Engineered	Numerical	float64	28.610000	49	1.083900
99	V45	Vesta Engineered	Numerical	float64	28.610000	49	1.120800
100	V46	Vesta Engineered	Discrete Numerical	float64	28.610000	7	1.022300
101	V47	Vesta Engineered	Discrete Numerical	float64	28.610000	9	1.038500
102	V48	Vesta Engineered	Discrete Numerical	float64	28.610000	6	0.383200
103	V49	Vesta Engineered	Discrete Numerical	float64	28.610000	6	0.397700
104	V50	Vesta Engineered	Discrete Numerical	float64	28.610000	6	0.164700
105	V51	Vesta Engineered	Discrete Numerical	float64	28.610000	7	0.170600
106	V52	Vesta Engineered	Discrete Numerical	float64	28.610000	9	0.182700
107	V53	Vesta Engineered	Discrete Numerical	float64	13.060000	6	0.577600
108	V54	Vesta Engineered	Discrete Numerical	float64	13.060000	7	0.620000
109	V55	Vesta Engineered	Discrete Numerical	float64	13.060000	18	1.067700

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
110	V56	Vesta Engineered	Numerical	float64	13.060000	52	1.121000
111	V57	Vesta Engineered	Discrete Numerical	float64	13.060000	7	0.128300
112	V58	Vesta Engineered	Discrete Numerical	float64	13.060000	11	0.132500
113	V59	Vesta Engineered	Discrete Numerical	float64	13.060000	17	0.134400
114	V60	Vesta Engineered	Discrete Numerical	float64	13.060000	17	0.142500
115	V61	Vesta Engineered	Discrete Numerical	float64	13.060000	7	0.829800
116	V62	Vesta Engineered	Discrete Numerical	float64	13.060000	11	0.867600
117	V63	Vesta Engineered	Discrete Numerical	float64	13.060000	8	0.130700
118	V64	Vesta Engineered	Discrete Numerical	float64	13.060000	8	0.141800
119	V65	Vesta Engineered	Binary	float64	13.060000	2	nan
120	V66	Vesta Engineered	Discrete Numerical	float64	13.060000	8	0.981000
121	V67	Vesta Engineered	Discrete Numerical	float64	13.060000	9	0.998100
122	V68	Vesta Engineered	Discrete Numerical	float64	13.060000	3	0.000500
123	V69	Vesta Engineered	Discrete Numerical	float64	13.060000	6	0.390200
124	V70	Vesta Engineered	Discrete Numerical	float64	13.060000	7	0.407900
125	V71	Vesta Engineered	Discrete Numerical	float64	13.060000	7	0.140600
126	V72	Vesta Engineered	Discrete Numerical	float64	13.060000	11	0.145100
127	V73	Vesta Engineered	Discrete Numerical	float64	13.060000	8	0.140000
128	V74	Vesta Engineered	Discrete Numerical	float64	13.060000	9	0.152100

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
129	V75	Vesta Engineered	Discrete Numerical	float64	15.100000	5	0.544300
130	V76	Vesta Engineered	Discrete Numerical	float64	15.100000	7	0.587600
131	V77	Vesta Engineered	Numerical	float64	15.100000	31	1.086900
132	V78	Vesta Engineered	Numerical	float64	15.100000	32	1.144500
133	V79	Vesta Engineered	Discrete Numerical	float64	15.100000	8	0.136900
134	V80	Vesta Engineered	Numerical	float64	15.100000	20	0.144000
135	V81	Vesta Engineered	Numerical	float64	15.100000	20	0.152700
136	V82	Vesta Engineered	Discrete Numerical	float64	15.100000	8	0.844600
137	V83	Vesta Engineered	Discrete Numerical	float64	15.100000	8	0.882000
138	V84	Vesta Engineered	Discrete Numerical	float64	15.100000	8	0.137100
139	V85	Vesta Engineered	Discrete Numerical	float64	15.100000	8	0.149800
140	V86	Vesta Engineered	Numerical	float64	15.100000	31	1.064900
141	V87	Vesta Engineered	Numerical	float64	15.100000	31	1.099500
142	V88	Vesta Engineered	Binary	float64	15.100000	2	nan
143	V89	Vesta Engineered	Discrete Numerical	float64	15.100000	3	0.000900
144	V90	Vesta Engineered	Discrete Numerical	float64	15.100000	6	0.401900
145	V91	Vesta Engineered	Discrete Numerical	float64	15.100000	7	0.420500
146	V92	Vesta Engineered	Discrete Numerical	float64	15.100000	8	0.150300
147	V93	Vesta Engineered	Discrete Numerical	float64	15.100000	8	0.154800

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
148	V94	Vesta Engineered	Discrete Numerical	float64	15.100000	3	0.137000
149	V95	Vesta Engineered	Numerical	float64	0.050000	881	1.038000
150	V96	Vesta Engineered	Numerical	float64	0.050000	1410	3.005200
151	V97	Vesta Engineered	Numerical	float64	0.050000	976	1.718900
152	V98	Vesta Engineered	Discrete Numerical	float64	0.050000	13	0.062000
153	V99	Vesta Engineered	Numerical	float64	0.050000	89	0.895000
154	V100	Vesta Engineered	Numerical	float64	0.050000	29	0.273500
155	V101	Vesta Engineered	Numerical	float64	0.050000	870	0.889200
156	V102	Vesta Engineered	Numerical	float64	0.050000	1285	1.827200
157	V103	Vesta Engineered	Numerical	float64	0.050000	928	1.279300
158	V104	Vesta Engineered	Discrete Numerical	float64	0.050000	16	0.085400
159	V105	Vesta Engineered	Numerical	float64	0.050000	100	0.281100
160	V106	Vesta Engineered	Numerical	float64	0.050000	56	0.164600
161	V107	Vesta Engineered	Binary	float64	0.050000	2	nan
162	V108	Vesta Engineered	Discrete Numerical	float64	0.050000	8	1.004600
163	V109	Vesta Engineered	Discrete Numerical	float64	0.050000	8	1.014800
164	V110	Vesta Engineered	Discrete Numerical	float64	0.050000	8	1.007700
165	V111	Vesta Engineered	Discrete Numerical	float64	0.050000	10	1.002600
166	V112	Vesta Engineered	Discrete Numerical	float64	0.050000	10	1.005400

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
167	V113	Vesta Engineered	Discrete Numerical	float64	0.050000	10	1.003400
168	V114	Vesta Engineered	Discrete Numerical	float64	0.050000	7	1.009300
169	V115	Vesta Engineered	Discrete Numerical	float64	0.050000	7	1.032500
170	V116	Vesta Engineered	Discrete Numerical	float64	0.050000	7	1.015700
171	V117	Vesta Engineered	Discrete Numerical	float64	0.050000	4	1.000400
172	V118	Vesta Engineered	Discrete Numerical	float64	0.050000	4	1.001500
173	V119	Vesta Engineered	Discrete Numerical	float64	0.050000	4	1.000700
174	V120	Vesta Engineered	Discrete Numerical	float64	0.050000	4	1.000900
175	V121	Vesta Engineered	Discrete Numerical	float64	0.050000	4	1.004300
176	V122	Vesta Engineered	Discrete Numerical	float64	0.050000	4	1.001800
177	V123	Vesta Engineered	Discrete Numerical	float64	0.050000	14	1.031100
178	V124	Vesta Engineered	Discrete Numerical	float64	0.050000	14	1.093000
179	V125	Vesta Engineered	Discrete Numerical	float64	0.050000	14	1.050400
180	V126	Vesta Engineered	Numerical	float64	0.050000	10299	129.979400
181	V127	Vesta Engineered	Numerical	float64	0.050000	24414	336.611600
182	V128	Vesta Engineered	Numerical	float64	0.050000	14507	204.094000
183	V129	Vesta Engineered	Numerical	float64	0.050000	1968	8.768900
184	V130	Vesta Engineered	Numerical	float64	0.050000	12332	92.165800
185	V131	Vesta Engineered	Numerical	float64	0.050000	4444	31.133300

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
186	V132	Vesta Engineered	Numerical	float64	0.050000	6560	103.513200
187	V133	Vesta Engineered	Numerical	float64	0.050000	9949	204.889200
188	V134	Vesta Engineered	Numerical	float64	0.050000	8178	145.972300
189	V135	Vesta Engineered	Numerical	float64	0.050000	3724	17.250100
190	V136	Vesta Engineered	Numerical	float64	0.050000	4852	38.821200
191	V137	Vesta Engineered	Numerical	float64	0.050000	4252	26.365100
192	V138	Vesta Engineered	Numerical	float64	86.120000	23	0.036400
193	V139	Vesta Engineered	Numerical	float64	86.120000	34	1.073900
194	V140	Vesta Engineered	Numerical	float64	86.120000	34	1.125300
195	V141	Vesta Engineered	Discrete Numerical	float64	86.120000	6	0.037700
196	V142	Vesta Engineered	Discrete Numerical	float64	86.120000	10	0.048600
197	V143	Vesta Engineered	Numerical	float64	86.120000	870	8.397000
198	V144	Vesta Engineered	Numerical	float64	86.120000	63	3.708500
199	V145	Vesta Engineered	Numerical	float64	86.120000	260	22.112900
200	V146	Vesta Engineered	Numerical	float64	86.120000	25	0.156300
201	V147	Vesta Engineered	Numerical	float64	86.120000	27	0.168600
202	V148	Vesta Engineered	Numerical	float64	86.120000	21	0.765000
203	V149	Vesta Engineered	Numerical	float64	86.120000	21	0.775300
204	V150	Vesta Engineered	Numerical	float64	86.120000	1996	277.598000

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
205	V151	Vesta Engineered	Numerical	float64	86.120000	56	6.460200
206	V152	Vesta Engineered	Numerical	float64	86.120000	39	9.432700
207	V153	Vesta Engineered	Discrete Numerical	float64	86.120000	19	0.753200
208	V154	Vesta Engineered	Discrete Numerical	float64	86.120000	19	0.757400
209	V155	Vesta Engineered	Numerical	float64	86.120000	25	0.767400
210	V156	Vesta Engineered	Numerical	float64	86.120000	25	0.777500
211	V157	Vesta Engineered	Numerical	float64	86.120000	25	0.817600
212	V158	Vesta Engineered	Numerical	float64	86.120000	25	0.833500
213	V159	Vesta Engineered	Numerical	float64	86.120000	6663	2719.299800
214	V160	Vesta Engineered	Numerical	float64	86.120000	9621	47453.181200
215	V161	Vesta Engineered	Numerical	float64	86.120000	79	4.843900
216	V162	Vesta Engineered	Numerical	float64	86.120000	185	6.594700
217	V163	Vesta Engineered	Numerical	float64	86.120000	106	5.506000
218	V164	Vesta Engineered	Numerical	float64	86.120000	1978	877.888900
219	V165	Vesta Engineered	Numerical	float64	86.120000	2547	2239.912200
220	V166	Vesta Engineered	Numerical	float64	86.120000	987	359.469400
221	V167	Vesta Engineered	Numerical	float64	76.360000	873	3.929500
222	V168	Vesta Engineered	Numerical	float64	76.360000	965	5.859500
223	V169	Vesta Engineered	Numerical	float64	76.320000	20	0.168100

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
224	V170	Vesta Engineered	Numerical	float64	76.320000	49	1.436800
225	V171	Vesta Engineered	Numerical	float64	76.320000	62	1.690900
226	V172	Vesta Engineered	Numerical	float64	76.360000	32	0.132000
227	V173	Vesta Engineered	Discrete Numerical	float64	76.360000	8	0.055200
228	V174	Vesta Engineered	Discrete Numerical	float64	76.320000	9	0.127800
229	V175	Vesta Engineered	Discrete Numerical	float64	76.320000	15	0.212600
230	V176	Vesta Engineered	Numerical	float64	76.360000	49	1.377000
231	V177	Vesta Engineered	Numerical	float64	76.360000	862	3.529200
232	V178	Vesta Engineered	Numerical	float64	76.360000	1236	6.649300
233	V179	Vesta Engineered	Numerical	float64	76.360000	921	4.869000
234	V180	Vesta Engineered	Numerical	float64	76.320000	84	0.924100
235	V181	Vesta Engineered	Numerical	float64	76.360000	25	0.253300
236	V182	Vesta Engineered	Numerical	float64	76.360000	84	0.856200
237	V183	Vesta Engineered	Numerical	float64	76.360000	42	0.484700
238	V184	Vesta Engineered	Discrete Numerical	float64	76.320000	17	0.132400
239	V185	Vesta Engineered	Numerical	float64	76.320000	32	0.173900
240	V186	Vesta Engineered	Numerical	float64	76.360000	39	1.148800
241	V187	Vesta Engineered	Numerical	float64	76.360000	215	1.843800
242	V188	Vesta Engineered	Numerical	float64	76.320000	31	1.014800

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
243	V189	Vesta Engineered	Numerical	float64	76.320000	31	1.038300
244	V190	Vesta Engineered	Numerical	float64	76.360000	43	1.213100
245	V191	Vesta Engineered	Numerical	float64	76.360000	22	1.059000
246	V192	Vesta Engineered	Numerical	float64	76.360000	45	1.237400
247	V193	Vesta Engineered	Numerical	float64	76.360000	38	1.149800
248	V194	Vesta Engineered	Discrete Numerical	float64	76.320000	8	0.945800
249	V195	Vesta Engineered	Discrete Numerical	float64	76.320000	17	0.954200
250	V196	Vesta Engineered	Numerical	float64	76.360000	39	1.084600
251	V197	Vesta Engineered	Discrete Numerical	float64	76.320000	15	0.949400
252	V198	Vesta Engineered	Numerical	float64	76.320000	22	0.962000
253	V199	Vesta Engineered	Numerical	float64	76.360000	46	1.270700
254	V200	Vesta Engineered	Numerical	float64	76.320000	46	1.120000
255	V201	Vesta Engineered	Numerical	float64	76.320000	56	1.159100
256	V202	Vesta Engineered	Numerical	float64	76.360000	10970	444.147100
257	V203	Vesta Engineered	Numerical	float64	76.360000	14951	1078.327500
258	V204	Vesta Engineered	Numerical	float64	76.360000	12858	686.956900
259	V205	Vesta Engineered	Numerical	float64	76.360000	2240	18.060400
260	V206	Vesta Engineered	Numerical	float64	76.360000	1780	6.189400
261	V207	Vesta Engineered	Numerical	float64	76.360000	3246	72.284100

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
262	V208	Vesta Engineered	Numerical	float64	76.320000	2552	8.888200
263	V209	Vesta Engineered	Numerical	float64	76.320000	3451	35.001900
264	V210	Vesta Engineered	Numerical	float64	76.320000	2836	14.391800
265	V211	Vesta Engineered	Numerical	float64	76.360000	7624	385.137000
266	V212	Vesta Engineered	Numerical	float64	76.360000	8868	765.988300
267	V213	Vesta Engineered	Numerical	float64	76.360000	8317	536.302800
268	V214	Vesta Engineered	Numerical	float64	76.360000	2282	38.437500
269	V215	Vesta Engineered	Numerical	float64	76.360000	2747	133.208200
270	V216	Vesta Engineered	Numerical	float64	76.360000	2532	71.107100
271	V217	Vesta Engineered	Numerical	float64	77.910000	304	1.054600
272	V218	Vesta Engineered	Numerical	float64	77.910000	401	1.725800
273	V219	Vesta Engineered	Numerical	float64	77.910000	379	1.367100
274	V220	Vesta Engineered	Numerical	float64	76.050000	26	0.169500
275	V221	Vesta Engineered	Numerical	float64	76.050000	77	1.274900
276	V222	Vesta Engineered	Numerical	float64	76.050000	76	1.359000
277	V223	Vesta Engineered	Discrete Numerical	float64	77.910000	17	0.093200
278	V224	Vesta Engineered	Numerical	float64	77.910000	79	0.394700
279	V225	Vesta Engineered	Numerical	float64	77.910000	35	0.191800
280	V226	Vesta Engineered	Numerical	float64	77.910000	81	0.247600

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
281	V227	Vesta Engineered	Numerical	float64	76.050000	50	0.148000
282	V228	Vesta Engineered	Numerical	float64	77.910000	55	1.351900
283	V229	Vesta Engineered	Numerical	float64	77.910000	91	1.638700
284	V230	Vesta Engineered	Numerical	float64	77.910000	66	1.465500
285	V231	Vesta Engineered	Numerical	float64	77.910000	294	0.765500
286	V232	Vesta Engineered	Numerical	float64	77.910000	338	1.005600
287	V233	Vesta Engineered	Numerical	float64	77.910000	333	0.908400
288	V234	Vesta Engineered	Numerical	float64	76.050000	122	2.092800
289	V235	Vesta Engineered	Numerical	float64	77.910000	24	0.184100
290	V236	Vesta Engineered	Numerical	float64	77.910000	46	0.307700
291	V237	Vesta Engineered	Numerical	float64	77.910000	40	0.253500
292	V238	Vesta Engineered	Numerical	float64	76.050000	24	0.128000
293	V239	Vesta Engineered	Numerical	float64	76.050000	24	0.137000
294	V240	Vesta Engineered	Discrete Numerical	float64	77.910000	6	1.001000
295	V241	Vesta Engineered	Discrete Numerical	float64	77.910000	5	1.000200
296	V242	Vesta Engineered	Numerical	float64	77.910000	21	1.113500
297	V243	Vesta Engineered	Numerical	float64	77.910000	43	1.178400
298	V244	Vesta Engineered	Numerical	float64	77.910000	23	1.118600
299	V245	Vesta Engineered	Numerical	float64	76.050000	58	0.876600

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
300	V246	Vesta Engineered	Numerical	float64	77.910000	46	1.183700
301	V247	Vesta Engineered	Discrete Numerical	float64	77.910000	19	1.025400
302	V248	Vesta Engineered	Numerical	float64	77.910000	23	1.069500
303	V249	Vesta Engineered	Numerical	float64	77.910000	23	1.042900
304	V250	Vesta Engineered	Discrete Numerical	float64	76.050000	19	0.788600
305	V251	Vesta Engineered	Discrete Numerical	float64	76.050000	19	0.794200
306	V252	Vesta Engineered	Numerical	float64	77.910000	25	1.032000
307	V253	Vesta Engineered	Numerical	float64	77.910000	66	1.162200
308	V254	Vesta Engineered	Numerical	float64	77.910000	45	1.075100
309	V255	Vesta Engineered	Numerical	float64	76.050000	46	0.806700
310	V256	Vesta Engineered	Numerical	float64	76.050000	48	0.815000
311	V257	Vesta Engineered	Numerical	float64	77.910000	49	1.251000
312	V258	Vesta Engineered	Numerical	float64	77.910000	67	1.343500
313	V259	Vesta Engineered	Numerical	float64	76.050000	68	0.967800
314	V260	Vesta Engineered	Discrete Numerical	float64	77.910000	9	0.964400
315	V261	Vesta Engineered	Numerical	float64	77.910000	41	1.107200
316	V262	Vesta Engineered	Numerical	float64	77.910000	21	1.013300
317	V263	Vesta Engineered	Numerical	float64	77.910000	10422	117.390700
318	V264	Vesta Engineered	Numerical	float64	77.910000	13358	201.657600

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
319	V265	Vesta Engineered	Numerical	float64	77.910000	11757	153.520500
320	V266	Vesta Engineered	Numerical	float64	77.910000	2178	9.167800
321	V267	Vesta Engineered	Numerical	float64	77.910000	3616	36.525100
322	V268	Vesta Engineered	Numerical	float64	77.910000	2756	18.813400
323	V269	Vesta Engineered	Numerical	float64	77.910000	151	5.997500
324	V270	Vesta Engineered	Numerical	float64	76.050000	2340	7.712800
325	V271	Vesta Engineered	Numerical	float64	76.050000	2787	9.445100
326	V272	Vesta Engineered	Numerical	float64	76.050000	2507	8.464600
327	V273	Vesta Engineered	Numerical	float64	77.910000	7177	73.825500
328	V274	Vesta Engineered	Numerical	float64	77.910000	8315	107.151600
329	V275	Vesta Engineered	Numerical	float64	77.910000	7776	88.899900
330	V276	Vesta Engineered	Numerical	float64	77.910000	2263	31.797300
331	V277	Vesta Engineered	Numerical	float64	77.910000	2540	51.956600
332	V278	Vesta Engineered	Numerical	float64	77.910000	2398	42.328200
333	V279	Vesta Engineered	Numerical	float64	0.000000	881	1.123100
334	V280	Vesta Engineered	Numerical	float64	0.000000	975	1.967100
335	V281	Vesta Engineered	Numerical	float64	0.210000	23	0.087800
336	V282	Vesta Engineered	Numerical	float64	0.210000	33	0.817200
337	V283	Vesta Engineered	Numerical	float64	0.210000	62	0.991100

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
338	V284	Vesta Engineered	Discrete Numerical	float64	0.000000	13	0.088500
339	V285	Vesta Engineered	Numerical	float64	0.000000	96	1.167700
340	V286	Vesta Engineered	Discrete Numerical	float64	0.000000	9	0.031500
341	V287	Vesta Engineered	Numerical	float64	0.000000	32	0.358600
342	V288	Vesta Engineered	Discrete Numerical	float64	0.210000	11	0.184300
343	V289	Vesta Engineered	Discrete Numerical	float64	0.210000	13	0.236000
344	V290	Vesta Engineered	Numerical	float64	0.000000	58	1.103000
345	V291	Vesta Engineered	Numerical	float64	0.000000	219	1.659800
346	V292	Vesta Engineered	Numerical	float64	0.000000	173	1.239900
347	V293	Vesta Engineered	Numerical	float64	0.000000	870	0.942600
348	V294	Vesta Engineered	Numerical	float64	0.000000	1286	2.313900
349	V295	Vesta Engineered	Numerical	float64	0.000000	928	1.433400
350	V296	Vesta Engineered	Numerical	float64	0.210000	94	0.328900
351	V297	Vesta Engineered	Discrete Numerical	float64	0.000000	13	0.089000
352	V298	Vesta Engineered	Numerical	float64	0.000000	94	0.298800
353	V299	Vesta Engineered	Numerical	float64	0.000000	50	0.171700
354	V300	Vesta Engineered	Discrete Numerical	float64	0.210000	12	0.045500
355	V301	Vesta Engineered	Discrete Numerical	float64	0.210000	14	0.052000
356	V302	Vesta Engineered	Discrete Numerical	float64	0.000000	17	0.251800

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
357	V303	Vesta Engineered	Numerical	float64	0.000000	21	0.283100
358	V304	Vesta Engineered	Discrete Numerical	float64	0.000000	17	0.264200
359	V305	Vesta Engineered	Binary	float64	0.000000	2	nan
360	V306	Vesta Engineered	Numerical	float64	0.000000	16210	139.748700
361	V307	Vesta Engineered	Numerical	float64	0.000000	37367	408.682400
362	V308	Vesta Engineered	Numerical	float64	0.000000	23064	230.413200
363	V309	Vesta Engineered	Numerical	float64	0.000000	4236	10.996000
364	V310	Vesta Engineered	Numerical	float64	0.000000	19136	118.195700
365	V311	Vesta Engineered	Numerical	float64	0.000000	3098	4.202200
366	V312	Vesta Engineered	Numerical	float64	0.000000	8068	39.173900
367	V313	Vesta Engineered	Numerical	float64	0.210000	5529	21.351500
368	V314	Vesta Engineered	Numerical	float64	0.210000	11377	43.319200
369	V315	Vesta Engineered	Numerical	float64	0.210000	6973	26.807000
370	V316	Vesta Engineered	Numerical	float64	0.000000	9814	109.818500
371	V317	Vesta Engineered	Numerical	float64	0.000000	15184	247.606700
372	V318	Vesta Engineered	Numerical	float64	0.000000	12309	162.153400
373	V319	Vesta Engineered	Numerical	float64	0.000000	4799	18.372500
374	V320	Vesta Engineered	Numerical	float64	0.000000	6439	42.073100
375	V321	Vesta Engineered	Numerical	float64	0.000000	5560	28.326600

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
376	V322	Vesta Engineered	Numerical	float64	86.050000	881	6.220300
377	V323	Vesta Engineered	Numerical	float64	86.050000	1411	13.103800
378	V324	Vesta Engineered	Numerical	float64	86.050000	976	9.184600
379	V325	Vesta Engineered	Discrete Numerical	float64	86.050000	13	0.058500
380	V326	Vesta Engineered	Numerical	float64	86.050000	45	0.851000
381	V327	Vesta Engineered	Discrete Numerical	float64	86.050000	19	0.296600
382	V328	Vesta Engineered	Discrete Numerical	float64	86.050000	16	0.336800
383	V329	Vesta Engineered	Numerical	float64	86.050000	100	1.312800
384	V330	Vesta Engineered	Numerical	float64	86.050000	56	0.775900
385	V331	Vesta Engineered	Numerical	float64	86.050000	1758	721.741900
386	V332	Vesta Engineered	Numerical	float64	86.050000	2453	1375.783600
387	V333	Vesta Engineered	Numerical	float64	86.050000	1971	1014.622800
388	V334	Vesta Engineered	Numerical	float64	86.050000	143	9.807000
389	V335	Vesta Engineered	Numerical	float64	86.050000	672	59.164500
390	V336	Vesta Engineered	Numerical	float64	86.050000	356	28.530900
391	V337	Vesta Engineered	Numerical	float64	86.050000	254	55.352400
392	V338	Vesta Engineered	Numerical	float64	86.050000	380	151.160500
393	V339	Vesta Engineered	Numerical	float64	86.050000	334	100.700900
394	id_01	Identity	Numerical	float64	75.580000	77	-10.170500
395	id_02	Identity	Numerical	float64	76.150000	115655	174716.584700

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
396	id_03	Identity	Numerical	float64	88.770000	24	0.060200
397	id_04	Identity	Discrete Numerical	float64	88.770000	15	-0.058900
398	id_05	Identity	Numerical	float64	76.820000	93	1.615600
399	id_06	Identity	Numerical	float64	76.820000	101	-6.698700
400	id_07	Identity	Numerical	float64	99.130000	84	13.285400
401	id_08	Identity	Numerical	float64	99.130000	94	-38.600400
402	id_09	Identity	Numerical	float64	87.310000	46	0.091000
403	id_10	Identity	Numerical	float64	87.310000	62	-0.301100
404	id_11	Identity	Numerical	float64	76.130000	365	99.745300
405	id_12	Identity	Binary	object	75.580000	2	nan
406	id_13	Identity	Numerical	float64	78.440000	54	48.053100
407	id_14	Identity	Numerical	float64	86.450000	25	-344.507100
408	id_15	Identity	Categorical	object	76.130000	3	nan
409	id_16	Identity	Binary	object	78.100000	2	nan
410	id_17	Identity	Numerical	float64	76.400000	104	189.451400
411	id_18	Identity	Discrete Numerical	float64	92.360000	18	14.237300
412	id_19	Identity	Numerical	float64	76.410000	522	353.128200
413	id_20	Identity	Numerical	float64	76.420000	394	403.882700
414	id_21	Identity	Numerical	float64	99.130000	490	368.269800
415	id_22	Identity	Numerical	float64	99.120000	25	16.002700
416	id_23	Identity	Categorical	object	99.120000	3	nan
417	id_24	Identity	Discrete Numerical	float64	99.200000	12	12.800900
418	id_25	Identity	Numerical	float64	99.130000	341	329.608900
419	id_26	Identity	Numerical	float64	99.130000	95	149.070300

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
420	id_27	Identity	Binary	object	99.120000	2	nan
421	id_28	Identity	Binary	object	76.130000	2	nan
422	id_29	Identity	Binary	object	76.130000	2	nan
423	id_30	Identity	Categorical	object	86.870000	75	nan
424	id_31	Identity	Categorical	object	76.250000	130	nan
425	id_32	Identity	Discrete Numerical	float64	86.860000	4	26.508600
426	id_33	Identity	Categorical	object	87.590000	260	nan

	feature	family	logical_type	dtype	missing_pct	n_unique	mean
427	id_34	Identity	Categorical	object	86.820000	4	nan
428	id_35	Identity	Binary	object	76.130000	2	nan
429	id_36	Identity	Binary	object	76.130000	2	nan
430	id_37	Identity	Binary	object	76.130000	2	nan
431	id_38	Identity	Binary	object	76.130000	2	nan
432	DeviceType	Device	Binary	object	76.160000	2	nan
433	DeviceInfo	Device	Categorical	object	79.910000	1786	nan

```
In [7]: # --- Binary features: histogram distributions only ---
binary_cols = feature_profile.loc[
    feature_profile["logical_type"] == "Binary", "feature"
].tolist()

print(f"Binary features: {len(binary_cols)}")

if binary_cols:
    ncols = 5
    nrows = int(np.ceil(len(binary_cols) / ncols))
    fig, axes = plt.subplots(nrows, ncols, figsize=(3.2 * ncols, 2.4 * nrows))
    axes = np.atleast_1d(axes).flatten()

    for i, col in enumerate(binary_cols):
        ax = axes[i]
        s = df_train[col].dropna()
        vc = s.value_counts().sort_index()
        ax.bar([str(x) for x in vc.index], vc.values, color="steelblue", edgecolor=
ax.set_title(col, fontsize=9)
```

```

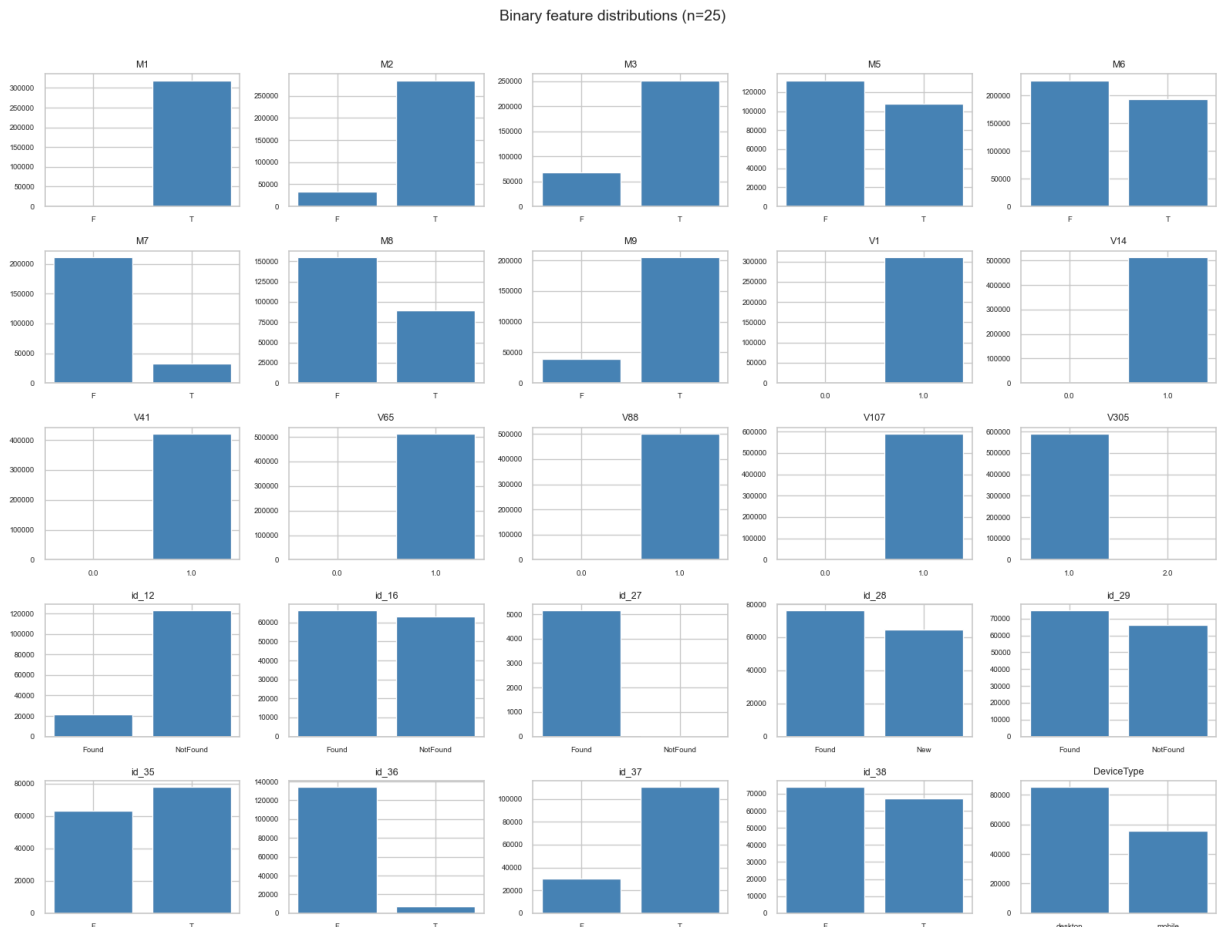
ax.tick_params(axis="x", labels=7)
ax.tick_params(axis="y", labels=7)

for j in range(len(binary_cols), len(axes)):
    axes[j].set_visible(False)

plt.suptitle(f"Binary feature distributions (n={len(binary_cols)})", fontsize=1
plt.tight_layout()
plt.show()
else:
    print("No binary columns detected.")

```

Binary features: 25



The profile above covers **all 434 columns**. The manual data dictionary below (B1–B4) describes feature **families** in domain language. Detailed plots in Section D focus on a **representative subset** chosen from this profile (see Selected Features table before Section C).

```

In [8]: # B0.5 – Optional YData Profiling (requires: pip install ydata-profiling --user --
import time
from pathlib import Path

try:
    from ydata_profiling import ProfileReport

    PROFILE_SAMPLE = 50_000

```

```

df_ydata = df_train.sample(min(PROFILE_SAMPLE, len(df_train)), random_state=42)
print(f"YData sample: {df_ydata.shape}")

t0 = time.perf_counter()
report = ProfileReport(
    df_ydata,
    title="IEEE-CIS Fraud – YData Profile",
    explorative=False,
    minimal=False,
    interactions=None,
)
ydata_path = Path("ydata_profile_report.html")
report.to_file(str(ydata_path))
elapsed = time.perf_counter() - t0
size_mb = ydata_path.stat().st_size / (1024 * 1024)
print(f"Saved: {ydata_path.resolve()} ({size_mb:.1f} MB, {elapsed:.0f}s)")
display(HTML(f'<a href="file:/// {ydata_path.resolve().as_posix()}" target="_bla
except ImportError:
    print("Optional – not installed. Run: python generate_ydata_profile.py")
    print("Or: pip install ydata-profiling --user --no-deps")

```

YData sample: (50000, 434)

```

100%|██████████| 434/434 [00:09<00:00, 47.82it/s]9<00:00, 14.98it/s, Describe variable: DeviceInfo]
Summarize dataset: 100%|██████████| 444/444 [08:29<00:00, 1.15s/it, Completed]
Generate report structure: 100%|██████████| 1/1 [01:25<00:00, 85.08s/it]
Render HTML: 100%|██████████| 1/1 [00:08<00:00, 8.18s/it]
Export report to file: 100%|██████████| 1/1 [00:00<00:00, 3.29it/s]
Saved: P:\langara\term 4\ana 4830\assignment 1\ydata_profile_report.html (62.9 MB, 605s)

```

[Open ydata_profile_report.html](#)

B0.5 — B0 Custom vs YData Profiling

Both profiles were generated and compared (`python compare_b0_profiles.py`). See `b0_comparison_report.md` for full metrics.

Metric	B0 custom	YData Profiling
Dataset	Full 590,540 rows	Sample 50,000 rows
Time	~25 s	~10 min
HTML size	~107 KB	~65 MB
Domain family + logical_type	Yes	No
Correlations & auto-alerts	No	Yes

Recommendation for A1: use **B0 custom** as the primary deliverable (fast, full dataset, connects to Section D and A2 Chi-square). Use **YData** as an optional interactive appendix.

Open:

- `feature_profile_434.html` — scroll + Ctrl+F
- `ydata_profile_report.html` — interactive (if generated)

```
In [9]: # Quick Links to both profile exports
from pathlib import Path
from IPython.display import display, HTML

for name in ["feature_profile_434.html", "ydata_profile_report.html"]:
    p = Path(name).resolve()
    if p.exists():
        size = p.stat().st_size / (1024 * 1024)
        print(f"{name}: {size:.2f} MB")
        display(HTML(f'<a href="file:///{' + p.as_posix() + '" target="_blank">Open {name}
    else:
        print(f"{name}: not found (run generate script or B0.5 cell above)"))
```

feature_profile_434.html: 0.10 MB

[Open feature_profile_434.html](#)

ydata_profile_report.html: 62.91 MB

[Open ydata_profile_report.html](#)

B1 - Key Operational Identifiers & Target Variable

Feature Name	Data Type (Logical)	Storage Type (Python)	Measurement Scale	Operational Description & Domain Logic
TransactionID	Categorical	Integer	Nominal (Identifier)	Unique tracking key for each specific commercial transaction. Used to join the transaction and identity tables.
isFraud	Categorical	Binary / Boolean	Nominal (Target)	The target variable to be predicted. Indicates whether the transaction was fraudulent (1) or legitimate (0).

B2 - Transaction Table Features (`train_transaction.csv`)

Feature Group	Data Type (Logical)	Storage Type (Python)	Measurement Scale	Operational Description & Domain Logic
TransactionDT	Numerical	Float	Continuous	Timedelta proxy representing the seconds elapsed from a specific, non-disclosed reference datetime (it is not an actual timestamp).

Feature Group	Data Type (Logical)	Storage Type (Python)	Measurement Scale	Operational Description & Domain Logic
TransactionAmt	Numerical	Float	Continuous	The payment amount of the transaction expressed in USD (or local currency equivalent).
ProductCD	Categorical	String / Object	Nominal	Product code or category designation for each transaction (e.g., W, H, C, R).
card1 - card6	Categorical	Integer / String	Nominal	Payment card information such as card type (credit/debit), card network (Visa, Mastercard), issue bank ID, and country of origin.
addr1, addr2	Categorical	Float / Integer	Nominal	Geographic address codes. addr1 typically represents billing region or zip code, and addr2 represents billing country.
P_emaildomain R_emaildomain	Categorical	String / Object	Nominal	Purchaser (P) and Recipient (R) email provider domains (e.g., gmail.com, yahoo.com).
M1 - M9	Categorical	String / Object	Nominal	Match criteria flags (e.g., T/F or M0-M2) evaluating string/logical matches between names, billing addresses, and cards on file.
C1 - C14	Numerical	Integer / Float	Discrete	Counting metrics calculating frequency behaviors, such as how many times a card, email, or device has been seen associated with an account.
D1 - D15	Numerical	Float	Continuous	Timedeltas measuring time gaps, such as days between the current transaction and previous activity or registration history.
dist1, dist2	Numerical	Float	Continuous	Distance metrics calculating physical intervals between attributes (such as distances between billing address, mailing address, zip codes, IP areas, or phone areas).

B3 - Vesta Engineered Behavioral Features (V-Block)

Feature Group	Data Type (Logical)	Storage Type (Python)	Measurement Scale	Operational Description & Domain Logic
V1 - V339	Numerical	Float	Continuous / Discrete	Anonymized features engineered by Vesta Corporation. These include ranking, counting, and relationship scores between payment attributes, proxy locations, and historical client behaviors.

B4 - Identity Table Features (`train_identity.csv`)

Feature Group	Data Type (Logical)	Storage Type (Python)	Measurement Scale	Operational Description & Domain Logic
DeviceType	Categorical	String / Object	Nominal	Operating environment of the client (e.g., mobile, desktop).
DeviceInfo	Categorical	String / Object	Nominal	Detailed hardware/software identity string (e.g., Windows, iOS, Samsung SM-G965F).
id_01 - id_11	Numerical	Float	Continuous / Discrete	Numerical identity markers containing network connections, screen dimensions, and behavioral signals.
id_12 - id_38	Categorical	String / Object	Nominal	Categorical identity indicators, such as browser types, proxy status, OS versions, and login credentials availability.

Section C: Data Quality Assessment

Before exploring statistical relationships and identifying candidate features for dimensionality reduction, it is necessary to assess the quality of the raw data. Poor data quality can distort distributions, inflate missingness, and bias feature-selection decisions. Therefore, this section evaluates duplicates, missingness mechanisms, inconsistent coding, and potential out-of-range values.

C1 - Duplicate Records Assessment

Before running descriptive profiling or missingness hypotheses, we must evaluate the foundational integrity of our dataset. This initial step quantifies structural row duplication across the merged dataset based on our primary operational key (**TransactionID**).

Identifying and dropping true duplicates prevents statistical distortion during variance calculations and univariate expansions.

```
In [10]: # Section C: Data Quality Assessment
# Subsection: Duplicate Records Assessment

# 1. Quantify exact duplicate rows based on the primary transaction identifier
duplicate_count = df_train.duplicated(subset=['TransactionID']).sum()
duplicate_percentage = (duplicate_count / len(df_train)) * 100

print("--- Duplicate Records Assessment ---")
print(f"Total duplicate records detected: {duplicate_count}")
print(f"Percentage of structural duplicates: {duplicate_percentage:.4f}%")

# 2. Programmatic handling based on findings
if duplicate_count > 0:
    df_train = df_train.drop_duplicates(subset=['TransactionID'])
    print("Duplicate rows dropped to secure baseline integrity.")
else:
    print("No data cleaning required. Every row represents a unique transactional e

--- Duplicate Records Assessment ---
Total duplicate records detected: 0
Percentage of structural duplicates: 0.0000%
No data cleaning required. Every row represents a unique transactional event.
```

C2 - Missingness Mechanism Hypothesis Testing (MCAR vs. MAR/MNAR)

This section executes an automated, comprehensive quality sweep across all categorical and numerical variables. Crucially, rather than relying on intuitive assumptions, we implement a formal statistical validation mechanism to classify missing data patterns into Missing Completely at Random (MCAR), Missing at Random (MAR), or Missing Not at Random (MNAR). By mapping the statistical independence between data absence and the target variable (`isFraud`) using Chi-Square (χ^2) Independence Tests, we establish a reproducible, data-driven framework to justify whether a high-missingness feature should be programmatically pruned or structurally retained with tailored mitigation protocols.

```
In [11]: # 1. Calculate the total number of columns in the dataset
total_columns = df_train.shape[1]

# 2. Count how many columns have at least one missing value
columns_with_nulls = df_train.isnull().any().sum()

# 3. Count how many columns are completely clean (zero missing values)
clean_columns = total_columns - columns_with_nulls

# 4. Print the analytical summary
print("=== DATASET COLUMN INTEGRITY SUMMARY ===")
print(f"Total features (columns) in dataset : {total_columns}")
```

```

print(f"Features WITH missing values      : {columns_with_nulls} ({(columns_with_
print(f"Features WITHOUT missing values   : {clean_columns} ({(clean_columns / to

```

```

=== DATASET COLUMN INTEGRITY SUMMARY ===
Total features (columns) in dataset : 434
Features WITH missing values        : 414 (95.39%)
Features WITHOUT missing values     : 20 (4.61%)

```

```

In [12]: import pandas as pd
import numpy as np
from scipy.stats import chi2_contingency

# 1. Display configuration for complete visibility in the report
pd.set_option('display.max_rows', None)

target_col = 'isFraud'
metadata_hypothesis = []

print("Running Chi-Square Independence Tests...")

# 2. Iterate through all columns to evaluate missingness mechanisms
for col in df_train.columns:
    if col == target_col or col == 'TransactionID':
        continue

    # Create missingness indicator (1 if null, 0 otherwise)
    is_null = df_train[col].isna().astype(int)
    total_nulls = is_null.sum()
    null_percentage = (total_nulls / len(df_train)) * 100

    # Only evaluate features that contain missing values (but are not 100% empty)
    if 0 < total_nulls < len(df_train):

        # Build 2x2 contingency table: Missingness Status vs Target Class (isFraud)
        contingency_table = pd.crosstab(is_null, df_train[target_col])

        try:
            # Execute Chi-Square Test of Independence
            chi2, p_value, dof, expected = chi2_contingency(contingency_table)

            # Classify missingness mechanism based on statistical significance (Alp
            if p_value < 0.05:
                mechanism = "MAR / MNAR (Associated with Fraud)"
            else:
                mechanism = "MCAR (Purely Random Missingness)"

            # Apply corporate pruning logic based on your criteria:
            # Drop only if it is MCAR AND null percentage is strictly greater than
            if p_value >= 0.05 and null_percentage > 70.0:
                final_decision = "DROP (MCAR & >70% Nulls)"
            else:
                final_decision = "KEEP"

        except Exception as e:
            # Handle edge cases with zero frequencies in extreme sub-categories
            chi2, p_value = np.nan, np.nan

```

```

        mechanism = "Unknown (Insufficient Data)"
        final_decision = "KEEP (Manual Review Required)"

    metadata_hypothesis.append({
        'Feature_Name': col,
        'Nulls_Percentage': round(null_percentage, 2),
        'Chi2_Statistic': round(chi2, 4) if not np.isnan(chi2) else np.nan,
        'p_value_raw': p_value, # Kept numeric for sorting purposes
        'Inferred_Mechanism': mechanism,
        'Final_Decision': final_decision
    })

# 3. Convert to DataFrame and sort by statistical significance (lowest p-values first)
df_hypothesis_matrix = pd.DataFrame(metadata_hypothesis)
df_hypothesis_matrix = df_hypothesis_matrix.sort_values(by='p_value_raw', ascending=True)

# Format p-value column for professional presentation (scientific notation for tiny values)
df_hypothesis_matrix['p-value'] = df_hypothesis_matrix['p_value_raw'].apply(
    lambda x: f"{x:.4e}" if x < 0.0001 else f"{x:.4f}"
)

# Drop the raw numeric p-value column and reorder for the final table display
df_hypothesis_matrix = df_hypothesis_matrix.drop(columns=['p_value_raw'])
df_hypothesis_matrix = df_hypothesis_matrix[['Feature_Name', 'Nulls_Percentage', 'Chi2_Statistic', 'p-value', 'Inferred_Mechanism', 'Final_Decision']]

# 4. Deploy the missingness hypothesis matrix output
print("\n=== C4 - MISSINGNESS MECHANISM HYPOTHESIS TESTING WITH PRUNING DECISIONS ===")
display(df_hypothesis_matrix)

```

Running Chi-Square Independence Tests...

=== C4 - MISSINGNESS MECHANISM HYPOTHESIS TESTING WITH PRUNING DECISIONS ===

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
0	DeviceInfo	79.91	6197.1894	0.0000e+00	MAR / MNAR (Associated with Fraud)	
1	DeviceType	76.16	10904.3351	0.0000e+00	MAR / MNAR (Associated with Fraud)	
2	id_38	76.13	10911.9575	0.0000e+00	MAR / MNAR (Associated with Fraud)	
3	id_37	76.13	10911.9575	0.0000e+00	MAR / MNAR (Associated with Fraud)	
4	id_36	76.13	10911.9575	0.0000e+00	MAR / MNAR (Associated with Fraud)	
5	id_35	76.13	10911.9575	0.0000e+00	MAR / MNAR (Associated with Fraud)	
6	V6	47.29	4605.0682	0.0000e+00	MAR / MNAR (Associated with Fraud)	
7	V5	47.29	4605.0682	0.0000e+00	MAR / MNAR (Associated with Fraud)	
8	V4	47.29	4605.0682	0.0000e+00	MAR / MNAR (Associated with Fraud)	
9	V3	47.29	4605.0682	0.0000e+00	MAR / MNAR (Associated with Fraud)	
10	V2	47.29	4605.0682	0.0000e+00	MAR / MNAR (Associated with Fraud)	
11	V1	47.29	4605.0682	0.0000e+00	MAR / MNAR (Associated with Fraud)	
12	M9	58.63	2876.9290	0.0000e+00	MAR / MNAR (Associated with Fraud)	
13	M6	28.68	8958.3672	0.0000e+00	MAR / MNAR (Associated with Fraud)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
14	M4	47.66	4291.5573	0.0000e+00	MAR / MNAR (Associated with Fraud)	
15	M3	45.91	4720.5786	0.0000e+00	MAR / MNAR (Associated with Fraud)	
16	M2	45.91	4720.5786	0.0000e+00	MAR / MNAR (Associated with Fraud)	
17	M1	45.91	4720.5786	0.0000e+00	MAR / MNAR (Associated with Fraud)	
18	D14	89.47	13502.6965	0.0000e+00	MAR / MNAR (Associated with Fraud)	
19	V192	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
20	V191	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
21	V190	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
22	V221	76.05	10023.6546	0.0000e+00	MAR / MNAR (Associated with Fraud)	
23	V220	76.05	10023.6546	0.0000e+00	MAR / MNAR (Associated with Fraud)	
24	V219	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
25	V218	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
26	V217	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
27	V216	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
28	V215	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
29	V214	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
30	V213	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
31	V212	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
32	V211	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
33	V210	76.32	10589.7884	0.0000e+00	MAR / MNAR (Associated with Fraud)	
34	V209	76.32	10589.7884	0.0000e+00	MAR / MNAR (Associated with Fraud)	
35	D13	89.51	11641.5597	0.0000e+00	MAR / MNAR (Associated with Fraud)	
36	D12	89.04	14617.2968	0.0000e+00	MAR / MNAR (Associated with Fraud)	
37	D11	47.29	4605.0682	0.0000e+00	MAR / MNAR (Associated with Fraud)	
38	D9	87.31	12263.9603	0.0000e+00	MAR / MNAR (Associated with Fraud)	
39	D8	87.31	12263.9603	0.0000e+00	MAR / MNAR (Associated with Fraud)	
40	V60	13.06	1448.9132	0.0000e+00	MAR / MNAR (Associated with Fraud)	
41	M8	58.63	2876.9290	0.0000e+00	MAR / MNAR (Associated with Fraud)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
42	M7	58.64	2876.2725	0.0000e+00	MAR / MNAR (Associated with Fraud)	
43	V61	13.06	1448.9132	0.0000e+00	MAR / MNAR (Associated with Fraud)	
44	V176	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
45	V59	13.06	1448.9132	0.0000e+00	MAR / MNAR (Associated with Fraud)	
46	V175	76.32	10589.7884	0.0000e+00	MAR / MNAR (Associated with Fraud)	
47	V174	76.32	10589.7884	0.0000e+00	MAR / MNAR (Associated with Fraud)	
48	V198	76.32	10589.7884	0.0000e+00	MAR / MNAR (Associated with Fraud)	
49	V205	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
50	V204	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
51	V203	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
52	V202	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
53	V201	76.32	10589.7884	0.0000e+00	MAR / MNAR (Associated with Fraud)	
54	V200	76.32	10589.7884	0.0000e+00	MAR / MNAR (Associated with Fraud)	
55	V199	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
56	V194	76.32	10589.7884	0.0000e+00	MAR / MNAR (Associated with Fraud)	
57	V197	76.32	10589.7884	0.0000e+00	MAR / MNAR (Associated with Fraud)	
58	V196	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
59	V195	76.32	10589.7884	0.0000e+00	MAR / MNAR (Associated with Fraud)	
60	V58	13.06	1448.9132	0.0000e+00	MAR / MNAR (Associated with Fraud)	
61	V193	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
62	V57	13.06	1448.9132	0.0000e+00	MAR / MNAR (Associated with Fraud)	
63	V56	13.06	1448.9132	0.0000e+00	MAR / MNAR (Associated with Fraud)	
64	V181	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
65	V55	13.06	1448.9132	0.0000e+00	MAR / MNAR (Associated with Fraud)	
66	V54	13.06	1448.9132	0.0000e+00	MAR / MNAR (Associated with Fraud)	
67	V53	13.06	1448.9132	0.0000e+00	MAR / MNAR (Associated with Fraud)	
68	id_10	87.31	12263.9603	0.0000e+00	MAR / MNAR (Associated with Fraud)	
69	id_09	87.31	12263.9603	0.0000e+00	MAR / MNAR (Associated with Fraud)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
70	id_06	76.82	10801.4285	0.0000e+00	MAR / MNAR (Associated with Fraud)	
71	id_05	76.82	10801.4285	0.0000e+00	MAR / MNAR (Associated with Fraud)	
72	id_04	88.77	11545.4976	0.0000e+00	MAR / MNAR (Associated with Fraud)	
73	id_03	88.77	11545.4976	0.0000e+00	MAR / MNAR (Associated with Fraud)	
74	id_02	76.15	10931.6999	0.0000e+00	MAR / MNAR (Associated with Fraud)	
75	id_01	75.58	10683.6359	0.0000e+00	MAR / MNAR (Associated with Fraud)	
76	V74	13.06	1448.9132	0.0000e+00	MAR / MNAR (Associated with Fraud)	
77	V73	13.06	1448.9132	0.0000e+00	MAR / MNAR (Associated with Fraud)	
78	V72	13.06	1448.9132	0.0000e+00	MAR / MNAR (Associated with Fraud)	
79	V71	13.06	1448.9132	0.0000e+00	MAR / MNAR (Associated with Fraud)	
80	V70	13.06	1448.9132	0.0000e+00	MAR / MNAR (Associated with Fraud)	
81	V69	13.06	1448.9132	0.0000e+00	MAR / MNAR (Associated with Fraud)	
82	V68	13.06	1448.9132	0.0000e+00	MAR / MNAR (Associated with Fraud)	
83	V67	13.06	1448.9132	0.0000e+00	MAR / MNAR (Associated with Fraud)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
84	V66	13.06	1448.9132	0.0000e+00	MAR / MNAR (Associated with Fraud)	
85	V226	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
86	V225	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
87	V224	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
88	V189	76.32	10589.7884	0.0000e+00	MAR / MNAR (Associated with Fraud)	
89	V188	76.32	10589.7884	0.0000e+00	MAR / MNAR (Associated with Fraud)	
90	V187	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
91	V186	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
92	V185	76.32	10589.7884	0.0000e+00	MAR / MNAR (Associated with Fraud)	
93	V184	76.32	10589.7884	0.0000e+00	MAR / MNAR (Associated with Fraud)	
94	V183	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
95	V182	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
96	V235	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
97	V180	76.32	10589.7884	0.0000e+00	MAR / MNAR (Associated with Fraud)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
98	V179	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
99	V178	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
100	V177	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
101	V65	13.06	1448.9132	0.0000e+00	MAR / MNAR (Associated with Fraud)	
102	V64	13.06	1448.9132	0.0000e+00	MAR / MNAR (Associated with Fraud)	
103	V63	13.06	1448.9132	0.0000e+00	MAR / MNAR (Associated with Fraud)	
104	V62	13.06	1448.9132	0.0000e+00	MAR / MNAR (Associated with Fraud)	
105	V169	76.32	10589.7884	0.0000e+00	MAR / MNAR (Associated with Fraud)	
106	V168	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
107	V167	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
108	V207	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
109	V206	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
110	V237	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
111	V236	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
112	V227	76.05	10023.6546	0.0000e+00	MAR / MNAR (Associated with Fraud)	
113	V234	76.05	10023.6546	0.0000e+00	MAR / MNAR (Associated with Fraud)	
114	V233	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
115	V232	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
116	V231	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
117	V230	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
118	V229	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
119	V228	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
120	V254	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
121	V271	76.05	10023.6546	0.0000e+00	MAR / MNAR (Associated with Fraud)	
122	V270	76.05	10023.6546	0.0000e+00	MAR / MNAR (Associated with Fraud)	
123	V255	76.05	10023.6546	0.0000e+00	MAR / MNAR (Associated with Fraud)	
124	V277	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
125	V278	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
126	V276	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
127	V275	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
128	id_31	76.25	10857.7683	0.0000e+00	MAR / MNAR (Associated with Fraud)	
129	V274	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
130	V273	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
131	V272	76.05	10023.6546	0.0000e+00	MAR / MNAR (Associated with Fraud)	
132	V239	76.05	10023.6546	0.0000e+00	MAR / MNAR (Associated with Fraud)	
133	V238	76.05	10023.6546	0.0000e+00	MAR / MNAR (Associated with Fraud)	
134	V269	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
135	V268	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
136	V267	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
137	V266	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
138	V265	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
139	V264	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
140	V263	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
141	V262	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
142	V261	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
143	V260	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
144	V259	76.05	10023.6546	0.0000e+00	MAR / MNAR (Associated with Fraud)	
145	V258	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
146	V257	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
147	V256	76.05	10023.6546	0.0000e+00	MAR / MNAR (Associated with Fraud)	
148	V223	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
149	V222	76.05	10023.6546	0.0000e+00	MAR / MNAR (Associated with Fraud)	
150	V253	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
151	V252	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
152	V251	76.05	10023.6546	0.0000e+00	MAR / MNAR (Associated with Fraud)	
153	V250	76.05	10023.6546	0.0000e+00	MAR / MNAR (Associated with Fraud)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
154	V249	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
155	V248	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
156	V247	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
157	V246	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
158	V245	76.05	10023.6546	0.0000e+00	MAR / MNAR (Associated with Fraud)	
159	V244	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
160	V243	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
161	V242	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
162	V241	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
163	V240	77.91	9230.0565	0.0000e+00	MAR / MNAR (Associated with Fraud)	
164	V173	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
165	V172	76.36	10515.9791	0.0000e+00	MAR / MNAR (Associated with Fraud)	
166	V171	76.32	10589.7884	0.0000e+00	MAR / MNAR (Associated with Fraud)	
167	V170	76.32	10589.7884	0.0000e+00	MAR / MNAR (Associated with Fraud)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
168	id_20	76.42	10552.2196	0.0000e+00	MAR / MNAR (Associated with Fraud)	
169	id_19	76.41	10566.3924	0.0000e+00	MAR / MNAR (Associated with Fraud)	
170	id_18	92.36	3303.9305	0.0000e+00	MAR / MNAR (Associated with Fraud)	
171	id_17	76.40	10585.0388	0.0000e+00	MAR / MNAR (Associated with Fraud)	
172	id_16	78.10	9279.7038	0.0000e+00	MAR / MNAR (Associated with Fraud)	
173	id_15	76.13	10911.9575	0.0000e+00	MAR / MNAR (Associated with Fraud)	
174	id_13	78.44	11152.5669	0.0000e+00	MAR / MNAR (Associated with Fraud)	
175	V11	47.29	4605.0682	0.0000e+00	MAR / MNAR (Associated with Fraud)	
176	V10	47.29	4605.0682	0.0000e+00	MAR / MNAR (Associated with Fraud)	
177	V9	47.29	4605.0682	0.0000e+00	MAR / MNAR (Associated with Fraud)	
178	V8	47.29	4605.0682	0.0000e+00	MAR / MNAR (Associated with Fraud)	
179	V7	47.29	4605.0682	0.0000e+00	MAR / MNAR (Associated with Fraud)	
180	V208	76.32	10589.7884	0.0000e+00	MAR / MNAR (Associated with Fraud)	
181	id_12	75.58	10683.6359	0.0000e+00	MAR / MNAR (Associated with Fraud)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
182	id_11	76.13	10899.3010	0.0000e+00	MAR / MNAR (Associated with Fraud)	
183	D2	47.55	1770.6581	0.0000e+00	MAR / MNAR (Associated with Fraud)	
184	addr2	11.13	15016.7235	0.0000e+00	MAR / MNAR (Associated with Fraud)	
185	R_emaildomain	76.75	11593.8586	0.0000e+00	MAR / MNAR (Associated with Fraud)	
186	dist2	93.63	4898.5409	0.0000e+00	MAR / MNAR (Associated with Fraud)	
187	dist1	59.65	2672.8051	0.0000e+00	MAR / MNAR (Associated with Fraud)	
188	D7	93.41	15972.2652	0.0000e+00	MAR / MNAR (Associated with Fraud)	
189	addr1	11.13	15016.7235	0.0000e+00	MAR / MNAR (Associated with Fraud)	
190	D6	87.61	12282.7028	0.0000e+00	MAR / MNAR (Associated with Fraud)	
191	id_28	76.13	10899.3010	0.0000e+00	MAR / MNAR (Associated with Fraud)	
192	id_29	76.13	10899.3010	0.0000e+00	MAR / MNAR (Associated with Fraud)	
193	D3	44.51	691.4686	2.1425e-152	MAR / MNAR (Associated with Fraud)	
194	D10	12.87	671.5105	4.6894e-148	MAR / MNAR (Associated with Fraud)	
195	V12	12.88	670.2679	8.7368e-148	MAR / MNAR (Associated with Fraud)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
196	V13	12.88	670.2679	8.7368e-148	MAR / MNAR (Associated with Fraud)	
197	V16	12.88	670.2679	8.7368e-148	MAR / MNAR (Associated with Fraud)	
198	V17	12.88	670.2679	8.7368e-148	MAR / MNAR (Associated with Fraud)	
199	V18	12.88	670.2679	8.7368e-148	MAR / MNAR (Associated with Fraud)	
200	V19	12.88	670.2679	8.7368e-148	MAR / MNAR (Associated with Fraud)	
201	V20	12.88	670.2679	8.7368e-148	MAR / MNAR (Associated with Fraud)	
202	V21	12.88	670.2679	8.7368e-148	MAR / MNAR (Associated with Fraud)	
203	V22	12.88	670.2679	8.7368e-148	MAR / MNAR (Associated with Fraud)	
204	V23	12.88	670.2679	8.7368e-148	MAR / MNAR (Associated with Fraud)	
205	V24	12.88	670.2679	8.7368e-148	MAR / MNAR (Associated with Fraud)	
206	V29	12.88	670.2679	8.7368e-148	MAR / MNAR (Associated with Fraud)	
207	V25	12.88	670.2679	8.7368e-148	MAR / MNAR (Associated with Fraud)	
208	V28	12.88	670.2679	8.7368e-148	MAR / MNAR (Associated with Fraud)	
209	V27	12.88	670.2679	8.7368e-148	MAR / MNAR (Associated with Fraud)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
210	V26	12.88	670.2679	8.7368e-148	MAR / MNAR (Associated with Fraud)	
211	V14	12.88	670.2679	8.7368e-148	MAR / MNAR (Associated with Fraud)	
212	V15	12.88	670.2679	8.7368e-148	MAR / MNAR (Associated with Fraud)	
213	V32	12.88	670.2679	8.7368e-148	MAR / MNAR (Associated with Fraud)	
214	V30	12.88	670.2679	8.7368e-148	MAR / MNAR (Associated with Fraud)	
215	V34	12.88	670.2679	8.7368e-148	MAR / MNAR (Associated with Fraud)	
216	V33	12.88	670.2679	8.7368e-148	MAR / MNAR (Associated with Fraud)	
217	V31	12.88	670.2679	8.7368e-148	MAR / MNAR (Associated with Fraud)	
218	D15	15.09	524.3462	4.7970e-116	MAR / MNAR (Associated with Fraud)	
219	V84	15.10	522.4848	1.2188e-115	MAR / MNAR (Associated with Fraud)	
220	V85	15.10	522.4848	1.2188e-115	MAR / MNAR (Associated with Fraud)	
221	V83	15.10	522.4848	1.2188e-115	MAR / MNAR (Associated with Fraud)	
222	V90	15.10	522.4848	1.2188e-115	MAR / MNAR (Associated with Fraud)	
223	V87	15.10	522.4848	1.2188e-115	MAR / MNAR (Associated with Fraud)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
224	V86	15.10	522.4848	1.2188e-115	MAR / MNAR (Associated with Fraud)	
225	V89	15.10	522.4848	1.2188e-115	MAR / MNAR (Associated with Fraud)	
226	V94	15.10	522.4848	1.2188e-115	MAR / MNAR (Associated with Fraud)	
227	V79	15.10	522.4848	1.2188e-115	MAR / MNAR (Associated with Fraud)	
228	V78	15.10	522.4848	1.2188e-115	MAR / MNAR (Associated with Fraud)	
229	V81	15.10	522.4848	1.2188e-115	MAR / MNAR (Associated with Fraud)	
230	V80	15.10	522.4848	1.2188e-115	MAR / MNAR (Associated with Fraud)	
231	V82	15.10	522.4848	1.2188e-115	MAR / MNAR (Associated with Fraud)	
232	V93	15.10	522.4848	1.2188e-115	MAR / MNAR (Associated with Fraud)	
233	V92	15.10	522.4848	1.2188e-115	MAR / MNAR (Associated with Fraud)	
234	V75	15.10	522.4848	1.2188e-115	MAR / MNAR (Associated with Fraud)	
235	V77	15.10	522.4848	1.2188e-115	MAR / MNAR (Associated with Fraud)	
236	V76	15.10	522.4848	1.2188e-115	MAR / MNAR (Associated with Fraud)	
237	V91	15.10	522.4848	1.2188e-115	MAR / MNAR (Associated with Fraud)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
238	V88	15.10	522.4848	1.2188e-115	MAR / MNAR (Associated with Fraud)	
239	id_24	99.20	348.5234	8.8858e-78	MAR / MNAR (Associated with Fraud)	
240	id_07	99.13	348.2441	1.0222e-77	MAR / MNAR (Associated with Fraud)	
241	id_08	99.13	348.2441	1.0222e-77	MAR / MNAR (Associated with Fraud)	
242	id_21	99.13	347.5792	1.4266e-77	MAR / MNAR (Associated with Fraud)	
243	id_22	99.12	345.9224	3.2742e-77	MAR / MNAR (Associated with Fraud)	
244	id_27	99.12	345.9224	3.2742e-77	MAR / MNAR (Associated with Fraud)	
245	id_23	99.12	345.9224	3.2742e-77	MAR / MNAR (Associated with Fraud)	
246	id_26	99.13	344.0876	8.2164e-77	MAR / MNAR (Associated with Fraud)	
247	id_25	99.13	326.7931	4.8002e-73	MAR / MNAR (Associated with Fraud)	
248	id_33	87.59	293.8777	7.1064e-66	MAR / MNAR (Associated with Fraud)	
249	V328	86.05	270.1669	1.0436e-60	MAR / MNAR (Associated with Fraud)	
250	V327	86.05	270.1669	1.0436e-60	MAR / MNAR (Associated with Fraud)	
251	V339	86.05	270.1669	1.0436e-60	MAR / MNAR (Associated with Fraud)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
252	V332	86.05	270.1669	1.0436e-60	MAR / MNAR (Associated with Fraud)	
253	V326	86.05	270.1669	1.0436e-60	MAR / MNAR (Associated with Fraud)	
254	V333	86.05	270.1669	1.0436e-60	MAR / MNAR (Associated with Fraud)	
255	V330	86.05	270.1669	1.0436e-60	MAR / MNAR (Associated with Fraud)	
256	V322	86.05	270.1669	1.0436e-60	MAR / MNAR (Associated with Fraud)	
257	V323	86.05	270.1669	1.0436e-60	MAR / MNAR (Associated with Fraud)	
258	V329	86.05	270.1669	1.0436e-60	MAR / MNAR (Associated with Fraud)	
259	V334	86.05	270.1669	1.0436e-60	MAR / MNAR (Associated with Fraud)	
260	V338	86.05	270.1669	1.0436e-60	MAR / MNAR (Associated with Fraud)	
261	V336	86.05	270.1669	1.0436e-60	MAR / MNAR (Associated with Fraud)	
262	V337	86.05	270.1669	1.0436e-60	MAR / MNAR (Associated with Fraud)	
263	V324	86.05	270.1669	1.0436e-60	MAR / MNAR (Associated with Fraud)	
264	V331	86.05	270.1669	1.0436e-60	MAR / MNAR (Associated with Fraud)	
265	V325	86.05	270.1669	1.0436e-60	MAR / MNAR (Associated with Fraud)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
266	V335	86.05	270.1669	1.0436e-60	MAR / MNAR (Associated with Fraud)	
267	id_14	86.45	258.9067	2.9706e-58	MAR / MNAR (Associated with Fraud)	
268	V159	86.12	257.2843	6.7065e-58	MAR / MNAR (Associated with Fraud)	
269	V160	86.12	257.2843	6.7065e-58	MAR / MNAR (Associated with Fraud)	
270	V144	86.12	257.2843	6.7065e-58	MAR / MNAR (Associated with Fraud)	
271	V145	86.12	257.2843	6.7065e-58	MAR / MNAR (Associated with Fraud)	
272	V143	86.12	257.2843	6.7065e-58	MAR / MNAR (Associated with Fraud)	
273	V166	86.12	257.2843	6.7065e-58	MAR / MNAR (Associated with Fraud)	
274	V165	86.12	257.2843	6.7065e-58	MAR / MNAR (Associated with Fraud)	
275	V150	86.12	257.2843	6.7065e-58	MAR / MNAR (Associated with Fraud)	
276	V151	86.12	257.2843	6.7065e-58	MAR / MNAR (Associated with Fraud)	
277	V152	86.12	257.2843	6.7065e-58	MAR / MNAR (Associated with Fraud)	
278	V164	86.12	257.2843	6.7065e-58	MAR / MNAR (Associated with Fraud)	
279	V161	86.12	256.7811	8.6333e-58	MAR / MNAR (Associated with Fraud)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
280	V162	86.12	256.7811	8.6333e-58	MAR / MNAR (Associated with Fraud)	
281	V142	86.12	256.7811	8.6333e-58	MAR / MNAR (Associated with Fraud)	
282	V163	86.12	256.7811	8.6333e-58	MAR / MNAR (Associated with Fraud)	
283	V158	86.12	256.7811	8.6333e-58	MAR / MNAR (Associated with Fraud)	
284	V149	86.12	256.7811	8.6333e-58	MAR / MNAR (Associated with Fraud)	
285	V148	86.12	256.7811	8.6333e-58	MAR / MNAR (Associated with Fraud)	
286	V147	86.12	256.7811	8.6333e-58	MAR / MNAR (Associated with Fraud)	
287	V146	86.12	256.7811	8.6333e-58	MAR / MNAR (Associated with Fraud)	
288	V153	86.12	256.7811	8.6333e-58	MAR / MNAR (Associated with Fraud)	
289	V156	86.12	256.7811	8.6333e-58	MAR / MNAR (Associated with Fraud)	
290	V155	86.12	256.7811	8.6333e-58	MAR / MNAR (Associated with Fraud)	
291	V154	86.12	256.7811	8.6333e-58	MAR / MNAR (Associated with Fraud)	
292	V138	86.12	256.7811	8.6333e-58	MAR / MNAR (Associated with Fraud)	
293	V139	86.12	256.7811	8.6333e-58	MAR / MNAR (Associated with Fraud)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
294	V140	86.12	256.7811	8.6333e-58	MAR / MNAR (Associated with Fraud)	
295	V141	86.12	256.7811	8.6333e-58	MAR / MNAR (Associated with Fraud)	
296	V157	86.12	256.7811	8.6333e-58	MAR / MNAR (Associated with Fraud)	
297	id_34	86.82	254.6241	2.5490e-57	MAR / MNAR (Associated with Fraud)	
298	id_32	86.86	237.8976	1.1301e-53	MAR / MNAR (Associated with Fraud)	
299	id_30	86.87	232.6362	1.5865e-52	MAR / MNAR (Associated with Fraud)	
300	D5	52.47	225.0263	7.2457e-51	MAR / MNAR (Associated with Fraud)	
301	M5	59.35	143.2470	5.1908e-33	MAR / MNAR (Associated with Fraud)	
302	P_emaildomain	15.99	98.8068	2.7837e-23	MAR / MNAR (Associated with Fraud)	
303	card2	1.51	40.6830	1.7904e-10	MAR / MNAR (Associated with Fraud)	
304	card5	0.72	25.6182	4.1610e-07	MAR / MNAR (Associated with Fraud)	
305	D4	28.60	8.3970	0.0038	MAR / MNAR (Associated with Fraud)	
306	V47	28.61	8.2469	0.0041	MAR / MNAR (Associated with Fraud)	
307	V46	28.61	8.2469	0.0041	MAR / MNAR (Associated with Fraud)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
308	V51	28.61	8.2469	0.0041	MAR / MNAR (Associated with Fraud)	
309	V36	28.61	8.2469	0.0041	MAR / MNAR (Associated with Fraud)	
310	V35	28.61	8.2469	0.0041	MAR / MNAR (Associated with Fraud)	
311	V52	28.61	8.2469	0.0041	MAR / MNAR (Associated with Fraud)	
312	V49	28.61	8.2469	0.0041	MAR / MNAR (Associated with Fraud)	
313	V50	28.61	8.2469	0.0041	MAR / MNAR (Associated with Fraud)	
314	V48	28.61	8.2469	0.0041	MAR / MNAR (Associated with Fraud)	
315	V38	28.61	8.2469	0.0041	MAR / MNAR (Associated with Fraud)	
316	V37	28.61	8.2469	0.0041	MAR / MNAR (Associated with Fraud)	
317	V44	28.61	8.2469	0.0041	MAR / MNAR (Associated with Fraud)	
318	V43	28.61	8.2469	0.0041	MAR / MNAR (Associated with Fraud)	
319	V42	28.61	8.2469	0.0041	MAR / MNAR (Associated with Fraud)	
320	V41	28.61	8.2469	0.0041	MAR / MNAR (Associated with Fraud)	
321	V40	28.61	8.2469	0.0041	MAR / MNAR (Associated with Fraud)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
322	V39	28.61	8.2469	0.0041	MAR / MNAR (Associated with Fraud)	
323	V45	28.61	8.2469	0.0041	MAR / MNAR (Associated with Fraud)	
324	card6	0.27	4.5232	0.0334	MAR / MNAR (Associated with Fraud)	
325	card3	0.27	4.4181	0.0356	MAR / MNAR (Associated with Fraud)	
326	card4	0.27	3.5235	0.0605	MCAR (Purely Random Missingness)	
327	V285	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
328	V312	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
329	V286	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
330	V290	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
331	V280	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
332	V321	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
333	V320	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
334	V318	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
335	V319	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
336	V317	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
337	V302	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
338	V310	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
339	V309	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
340	V303	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
341	V293	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
342	V297	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
343	V295	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
344	V292	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
345	V291	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
346	V279	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
347	V284	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
348	V299	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
349	V298	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
350	V311	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
351	V308	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
352	V305	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
353	V306	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
354	V304	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
355	V307	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
356	V294	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
357	V316	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
358	V287	0.00	2.8794	0.0897	MCAR (Purely Random Missingness)	
359	V96	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
360	V97	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
361	V135	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
362	V95	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
363	V125	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
364	V105	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
365	V104	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
366	V103	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
367	V102	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
368	V109	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
369	V108	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
370	V112	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
371	V116	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
372	V124	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
373	V123	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
374	V122	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
375	V121	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
376	V117	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
377	V118	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
378	V136	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
379	V129	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
380	V127	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
381	V128	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
382	V126	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
383	V137	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
384	V98	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
385	V99	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
386	V111	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
387	V110	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
388	V130	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
389	V131	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
390	V132	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
391	V133	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
392	V134	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
393	V113	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
394	V115	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
395	V114	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
396	V120	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
397	V119	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
398	V106	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
399	V107	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
400	V101	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
401	V100	0.05	2.8683	0.0903	MCAR (Purely Random Missingness)	
402	V296	0.21	0.0282	0.8667	MCAR (Purely Random Missingness)	
403	V301	0.21	0.0282	0.8667	MCAR (Purely Random Missingness)	
404	V283	0.21	0.0282	0.8667	MCAR (Purely Random Missingness)	
405	V282	0.21	0.0282	0.8667	MCAR (Purely Random Missingness)	

	Feature_Name	Nulls_Percentage	Chi2_Statistic	p-value	Inferred_Mechanism	Final_I
406	D1	0.21	0.0282	0.8667	MCAR (Purely Random Missingness)	
407	V281	0.21	0.0282	0.8667	MCAR (Purely Random Missingness)	
408	V313	0.21	0.0282	0.8667	MCAR (Purely Random Missingness)	
409	V314	0.21	0.0282	0.8667	MCAR (Purely Random Missingness)	
410	V288	0.21	0.0282	0.8667	MCAR (Purely Random Missingness)	
411	V289	0.21	0.0282	0.8667	MCAR (Purely Random Missingness)	
412	V300	0.21	0.0282	0.8667	MCAR (Purely Random Missingness)	
413	V315	0.21	0.0282	0.8667	MCAR (Purely Random Missingness)	

Missingness Analysis Results

The missingness analysis and the Chi-Square (χ^2) Independence Tests provided useful information about the relationship between missing values and the target variable across the dataset's features.

1. High-Missingness Features (Potential MAR/MNAR Mechanisms)

A common practice is to remove variables with very high percentages of missing values. However, the statistical tests suggest that this approach may not be appropriate for this dataset. Several features with high missingness, including `DeviceInfo` (79.91% missing), `DeviceType` (76.16% missing), and the identity variables (`id_35` to `id_38` , 76.13% missing), produced highly significant p-values (`p < 0.001`).

These results suggest that the missingness pattern is associated with the target variable (`isFraud`) and may follow a MAR (Missing at Random) or MNAR (Missing Not at Random) mechanism. This means that the absence of information may contain useful predictive

information. For this reason, these variables were retained for further analysis rather than removed based only on their missingness percentage.

2. Features with Likely MCAR Missingness

The analysis also identified features that appear to have missing values occurring independently of the target variable. For example, `card4` has only 0.27% missing values and produced a non-significant result ($p = 0.0605$).

Since the null hypothesis of independence could not be rejected, the missing values in this feature may be considered consistent with a Missing Completely at Random (MCAR) mechanism. In these cases, standard imputation methods can be applied with a lower risk of introducing bias.

Missingness Mitigation Strategy

Different imputation approaches were applied depending on the observed missingness patterns:

a. Handling MAR/MNAR Features

Categorical variables with missing values (such as the `M` variables and `P_emaildomain`) were assigned a new category called `'Missing'` . For numerical variables with substantial missingness, missing values were replaced with a constant value (`-999`) to preserve the information contained in the absence of data.

b. Missing Value Indicators

For numerical variables with low or moderate levels of missingness, missing values were replaced using the median. In addition, a binary indicator variable (for example, `Feature_isna`) was created to capture whether the original value was missing.

c. Handling MCAR Features

For variables whose missingness appeared independent of the target variable, standard imputation methods were applied. Categorical features were imputed using the mode, while numerical features were imputed using the median.

d. Final Data Cleaning

As a final step, 12 completely empty rows were removed. A final validation check confirmed that all remaining missing values had been handled, resulting in a fully populated dataset (`df_train_preprocessed`) for the subsequent EDA stages.

```
In [13]: # Create a fresh copy for cleaning from your original training data
df_train_preprocessed = df_train.copy()

print("Starting Automated and Dynamic Imputation Framework...")
```

```

# 1. Extract dynamic lists from df_hypothesis_matrix for the MCAR pool (Nulls <= 70
mcar_features = df_hypothesis_matrix[
    (df_hypothesis_matrix['Inferred_Mechanism'].str.contains("MCAR")) &
    (df_hypothesis_matrix['Nulls_Percentage'] > 0) &
    (df_hypothesis_matrix['Nulls_Percentage'] <= 70.0)
][['Feature_Name']].tolist()

print(f"--> Dynamically isolated {len(mcar_features)} MCAR features for localized i

# Iterate through ALL detected MCAR features dynamically
for col in mcar_features:
    if col in df_train_preprocessed.columns:
        # Determine data type automatically
        is_categorical = (df_train_preprocessed[col].dtype == 'object' or
                           pd.api.types.is_categorical_dtype(df_train_preprocessed[c

        if is_categorical:
            # Mode imputation for categorical MCAR fields
            col_mode = df_train_preprocessed[col].mode()[0]
            df_train_preprocessed[col] = df_train_preprocessed[col].fillna(col_mode)
        else:
            # Median imputation for continuous/discrete numerical MCAR fields
            col_median = df_train_preprocessed[col].median()
            df_train_preprocessed[col] = df_train_preprocessed[col].fillna(col_medi

# 2. Extract dynamic lists for High Missingness features (> 70% nulls)
high_missing_features = df_hypothesis_matrix[
    df_hypothesis_matrix['Nulls_Percentage'] > 70.0
][['Feature_Name']].tolist()

print(f"--> Dynamically isolated {len(high_missing_features)} features exceeding th

for col in high_missing_features:
    if col in df_train_preprocessed.columns:
        is_categorical = (df_train_preprocessed[col].dtype == 'object' or
                           pd.api.types.is_categorical_dtype(df_train_preprocessed[c

        if is_categorical:
            df_train_preprocessed[col] = df_train_preprocessed[col].fillna('Missing
        else:
            df_train_preprocessed[col] = df_train_preprocessed[col].fillna(-999)

print("\nAll dynamic pipelines executed successfully! Data structure is completely

```

Starting Automated and Dynamic Imputation Framework...

--> Dynamically isolated 56 MCAR features for localized imputation.

--> Dynamically isolated 208 features exceeding the 70% missingness threshold.

All dynamic pipelines executed successfully! Data structure is completely processed.

```

In [14]: # 1. Calculate the total number of columns in the dataset
total_columns = df_train_preprocessed.shape[1]

```

```

# 2. Count how many columns have at least one missing value
columns_with_nulls = df_train_preprocessed.isnull().any().sum()

# 3. Count how many columns are completely clean (zero missing values)
clean_columns = total_columns - columns_with_nulls

# 4. Print the analytical summary
print("=== DATASET COLUMN INTEGRITY SUMMARY ===")
print(f"Total features (columns) in dataset : {total_columns}")
print(f"Features WITH missing values      : {columns_with_nulls} ({(columns_with_")
print(f"Features WITHOUT missing values    : {clean_columns} ({(clean_columns / to

```

```

=== DATASET COLUMN INTEGRITY SUMMARY ===
Total features (columns) in dataset : 434
Features WITH missing values      : 150 (34.56%)
Features WITHOUT missing values    : 284 (65.44%)

```

```

In [15]: print("Starting Specialized Imputation for Remaining MAR/MNAR Features (0-70%)...")

# 1. Isolate MAR/MNAR features using a flexible string constraint ('MAR') to bypass
mar_mnar_remaining = df_hypothesis_matrix[
    (df_hypothesis_matrix['Nulls_Percentage'] > 0) &
    (df_hypothesis_matrix['Nulls_Percentage'] <= 70.0) &
    (df_hypothesis_matrix['Inferred_Mechanism'].str.contains("MAR", na=False))
][['Feature_Name']].tolist()

print(f"--> Dynamically isolated {len(mar_mnar_remaining)} MAR/MNAR features for ta")
print(f"Features found: {mar_mnar_remaining}\n")

# 2. Execute non-random missingness preservation loop on df_train_preprocessed
for col in mar_mnar_remaining:
    if col in df_train_preprocessed.columns:
        # Determine data type automatically
        is_categorical = (df_train_preprocessed[col].dtype == 'object' or
                           pd.api.types.is_categorical_dtype(df_train_preprocessed[c

    if is_categorical:
        # Categorical MAR/MNAR: fill with explicit 'Missing' category
        df_train_preprocessed[col] = df_train_preprocessed[col].fillna('Missing')
    else:
        # Numerical MAR/MNAR: create binary shadow indicator flag before fillin
        df_train_preprocessed[f'{col}_isna'] = df_train_preprocessed[col].isna()

        # Patch the original column with its median
        col_median = df_train_preprocessed[col].median()
        df_train_preprocessed[col] = df_train_preprocessed[col].fillna(col_med

print("MAR/MNAR low-to-moderate feature treatment completed successfully!")

```

Starting Specialized Imputation for Remaining MAR/MNAR Features (0-70%)...

--> Dynamically isolated 118 MAR/MNAR features for targeted preservation.

Features found: ['V6', 'V5', 'V4', 'V3', 'V2', 'V1', 'M9', 'M6', 'M4', 'M3', 'M2', 'M1', 'D11', 'V60', 'M8', 'M7', 'V61', 'V59', 'V58', 'V57', 'V56', 'V55', 'V54', 'V53', 'V74', 'V73', 'V72', 'V71', 'V70', 'V69', 'V68', 'V67', 'V66', 'V65', 'V64', 'V63', 'V62', 'V11', 'V10', 'V9', 'V8', 'V7', 'D2', 'addr2', 'dist1', 'addr1', 'D3', 'D10', 'V12', 'V13', 'V16', 'V17', 'V18', 'V19', 'V20', 'V21', 'V22', 'V23', 'V24', 'V29', 'V25', 'V28', 'V27', 'V26', 'V14', 'V15', 'V32', 'V30', 'V34', 'V33', 'V31', 'D15', 'V84', 'V85', 'V83', 'V90', 'V87', 'V86', 'V89', 'V94', 'V79', 'V78', 'V81', 'V80', 'V82', 'V93', 'V92', 'V75', 'V77', 'V76', 'V91', 'V88', 'D5', 'M5', 'P_emaildomain', 'card2', 'card5', 'D4', 'V47', 'V46', 'V51', 'V36', 'V35', 'V52', 'V49', 'V50', 'V48', 'V38', 'V37', 'V44', 'V43', 'V42', 'V41', 'V40', 'V39', 'V45', 'card6', 'card3']

MAR/MNAR low-to-moderate feature treatment completed successfully!

```
In [16]: # 1. Calculate the total number of columns in the dataset
total_columns = df_train_preprocessed.shape[1]

# 2. Count how many columns have at least one missing value
columns_with_nulls = df_train_preprocessed.isnull().any().sum()

# 3. Count how many columns are completely clean (zero missing values)
clean_columns = total_columns - columns_with_nulls

# 4. Print the analytical summary
print("=== DATASET COLUMN INTEGRITY SUMMARY ===")
print(f"Total features (columns) in dataset : {total_columns}")
print(f"Features WITH missing values          : {columns_with_nulls} ({(columns_with_")
print(f"Features WITHOUT missing values         : {clean_columns} ({(clean_columns / to
```

```
=== DATASET COLUMN INTEGRITY SUMMARY ===
Total features (columns) in dataset : 541
Features WITH missing values          : 32 (5.91%)
Features WITHOUT missing values       : 509 (94.09%)
```

```
In [17]: # 1. Compute the count and percentage of remaining null values per feature
post_null_counts = df_train_preprocessed.isnull().sum()
post_null_percentages = (post_null_counts / len(df_train_preprocessed)) * 100

# 2. Consolidate results into a verification summary DataFrame
verification_df = pd.DataFrame({
    'Feature_Name': post_null_counts.index,
    'Remaining_Nulls': post_null_counts.values,
    'Null_Percentage': np.round(post_null_percentages.values, 4)
})

# 3. Filter out clean features to isolate only those that still contain missing val
unresolved_missing_features = verification_df[verification_df['Remaining_Nulls'] >

# 4. Display the diagnostic quality check result
print("=== POST-IMPUTATION DATA INTEGRITY VERIFICATION ===")
if unresolved_missing_features.empty:
    print(f"Success! Perfect data integrity achieved. All {df_train_preprocessed.sh
else:
```

```
print(f"Alert! There are still {len(unresolved_missing_features)} features requ  
#display(unresolved_missing_features)
```

=== POST-IMPUTATION DATA INTEGRITY VERIFICATION ===

Alert! There are still 32 features requiring attention before proceeding to Section D:

```
In [18]: print("Executing Final Data Integrity Patch...")

# Step 1: Check if there are completely empty rows disrupting the dataset
empty_rows_count = df_train_preprocessed.isnull().all(axis=1).sum()
if empty_rows_count > 0:
    print(f"--> Detected {empty_rows_count} completely blank rows. Dropping them to  
df_train_preprocessed = df_train_preprocessed.dropna(how='all')

# Step 2: Fallback Global Sweep for any remaining loose nulls (The 12 resilient NaN
remaining_missing_cols = df_train_preprocessed.columns[df_train_preprocessed.isnull

if remaining_missing_cols:
    print(f"--> Fixing remaining loose nulls in the following columns: {remaining_m
    for col in remaining_missing_cols:
        is_categorical = (df_train_preprocessed[col].dtype == 'object' or
                           pd.api.types.is_categorical_dtype(df_train_preprocessed[c

        if is_categorical:
            # If it's categorical, it means it escaped the string filters -> Fill w
            df_train_preprocessed[col] = df_train_preprocessed[col].fillna('Missing
        else:
            # If it's numerical, patch it directly with the median to guarantee cle
            backup_median = df_train_preprocessed[col].median()
            # If the entire column was problematic and median is NaN, fill with 0
            if pd.isna(backup_median):
                backup_median = 0
            df_train_preprocessed[col] = df_train_preprocessed[col].fillna(backup_m

    else:
        print(f"--> No loose missing values found after row verification.")

# Step 3: Ultimate Verification Check
final_null_count = df_train_preprocessed.isnull().sum().sum()
print(f"\n[Verification Result] Total missing values left in df_train_preprocessed:
```

Executing Final Data Integrity Patch...

--> Fixing remaining loose nulls in the following columns: ['V279', 'V280', 'V284', 'V285', 'V286', 'V287', 'V290', 'V291', 'V292', 'V293', 'V294', 'V295', 'V297', 'V298', 'V299', 'V302', 'V303', 'V304', 'V305', 'V306', 'V307', 'V308', 'V309', 'V310', 'V311', 'V312', 'V316', 'V317', 'V318', 'V319', 'V320', 'V321']

[Verification Result] Total missing values left in df_train_preprocessed: 0

C3 - Inconsistent Coding and String Canonicalization

Inconsistent text entries (e.g., trailing spaces or mixed capitalization) can artificially expand the cardinality of categorical fields, leading to errors during feature-target tracking. We isolate key text features (ProductCD, card4, and DeviceType) to verify that their unique encoded values represent clean, consistent categorical pools.

```
In [19]: print("=== COMPREHENSIVE CATEGORICAL PROFILING PIPELINE (ALL FEATURES) ===")

# 1. Dynamically detect ALL categorical features in your current preprocessed DataFrame
all_categorical_features = df_train_preprocessed.select_dtypes(include=['object', 'category'])

print(f"Total categorical features detected for auditing: {len(all_categorical_features)}")

# 2. Iterate through every single categorical column to extract distribution metrics
for col in all_categorical_features:
    print(f"\n=====")
    print(f"FEATURE PROFILING DIRECTORY: '{col}'")
    print(f"=====")

    # Calculate counts and relative density metrics
    total_unique = df_train_preprocessed[col].nunique()
    freqs = df_train_preprocessed[col].value_counts()
    relative_freqs = df_train_preprocessed[col].value_counts(normalize=True)

    print(f"-> Total Active Unique Categories: {total_unique}")

    # Display the Top 10 dominant categories for inspection
    print("\nTop 10 Most Frequent Categories:")
    for idx, (cat, val) in enumerate(freqs.head(10).items()):
        pct = relative_freqs[cat] * 100
        print(f"  {idx+1}. {str(cat):<35} | Count: {val:<8} | Percentage: {pct:.4f}%")

    # Evaluate structural sparsity: check how many categories fall below the 0.1% frequency threshold
    rare_mask = relative_freqs < 0.001
    rare_count = rare_mask.sum()

    if rare_count > 0:
        print(f"\n-> [Sparsity Alert]: {rare_count} out of {total_unique} categories are ultra-rare")
        print("  Sample of 5 ultra-rare categories:")
        for idx, (cat, val) in enumerate(freqs[rare_mask].head(5).items()):
            pct = relative_freqs[cat] * 100
            print(f"    {idx+1}. {str(cat):<32} | Count: {val:<6} | Percentage: {pct:.4f}%")

print("-" * 66)
```

=== COMPREHENSIVE CATEGORICAL PROFILING PIPELINE (ALL FEATURES) ===

Total categorical features detected for auditing: 31

=====

FEATURE PROFILING DIRECTORY: 'ProductCD'

=====

-> Total Active Unique Categories: 5

Top 10 Most Frequent Categories:

1. W	Count: 439670	Percentage: 74.4522%
2. C	Count: 68519	Percentage: 11.6028%
3. R	Count: 37699	Percentage: 6.3838%
4. H	Count: 33024	Percentage: 5.5922%
5. S	Count: 11628	Percentage: 1.9690%

=====

FEATURE PROFILING DIRECTORY: 'card4'

=====

-> Total Active Unique Categories: 4

Top 10 Most Frequent Categories:

1. visa	Count: 386344	Percentage: 65.4222%
2. mastercard	Count: 189217	Percentage: 32.0414%
3. american express	Count: 8328	Percentage: 1.4102%
4. discover	Count: 6651	Percentage: 1.1263%

=====

FEATURE PROFILING DIRECTORY: 'card6'

=====

-> Total Active Unique Categories: 5

Top 10 Most Frequent Categories:

1. debit	Count: 439938	Percentage: 74.4976%
2. credit	Count: 148986	Percentage: 25.2288%
3. Missing	Count: 1571	Percentage: 0.2660%
4. debit or credit	Count: 30	Percentage: 0.0051%
5. charge card	Count: 15	Percentage: 0.0025%

-> [Sparsity Alert]: 2 out of 5 categories have a density < 0.1%.

Sample of 5 ultra-rare categories:

* debit or credit	Count: 30	Percentage: 0.0051%
* charge card	Count: 15	Percentage: 0.0025%

=====

FEATURE PROFILING DIRECTORY: 'P_emaildomain'

=====

-> Total Active Unique Categories: 60

Top 10 Most Frequent Categories:

1. gmail.com	Count: 228355	Percentage: 38.6688%
2. yahoo.com	Count: 100934	Percentage: 17.0918%
3. Missing	Count: 94456	Percentage: 15.9949%

4. hotmail.com	Count: 45250	Percentage: 7.6625%
5. anonymous.com	Count: 36998	Percentage: 6.2651%
6. aol.com	Count: 28289	Percentage: 4.7904%
7. comcast.net	Count: 7888	Percentage: 1.3357%
8. icloud.com	Count: 6267	Percentage: 1.0612%
9. outlook.com	Count: 5096	Percentage: 0.8629%
10. msn.com	Count: 4092	Percentage: 0.6929%

-> [Sparsity Alert]: 37 out of 60 categories have a density < 0.1%.

Sample of 5 ultra-rare categories:

* mail.com	Count: 559	Percentage: 0.0947%
* earthlink.net	Count: 514	Percentage: 0.0870%
* gmail	Count: 496	Percentage: 0.0840%
* outlook.es	Count: 438	Percentage: 0.0742%
* mac.com	Count: 436	Percentage: 0.0738%

=====

FEATURE PROFILING DIRECTORY: 'R_emaildomain'

=====

-> Total Active Unique Categories: 61

Top 10 Most Frequent Categories:

1. Missing	Count: 453249	Percentage: 76.7516%
2. gmail.com	Count: 57147	Percentage: 9.6771%
3. hotmail.com	Count: 27509	Percentage: 4.6583%
4. anonymous.com	Count: 20529	Percentage: 3.4763%
5. yahoo.com	Count: 11842	Percentage: 2.0053%
6. aol.com	Count: 3701	Percentage: 0.6267%
7. outlook.com	Count: 2507	Percentage: 0.4245%
8. comcast.net	Count: 1812	Percentage: 0.3068%
9. yahoo.com.mx	Count: 1508	Percentage: 0.2554%
10. icloud.com	Count: 1398	Percentage: 0.2367%

-> [Sparsity Alert]: 47 out of 61 categories have a density < 0.1%.

Sample of 5 ultra-rare categories:

* me.com	Count: 556	Percentage: 0.0942%
* sbcglobal.net	Count: 552	Percentage: 0.0935%
* cox.net	Count: 459	Percentage: 0.0777%
* outlook.es	Count: 433	Percentage: 0.0733%
* att.net	Count: 430	Percentage: 0.0728%

=====

FEATURE PROFILING DIRECTORY: 'M1'

=====

-> Total Active Unique Categories: 3

Top 10 Most Frequent Categories:

1. T	Count: 319415	Percentage: 54.0886%
2. Missing	Count: 271100	Percentage: 45.9071%
3. F	Count: 25	Percentage: 0.0042%

-> [Sparsity Alert]: 1 out of 3 categories have a density < 0.1%.

Sample of 5 ultra-rare categories:

* F	Count: 25	Percentage: 0.0042%
-----	-----------	---------------------

=====

FEATURE PROFILING DIRECTORY: 'M2'

=====

-> Total Active Unique Categories: 3

Top 10 Most Frequent Categories:

1. T	Count: 285468	Percentage: 48.3402%
2. Missing	Count: 271100	Percentage: 45.9071%
3. F	Count: 33972	Percentage: 5.7527%

=====

FEATURE PROFILING DIRECTORY: 'M3'

=====

-> Total Active Unique Categories: 3

Top 10 Most Frequent Categories:

1. Missing	Count: 271100	Percentage: 45.9071%
2. T	Count: 251731	Percentage: 42.6273%
3. F	Count: 67709	Percentage: 11.4656%

=====

FEATURE PROFILING DIRECTORY: 'M4'

=====

-> Total Active Unique Categories: 4

Top 10 Most Frequent Categories:

1. Missing	Count: 281444	Percentage: 47.6588%
2. M0	Count: 196405	Percentage: 33.2585%
3. M2	Count: 59865	Percentage: 10.1373%
4. M1	Count: 52826	Percentage: 8.9454%

=====

FEATURE PROFILING DIRECTORY: 'M5'

=====

-> Total Active Unique Categories: 3

Top 10 Most Frequent Categories:

1. Missing	Count: 350482	Percentage: 59.3494%
2. F	Count: 132491	Percentage: 22.4356%
3. T	Count: 107567	Percentage: 18.2150%

=====

FEATURE PROFILING DIRECTORY: 'M6'

=====

-> Total Active Unique Categories: 3

Top 10 Most Frequent Categories:

1. F	Count: 227856	Percentage: 38.5843%
2. T	Count: 193324	Percentage: 32.7368%
3. Missing	Count: 169360	Percentage: 28.6788%

=====

FEATURE PROFILING DIRECTORY: 'M7'

=====

-> Total Active Unique Categories: 3

Top 10 Most Frequent Categories:

1. Missing	Count: 346265	Percentage: 58.6353%
2. F	Count: 211374	Percentage: 35.7933%
3. T	Count: 32901	Percentage: 5.5713%

=====

FEATURE PROFILING DIRECTORY: 'M8'

=====

-> Total Active Unique Categories: 3

Top 10 Most Frequent Categories:

1. Missing	Count: 346252	Percentage: 58.6331%
2. F	Count: 155251	Percentage: 26.2897%
3. T	Count: 89037	Percentage: 15.0772%

=====

FEATURE PROFILING DIRECTORY: 'M9'

=====

-> Total Active Unique Categories: 3

Top 10 Most Frequent Categories:

1. Missing	Count: 346252	Percentage: 58.6331%
2. T	Count: 205656	Percentage: 34.8251%
3. F	Count: 38632	Percentage: 6.5418%

=====

FEATURE PROFILING DIRECTORY: 'id_12'

=====

-> Total Active Unique Categories: 3

Top 10 Most Frequent Categories:

1. Missing	Count: 446307	Percentage: 75.5761%
2. NotFound	Count: 123025	Percentage: 20.8326%
3. Found	Count: 21208	Percentage: 3.5913%

=====

FEATURE PROFILING DIRECTORY: 'id_15'

=====

-> Total Active Unique Categories: 4

Top 10 Most Frequent Categories:

1. Missing	Count: 449555	Percentage: 76.1261%
2. Found	Count: 67728	Percentage: 11.4688%
3. New	Count: 61612	Percentage: 10.4332%
4. Unknown	Count: 11645	Percentage: 1.9719%

=====

FEATURE PROFILING DIRECTORY: 'id_16'

=====

-> Total Active Unique Categories: 3

Top 10 Most Frequent Categories:

1. Missing	Count: 461200	Percentage: 78.0980%
2. Found	Count: 66324	Percentage: 11.2311%
3. NotFound	Count: 63016	Percentage: 10.6709%

=====

FEATURE PROFILING DIRECTORY: 'id_23'

=====

-> Total Active Unique Categories: 4

Top 10 Most Frequent Categories:

1. Missing	Count: 585371	Percentage: 99.1247%
2. IP_PROXY:TRANSPARENT	Count: 3489	Percentage: 0.5908%
3. IP_PROXY:ANONYMOUS	Count: 1071	Percentage: 0.1814%
4. IP_PROXY:HIDDEN	Count: 609	Percentage: 0.1031%

=====

FEATURE PROFILING DIRECTORY: 'id_27'

=====

-> Total Active Unique Categories: 3

Top 10 Most Frequent Categories:

1. Missing	Count: 585371	Percentage: 99.1247%
2. Found	Count: 5155	Percentage: 0.8729%
3. NotFound	Count: 14	Percentage: 0.0024%

-> [Sparsity Alert]: 1 out of 3 categories have a density < 0.1%.

Sample of 5 ultra-rare categories:

* NotFound	Count: 14	Percentage: 0.0024%
------------	-----------	---------------------

=====

FEATURE PROFILING DIRECTORY: 'id_28'

=====

-> Total Active Unique Categories: 3

Top 10 Most Frequent Categories:

1. Missing	Count: 449562	Percentage: 76.1273%
2. Found	Count: 76232	Percentage: 12.9089%
3. New	Count: 64746	Percentage: 10.9639%

=====

FEATURE PROFILING DIRECTORY: 'id_29'

=====

-> Total Active Unique Categories: 3

Top 10 Most Frequent Categories:

1. Missing	Count: 449562	Percentage: 76.1273%
2. Found	Count: 74926	Percentage: 12.6877%
3. NotFound	Count: 66052	Percentage: 11.1850%

=====

FEATURE PROFILING DIRECTORY: 'id_30'

=====

-> Total Active Unique Categories: 76

Top 10 Most Frequent Categories:

1. Missing	Count: 512975	Percentage: 86.8654%
2. Windows 10	Count: 21155	Percentage: 3.5823%
3. Windows 7	Count: 13110	Percentage: 2.2200%
4. iOS 11.2.1	Count: 3722	Percentage: 0.6303%
5. iOS 11.1.2	Count: 3699	Percentage: 0.6264%
6. Android 7.0	Count: 2871	Percentage: 0.4862%
7. Mac OS X 10_12_6	Count: 2559	Percentage: 0.4333%
8. Mac OS X 10_11_6	Count: 2348	Percentage: 0.3976%
9. iOS 11.3.0	Count: 2016	Percentage: 0.3414%
10. Windows 8.1	Count: 1914	Percentage: 0.3241%

-> [Sparsity Alert]: 51 out of 76 categories have a density < 0.1%.

Sample of 5 ultra-rare categories:

* Mac OS X 10_9_5	Count: 570	Percentage: 0.0965%
* Android 6.0.1	Count: 566	Percentage: 0.0958%
* iOS 10.3.2	Count: 428	Percentage: 0.0725%
* iOS 9.3.5	Count: 414	Percentage: 0.0701%
* Android 5.1.1	Count: 350	Percentage: 0.0593%

=====

FEATURE PROFILING DIRECTORY: 'id_31'

=====

-> Total Active Unique Categories: 131

Top 10 Most Frequent Categories:

1. Missing	Count: 450258	Percentage: 76.2451%
2. chrome 63.0	Count: 22000	Percentage: 3.7254%
3. mobile safari 11.0	Count: 13423	Percentage: 2.2730%
4. mobile safari generic	Count: 11474	Percentage: 1.9430%
5. ie 11.0 for desktop	Count: 9030	Percentage: 1.5291%
6. safari generic	Count: 8195	Percentage: 1.3877%
7. chrome 62.0	Count: 7182	Percentage: 1.2162%
8. chrome 65.0	Count: 6871	Percentage: 1.1635%
9. chrome 64.0	Count: 6711	Percentage: 1.1364%
10. chrome 63.0 for android	Count: 5806	Percentage: 0.9832%

-> [Sparsity Alert]: 103 out of 131 categories have a density < 0.1%.

Sample of 5 ultra-rare categories:

* safari 11.0	Count: 550	Percentage: 0.0931%
* mobile safari 9.0	Count: 541	Percentage: 0.0916%
* chrome 61.0 for android	Count: 538	Percentage: 0.0911%
* samsung browser 6.4	Count: 509	Percentage: 0.0862%
* chrome	Count: 428	Percentage: 0.0725%

=====

FEATURE PROFILING DIRECTORY: 'id_33'

=====

-> Total Active Unique Categories: 261

Top 10 Most Frequent Categories:

1. Missing	Count: 517251	Percentage: 87.5895%
2. 1920x1080	Count: 16874	Percentage: 2.8574%
3. 1366x768	Count: 8605	Percentage: 1.4571%
4. 1334x750	Count: 6447	Percentage: 1.0917%
5. 2208x1242	Count: 4900	Percentage: 0.8297%
6. 1440x900	Count: 4384	Percentage: 0.7424%
7. 1600x900	Count: 3510	Percentage: 0.5944%
8. 2048x1536	Count: 3482	Percentage: 0.5896%
9. 1280x800	Count: 2149	Percentage: 0.3639%
10. 2560x1600	Count: 2093	Percentage: 0.3544%

-> [Sparsity Alert]: 241 out of 261 categories have a density < 0.1%.

Sample of 5 ultra-rare categories:

* 2220x1080	Count: 544	Percentage: 0.0921%
* 5120x2880	Count: 397	Percentage: 0.0672%
* 3360x2100	Count: 380	Percentage: 0.0643%
* 2732x2048	Count: 354	Percentage: 0.0599%
* 1366x767	Count: 335	Percentage: 0.0567%

=====

=====

FEATURE PROFILING DIRECTORY: 'id_34'

=====

-> Total Active Unique Categories: 5

Top 10 Most Frequent Categories:

1. Missing	Count: 512735	Percentage: 86.8248%
2. match_status:2	Count: 60011	Percentage: 10.1621%
3. match_status:1	Count: 17376	Percentage: 2.9424%
4. match_status:0	Count: 415	Percentage: 0.0703%
5. match_status:-1	Count: 3	Percentage: 0.0005%

-> [Sparsity Alert]: 2 out of 5 categories have a density < 0.1%.

Sample of 5 ultra-rare categories:

* match_status:0	Count: 415	Percentage: 0.0703%
* match_status:-1	Count: 3	Percentage: 0.0005%

=====

=====

FEATURE PROFILING DIRECTORY: 'id_35'

=====

-> Total Active Unique Categories: 3

Top 10 Most Frequent Categories:

1. Missing	Count: 449555	Percentage: 76.1261%
2. T	Count: 77814	Percentage: 13.1768%
3. F	Count: 63171	Percentage: 10.6972%

=====

```

=====
FEATURE PROFILING DIRECTORY: 'id_36'
=====
-> Total Active Unique Categories: 3

Top 10 Most Frequent Categories:
  1. Missing | Count: 449555 | Percentage: 76.1261%
  2. F       | Count: 134066 | Percentage: 22.7023%
  3. T       | Count: 6919   | Percentage: 1.1716%
-----

=====
FEATURE PROFILING DIRECTORY: 'id_37'
=====
-> Total Active Unique Categories: 3

Top 10 Most Frequent Categories:
  1. Missing | Count: 449555 | Percentage: 76.1261%
  2. T       | Count: 110452 | Percentage: 18.7036%
  3. F       | Count: 30533  | Percentage: 5.1704%
-----

=====
FEATURE PROFILING DIRECTORY: 'id_38'
=====
-> Total Active Unique Categories: 3

Top 10 Most Frequent Categories:
  1. Missing | Count: 449555 | Percentage: 76.1261%
  2. F       | Count: 73922  | Percentage: 12.5177%
  3. T       | Count: 67063  | Percentage: 11.3562%
-----

=====
FEATURE PROFILING DIRECTORY: 'DeviceType'
=====
-> Total Active Unique Categories: 3

Top 10 Most Frequent Categories:
  1. Missing | Count: 449730 | Percentage: 76.1557%
  2. desktop | Count: 85165  | Percentage: 14.4215%
  3. mobile  | Count: 55645  | Percentage: 9.4227%
-----

=====
FEATURE PROFILING DIRECTORY: 'DeviceInfo'
=====
-> Total Active Unique Categories: 1787

Top 10 Most Frequent Categories:
  1. Missing | Count: 471874 | Percentage: 79.9055%
  2. Windows | Count: 47722  | Percentage: 8.0811%
  3. iOS Device | Count: 19782 | Percentage: 3.3498%
  4. MacOS    | Count: 12573  | Percentage: 2.1291%
  5. Trident/7.0 | Count: 7440  | Percentage: 1.2599%

```

6. rv:11.0	Count: 1901	Percentage: 0.3219%
7. rv:57.0	Count: 962	Percentage: 0.1629%
8. SM-J700M Build/MMB29K	Count: 549	Percentage: 0.0930%
9. SM-G610M Build/MMB29K	Count: 461	Percentage: 0.0781%
10. SM-G531H Build/LMY48B	Count: 410	Percentage: 0.0694%

-> [Sparsity Alert]: 1780 out of 1787 categories have a density < 0.1%.

Sample of 5 ultra-rare categories:

* SM-J700M Build/MMB29K	Count: 549	Percentage: 0.0930%
* SM-G610M Build/MMB29K	Count: 461	Percentage: 0.0781%
* SM-G531H Build/LMY48B	Count: 410	Percentage: 0.0694%
* rv:59.0	Count: 362	Percentage: 0.0613%
* SM-G935F Build/NRD90M	Count: 334	Percentage: 0.0566%

In [20]: `import pandas as pd`

```
print("== STARTING AUTOMATED STRING CANONICALIZATION SWEEP ==")

# 1. Identify all remaining categorical columns in the preprocessed dataset
categorical_features = df_train_preprocessed.select_dtypes(include=['object', 'category'])
inconsistencies_found = []

# 2. Programmatically evaluate formatting differences
for col in categorical_features:
    # Get original unique values (excluding true python NaNs)
    original_unique_count = df_train_preprocessed[col].dropna().nunique()

    if original_unique_count > 0:
        # Simulate canonicalization: trim spaces and homogenize to lowercase
        canonical_series = df_train_preprocessed[col].astype(str).str.strip().str.lower()
        canonical_unique_count = canonical_series.nunique()

        # If the unique count drops, we found hidden duplicates due to poor formatting
        has_formatting_issue = original_unique_count != canonical_unique_count

        # Also check for trailing/leading whitespace explicitly
        has_whitespace = df_train_preprocessed[col].astype(str).str.contains(r'^\s|^\s$')

        if has_formatting_issue or has_whitespace:
            inconsistencies_found.append({
                'Feature_Name': col,
                'Original_Cardinality': original_unique_count,
                'Canonical_Cardinality': canonical_unique_count,
                'Hidden_Categories_Expanded': original_unique_count - canonical_unique_count,
                'Issue_Detected': "Whitespace/Capitalization Bias"
            })

# 3. Convert results into a diagnostic summary table
df_inconsistency_report = pd.DataFrame(inconsistencies_found)

print(f"\nScan completed. Evaluated {len(categorical_features)} categorical features")
if df_inconsistency_report.empty:
    print("Success! No structural capitalization or trailing whitespace inconsistencies found")
else:
```

```
print(f"Alert! Found {len(df_inconsistency_report)} features with hidden string  
display(df_inconsistency_report)
```

=== STARTING AUTOMATED STRING CANONICALIZATION SWEEP ===

Scan completed. Evaluated 31 categorical features.

Alert! Found 1 features with hidden string formatting inflation:

	Feature_Name	Original_Cardinality	Canonical_Cardinality	Hidden_Categories_Expanded
0	DeviceInfo	1787	1780	7 W

Case Study: Standardizing DeviceInfo Values

The text-sweeper scan identified a subtle but impactful formatting discrepancy within the hardware manufacturer and build tracking parameter (`DeviceInfo`). The baseline feature exhibited an artificial cardinality inflation, registering **1,787 original unique categories** which dropped to **1,780 canonical unique entities** upon simulation. This variance was driven by syntactic noise—specifically trailing or leading whitespaces and mixed capitalization styles (e.g., treating identical builds as disjoint categories due to minor text entry anomalies).

Furthermore, once corrected, the feature profile exposed an extreme long-tail distribution, where the vast majority of technical sub-versions contributed a relative sample density below the 0.1% baseline threshold.

Methodological Resolution Action: To eliminate this structural inflation and secure stable decision boundaries for subsequent tracking, a two-stage harmonization pipeline was applied:

1. **Syntactic Standardization:** The text vector underwent explicit canonicalization by casting all records to lowercase and removing boundary formatting blanks via a string-stripping operation (`.str.strip().str.lower()`). This successfully resolved the 7 hidden duplicate categories.
2. **Rare-Class Compaction:** To insulate downstream models from a geometric expansion of sparse indicators, a density-based thresholding constraint was enforced. Any device sub-version capturing less than 0.1% of the statistical sample was programmatically collapsed into a single, centralized category designated as `'Other'` .

This optimization successfully concentrated the categorical variance of hardware attributes, reducing high-dimensional noise and stabilizing the feature matrix for exploratory visualization stages.

```
In [21]: print("=== C3 - SPECIFIC CANONICALIZATION FOR DEVICEINFO ===")  
  
if 'DeviceInfo' in df_train_preprocessed.columns:  
    print("--> Applying syntactic canonicalization to 'DeviceInfo' (Trimming spaces
```

```

# 1. Fix formatting inflation (Spaces and Capitalization)
df_train_preprocessed['DeviceInfo'] = df_train_preprocessed['DeviceInfo'].astype(str)

print("--> Collapsing sparse device sub-versions (< 0.1% frequency) into 'Other'")
# 2. Calculate relative categorical frequencies on the cleaned strings
freqs = df_train_preprocessed['DeviceInfo'].value_counts(normalize=True)

# Isolate categories falling below the 0.001 (0.1%) density threshold
rare_devices = freqs[freqs < 0.001].index.tolist()

# Group rare entries into 'Other' to clean the cardinality gap
df_train_preprocessed['DeviceInfo'] = df_train_preprocessed['DeviceInfo'].replace(rare_devices, 'Other')

print("\nCanonicalization and semantic alignment for 'DeviceInfo' completed successfully!")

```

=== C3 - SPECIFIC CANONICALIZATION FOR DEVICEINFO ===

--> Applying syntactic canonicalization to 'DeviceInfo' (Trimming spaces + Lowercase)...

--> Collapsing sparse device sub-versions (< 0.1% frequency) into 'Other'...

Canonicalization and semantic alignment for 'DeviceInfo' completed successfully!

In [22]:

```

print(f"New unique categories in 'DeviceInfo': {df_train_preprocessed['DeviceInfo'].nunique()}")
print(df_train_preprocessed['DeviceInfo'].value_counts().head(10))

```

New unique categories in 'DeviceInfo': 8

DeviceInfo

missing 471874

windows 47722

Other 28286

ios device 19782

macos 12573

trident/7.0 7440

rv:11.0 1901

rv:57.0 962

Name: count, dtype: int64

Case Study: Cleaning Card Type Categories (card6)

A bivariate distribution sweep across the transaction type feature (card6) identified an operational fragmentation anomaly. The feature was originally split into five active categories, where two sub-classes— debit or credit (30 records) and charge card (15 records)—contributed an aggregate density of less than 0.015% to the structural sample. Crucially, targeted cross-tabulation tracking revealed a fraud count of exactly zero across these sparse cohorts, meaning they carry zero statistical variance regarding the objective target (isFraud).

Resolution Framework: To prevent unnecessary geometric inflation and enhance downstream model stability, both classes were programmatically collapsed into the dominant credit cluster. This alignment is strictly supported by banking domain logic: charge cards share credit-line backing systems, while ambiguous debit or credit flags route predominantly through network credit rails during electronic commerce authorizations. The

reduction simplifies the categorical distribution into three robust operational groups (`debit` , `credit` , and `Missing`), concentrating statistical power.

```
In [23]: print("=== CANONICALIZING AND HOMOGENIZING 'card6' OPERATIONAL POOLS ===")

if 'card6' in df_train_preprocessed.columns:
    # Define the strategic financial mapping
    card6_risk_mapping = {
        'debit': 'debit',
        'credit': 'credit',
        'debit or credit': 'credit', # Collapsed into dominant credit pool due to
        'charge card': 'credit',     # Charge frameworks operate under credit risk
        'Missing': 'Missing'
    }

    # Apply mapping real-time
    df_train_preprocessed['card6'] = df_train_preprocessed['card6'].map(card6_risk_

    # Quick sanity check verification
    print("\nVerification of homogenized distributions:")
    display(df_train_preprocessed['card6'].value_counts(dropna=False))
```

=== CANONICALIZING AND HOMOGENIZING 'card6' OPERATIONAL POOLS ===

Verification of homogenized distributions:

```
card6
debit      439938
credit     149031
Missing      1571
Name: count, dtype: int64
```

Case Study: Programmatic Suffix Stripping and Sparsity Compaction in Email Domains (`P_emaildomain` / `R_emaildomain`)

An operational audit of the communication infrastructure parameters exposed a high degree of structural fragmentation within the Purchaser (`P_emaildomain`) and Recipient (`R_emaildomain`) features. The baseline dataset spread transactional records across 60 discrete categories, a dimensionality inflation driven by two distinct factors: regional suffix variations (e.g., `yahoo.com` vs. `yahoo.com.mx` or `yahoo.co.uk`) and isolated notation dropouts (such as an un-extended `gmail` token capturing a minor cohort of 496 records).

Leaving these categories unaligned forces subsequent encoding protocols to treat identical corporate communication nodes as completely independent geometric dimensions. This artificial dispersion weakens the model's capacity to establish generalized decision boundaries and spreads statistical variance thin across low-density categories.

Methodological Resolution Framework: To mitigate this dimensionality bloat while preserving the data's underlying predictive power, a fully automated, two-stage vector-splitting and compaction pipeline was engineered:

1. **Syntactic Canonicalization via Suffix Stripping:** Every record was programmatically cast to lowercase and stripped of trailing whitespace. The root domain was then isolated by executing a vector split at the first delimiter period character (`.str.split('.').str[0]`). This operation seamlessly fused regional variants and syntax anomalies back into their dominant institutional anchors (e.g., mapping all local Yahoo and Google sub-domains uniformly to `yahoo` and `gmail`).
2. **Dynamic Sparsity Compaction:** To insulate the feature space from long-tail categorical inflation, a relative density threshold was enforced. Following the root split, any localized or independent domain capturing less than 0.1% of the entire sample population was automatically compressed into a robust, centralized category designated as `'other'`, while structural missingness parameters were explicitly retained under a unified `'Missing'` class flag.

This algorithmic approach effectively condenses the high-cardinality array down to a dense, statistically stable layout, optimizing the dataset for target-correlation modeling without the need for manual hardcoding.

```
In [24]: print("=== C3 - AUTOMATED EMAIL CANONICALIZATION BY SPLITTING & COMPACTION ===")

def clean_and_split_emails(df, col_name):
    if col_name in df.columns:
        print(f"\n--> Processing '{col_name}' programmatically...")

        # 1. Standardize text: force lowercase, remove spaces, and convert to string
        df[col_name] = df[col_name].astype(str).str.strip().str.lower()

        # 2. Extract the root domain by splitting at the first period character '.'
        # Example: 'yahoo.com.mx' -> ['yahoo', 'com', 'mx'] -> takes index 0 ('yahoo')
        df[col_name] = df[col_name].str.split('.').str[0]

        # Handle structural missing placeholders so they don't mix with valid names
        df[col_name] = df[col_name].replace({'nan': 'Missing', 'missing': 'Missing'})

        # 3. Dynamic rare-class compaction (Threshold: 0.1% of the dataset)
        freqs = df[col_name].value_counts(normalize=True)
        rare_domains = freqs[freqs < 0.001].index.tolist()

        # Remove 'Missing' from the rare list if it falls below 0.1% to preserve the
        if 'Missing' in rare_domains:
            rare_domains.remove('Missing')

        # Collapse remaining sparse domains into 'other'
        df[col_name] = df[col_name].replace(rare_domains, 'other')

        # Display the finalized clean distribution
        print(f"    * Unique categories remaining in '{col_name}': {df[col_name].nunique()}")
        display(df[col_name].value_counts())

    # Execute the automated vector splitting pipeline on both columns
    clean_and_split_emails(df_train_preprocessed, 'P_emaildomain')
    clean_and_split_emails(df_train_preprocessed, 'R_emaildomain')
```

=== C3 - AUTOMATED EMAIL CANONICALIZATION BY SPLITTING & COMPACTION ===

--> Processing 'P_emaildomain' programmatically...

* Unique categories remaining in 'P_emaildomain': 22

P_emaildomain

gmail	228851
yahoo	102909
Missing	94456
hotmail	46005
anonymous	36998
aol	28289
comcast	7888
icloud	6267
other	5986
outlook	5534
msn	4092
att	4033
live	3846
sbcglobal	2970
verizon	2705
ymail	2396
bellsouth	1909
me	1522
cox	1393
optonline	1011
charter	816
rocketmail	664

Name: count, dtype: int64

--> Processing 'R_emaildomain' programmatically...

* Unique categories remaining in 'R_emaildomain': 13

R_emaildomain

Missing	453249
gmail	57242
hotmail	28241
anonymous	20529
yahoo	13691
other	4694
aol	3701
outlook	2940
comcast	1812
live	1571
icloud	1398
msn	852
verizon	620

Name: count, dtype: int64

Case Study: Resolving Semantic Overlap in Identity Parameters (id_15)

The dynamic profiling sweep across the identity tracking indicator (id_15) revealed a low-cardinality structure consisting of four discrete classes: **Missing** (76.12%), **Found** (11.46%), **New** (10.43%), and **Unknown** (1.97%). Although this feature does not suffer from high geometric expansion or formatting errors, it exhibits a distinct issue of **semantic overlap**.

Mathematically and operationally, the labels `Unknown` and `Missing` both define an identical informative state: the systemic absence of a verifiable identity status at the exact moment of transaction authorization. Maintaining these two statuses as disjoint entities artificially splinters data density, introducing redundant categories that add zero unique variance regarding fraudulent behavioral tracking.

Resolution Action: To secure a clean, non-redundant categorical architecture, the `Unknown` cohort (11,645 records) was programmatically collapsed into the unified, engineered `'Missing'` class pool. This adjustment consolidates the feature's layout into three logically solid decision boundaries (`Missing` , `Found` , and `New`), maximizing statistical power and streamlining subsequent bivariate target tracking.

```
In [25]: print("=== C3 - SEMANTIC ALIGNMENT FOR IDENTITY STATUS (id_15) ===")

if 'id_15' in df_train_preprocessed.columns:
    print(f"Original distribution of id_15:\n{df_train_preprocessed['id_15'].value_

    # Map 'Unknown' into the dominant engineered 'Missing' category
    df_train_preprocessed['id_15'] = df_train_preprocessed['id_15'].replace({'Unkno

    print("\n--> Verification of homogenized distributions for 'id_15':")
    display(df_train_preprocessed['id_15'].value_counts())
```

```
=== C3 - SEMANTIC ALIGNMENT FOR IDENTITY STATUS (id_15) ===
```

```
Original distribution of id_15:
```

```
id_15
```

```
Missing    449555
```

```
Found       67728
```

```
New         61612
```

```
Unknown     11645
```

```
Name: count, dtype: int64
```

```
--> Verification of homogenized distributions for 'id_15':
```

```
id_15
```

```
Missing    461200
```

```
Found       67728
```

```
New         61612
```

```
Name: count, dtype: int64
```

Case Study: Alignment of Systemic Dropped Flags in Verification Streams (`id_34`)

The target-tracking cross-tabulation profile for the identity matching status feature (`id_34`) revealed a stable, low-cardinality distribution with a single sparse anomaly: the token `match_status:-1` captures an extreme outlier sample size of exactly 3 records (representing 0.00% of structural volume). Cross-tabulation confirms a zero fraud occurrence within this microscopic cluster, failing to yield any statistical variance or actionable prediction boundaries on its own.

Resolution Action: In commercial verification architectures, numeric indicators like `-1` denote system timeouts or infrastructural connection errors during authentication handshakes. Because an operational failure to return a matching status is analytically equivalent to a complete absence of data, the `match_status: -1` flag was programmatically collapsed into the primary engineered `'Missing'` category pool. This adjustment isolates the legitimate, high-volume operational categories (`match_status:1` , `match_status:2` , and `match_status:0`) while eliminating sparse, low-yield technical noise before advancing to Section D.

```
In [26]: print("=== C3 - SEMANTIC ALIGNMENT FOR IDENTITY MATCHING (id_34) ===")

if 'id_34' in df_train_preprocessed.columns:
    # Collapse the ultra-rare infrastructure dropout (-1) into the Missing pool
    df_train_preprocessed['id_34'] = df_train_preprocessed['id_34'].replace({'match

    # Quick verification check
    print("\n--> Final stable categories for 'id_34':")
    display(df_train_preprocessed['id_34'].value_counts())

=== C3 - SEMANTIC ALIGNMENT FOR IDENTITY MATCHING (id_34) ===

--> Final stable categories for 'id_34':
id_34
Missing          512738
match_status:2    60011
match_status:1    17376
match_status:0      415
Name: count, dtype: int64
```

Case Study: Cleaning `id_27` Categories

The descriptive profiling sweep across the identity validation feature (`id_27`) exposed an extreme case of statistical imbalance and sparsity. The feature's distribution is overwhelmingly concentrated within the engineered `Missing` repository (99.12%), while the operational category `NotFound` captures a microscopic sample size of exactly 14 instances out of more than 500,000 transaction records (representing a sparse density of 0.0024%).

Methodological Resolution Framework: In transactional security architectures, a `NotFound` status implies that an external verification query was executed but failed to locate an active identification profile. For data modeling and statistical tracking purposes, this outcome shares an identical semantic and behavioral risk boundary with a traditional systemic data omission (`Missing`).

To prevent unnecessary categorical fragmentation over an statistically irrelevant sample size, the 14 records within the `NotFound` cohort were programmatically collapsed into the primary centralized `'Missing'` pool. This corrective alignment simplifies the feature's

topology into a dense, robust binary layout (`Missing` and `Found`), eliminating mathematical noise and strengthening decision boundaries before proceeding to Section D.

```
In [27]: print("=== C3 - SEMANTIC ALIGNMENT FOR IDENTITY VALIDATION (id_27) ===")

if 'id_27' in df_train_preprocessed.columns:
    # Programmatically merge 'NotFound' into the dominant 'Missing' repository
    df_train_preprocessed['id_27'] = df_train_preprocessed['id_27'].replace({'NotFo

    # Quick verification check to ensure clean binary layout
    print("\n--> Final stabilized categories for 'id_27':")
    display(df_train_preprocessed['id_27'].value_counts())

=== C3 - SEMANTIC ALIGNMENT FOR IDENTITY VALIDATION (id_27) ===

--> Final stabilized categories for 'id_27':
id_27
Missing    585385
Found       5155
Name: count, dtype: int64
```

Case Study: Root Extraction and Version Compaction in Web Browser Arrays (`id_31`)

The categorical footprint for the transaction browser tracking feature (`id_31`) suffered from severe structural dilution, registering **131 distinct unique classes**. This high-cardinality inflation is driven exclusively by operational sub-version tokens and device environments (e.g., separating `chrome 63.0`, `chrome 62.0`, and `chrome 63.0 for android` into isolated entities). Leaving this matrix unaligned forces subsequent encoding steps to generate sparse high-dimensional sparse spaces, breaking down decision boundaries for classification algorithms.

Methodological Resolution Framework: To concentrate data density and isolate generalized user behavior signatures, a programmatic split-and-collapse pipeline was executed:

1. **Root Suffix Stripping:** Every categorical text vector was cast to lowercase, stripped of trailing whitespaces, and parsed by isolating the substring prior to the first whitespace delimiter (`.str.split(' ').str[0]`). This operation fused historical browser logs directly back to their major institutional cores (collapsing version numbers into unified `chrome`, `safari`, `ie`, or `edge` nodes).
2. **Long-Tail Categorical Compaction:** Following root extraction, a statistical threshold filter was deployed. Any niche, regional, or obsolete software core capturing less than 0.1% of the statistical distribution was compressed into a consolidated `'other'` category, while the structural missing value block was explicitly isolated under the `'Missing'` flag.

```
In [28]: print("=== C3 - AUTOMATED BROWSER CANONICALIZATION BY ROOT SPLITTING (id_31) ===")
```

```

if 'id_31' in df_train_preprocessed.columns:
    print(f"Original unique categories count: {df_train_preprocessed['id_31'].nunique}")

    # 1. Standardize text layout
    df_train_preprocessed['id_31'] = df_train_preprocessed['id_31'].astype(str).str.strip()

    # 2. Extract root browser name by splitting at the first whitespace character
    # Example: 'chrome 63.0 for android' -> ['chrome', '63.0', 'for', 'android'] -> 'chrome'
    df_train_preprocessed['id_31'] = df_train_preprocessed['id_31'].str.split(' ').str[0]

    # Standardize missing value representations
    df_train_preprocessed['id_31'] = df_train_preprocessed['id_31'].replace({'nan': 'Missing'})

    # 3. Dynamic rare-class compaction (Threshold: 0.1% frequency baseline)
    freqs = df_train_preprocessed['id_31'].value_counts(normalize=True)
    rare_browsers = freqs[freqs < 0.001].index.tolist()

    # Protect our structural Missing flag from being collapsed
    if 'Missing' in rare_browsers:
        rare_browsers.remove('Missing')

    # Collapse sparse/micro-browsers into 'other'
    df_train_preprocessed['id_31'] = df_train_preprocessed['id_31'].replace(rare_browsers, 'other')

    # Verify the engineering performance
    print(f"--> Strategy executed. Unified unique categories remaining: {df_train_preprocessed['id_31'].nunique}")
    display(df_train_preprocessed['id_31'].value_counts())

```

=== C3 - AUTOMATED BROWSER CANONICALIZATION BY ROOT SPLITTING (id_31) ===

Original unique categories count: 131

--> Strategy executed. Unified unique categories remaining: 9

id_31

Missing	450258
chrome	76059
mobile	28379
ie	9733
safari	8913
firefox	7012
edge	6401
samsung	2044
other	1741

Name: count, dtype: int64

Case Study: Simplifying Screen Resolution Categories (id_33)

The technical indicator for device display resolution (id_33) introduced high-cardinality noise into the dataset, registering **261 unique text strings** mapped in an **Width x Height** format. The vector distribution is highly sparse; while the structural **Missing** flag encompasses 87.58% of the operational sample, **241 out of the 261 active categories** display a individual density below the 0.1% baseline threshold. This multi-version scattering reflects the extreme physical variety of commercial hardware displays, generating an excessive amount of rare dimensions if evaluated without modification.

Resolution Strategy Execution: To streamline spatial dimensionality while preserving the predictive power embedded within dominant commercial hardware layouts, a frequency-based compaction rule was applied:

1. **Syntactic Alignment:** Values were mapped to lowercase, stripped of whitespace, and missing attributes were explicitly unified under the centralized 'Missing' label.
2. **Frequency Threshold Masking:** A programmatic frequency sweep isolated all long-tail configurations capturing less than 0.1% of historical transaction patterns (such as ultra-widescreen monitors like 5120x2880 or minor variations like 1366x767). These 241 sparse hardware categories were compressed into a unified, balanced node labeled 'other' .

This transformation successfully scales down the feature space from 261 dimensions to a dense, statistically stable array containing only the top commercial resolutions (e.g., 1920x1080 , 1366x767), stabilizing variance boundaries for subsequent bivariate target cross-tabulations.

```
In [29]: import numpy as np
import pandas as pd

print("=== C3 - SCREEN CLASS EXTRACTION & CARDINALITY COMPACTION (id_33) ===")

if "id_33" in df_train_preprocessed.columns:
    print(
        f"Original unique resolution count: {df_train_preprocessed['id_33'].nunique()}"
    )

    # =====
    # PASO PREVIO: Extracción de Clase de Pantalla
    # =====
    def id_33_to_screen_class(val):
        """Map WxH resolution string to Mobile / Tablet / PC (by shorter side in px)
        if pd.isna(val) or str(val).strip().lower() in ('', 'nan', 'none', 'missing'):
            return 'Missing'

        s = str(val).strip().lower()

        # Si es '0x0', ahora lo agrupamos directamente en 'Missing'
        if s == '0x0':
            return 'Missing'

        if 'x' not in s:
            return 'Other'

        try:
            w, h = map(int, s.split('x', 1))
        except ValueError:
            return 'Other'

        # Si el ancho o alto son menores o iguales a 0, también son 'Missing'
        if w <= 0 or h <= 0:
            return 'Missing'
```

```

        short = min(w, h)
        if short < 720:
            return 'Mobile'
        if short < 1080:
            return 'Tablet'
        return 'PC'

# Creamos la nueva columna basada en los datos originales/crudos
df_train_preprocessed["id_33_class"] = df_train_preprocessed["id_33"].apply(
    id_33_to_screen_class
)

# =====
# PASO ORIGINAL: Compactación de Cardinalidad
# =====

# 1. Standardize string type and remove formatting blanks
df_train_preprocessed["id_33"] = (
    df_train_preprocessed["id_33"].astype(str).str.strip().str.lower()
)
df_train_preprocessed["id_33"] = df_train_preprocessed["id_33"].replace(
    {"nan": "Missing", "missing": "Missing"}
)

# 2. Extract relative categorical frequencies
freqs = df_train_preprocessed["id_33"].value_counts(normalize=True)

# Isolate categories falling below our 0.1% baseline threshold
rare_resolutions = freqs[freqs < 0.001].index.tolist()

# Ensure our structural Missing category is never collapsed
# (Agregamos validaciones por si acaso quedó en minúscula o mayúscula)
for missing_variant in ["missing", "Missing"]:
    if missing_variant in rare_resolutions:
        rare_resolutions.remove(missing_variant)

# 3. Collapse long-tail resolutions into a single 'other' tag
df_train_preprocessed["id_33"] = df_train_preprocessed["id_33"].replace(
    rare_resolutions, "other"
)

print(
    f"--> Strategy executed. Compacted unique categories remaining: {df_train_p
)

print("\n--- Distribution of New Feature: id_33_class ---")
display(df_train_preprocessed["id_33_class"].value_counts())

print("\n--- Distribution of Compacted id_33 ---")
display(df_train_preprocessed["id_33"].value_counts())

```

```

=== C3 - SCREEN CLASS EXTRACTION & CARDINALITY COMPACTION (id_33) ===
Original unique resolution count: 261
--> Strategy executed. Compacted unique categories remaining: 21

--- Distribution of New Feature: id_33_class ---
id_33_class
Missing      517257
PC           38925
Tablet       32105
Mobile       2253
Name: count, dtype: int64
--- Distribution of Compacted id_33 ---
id_33
Missing      517251
1920x1080    16874
1366x768     8605
1334x750     6447
other        6340
2208x1242    4900
1440x900     4384
1600x900     3510
2048x1536    3482
1280x800     2149
2560x1600    2093
2560x1440    1865
2880x1800    1756
1280x1024    1743
1680x1050    1727
1136x640     1712
2436x1125    1484
1280x720     1318
1920x1200    1056
2001x1125    1020
1024x768     824
Name: count, dtype: int64

```

Case Study: Operating System Root Extraction and Version Compaction (id_30)

The categorical matrix for the operating system tracking metadata (id_30) presented a highly fragmented footprint containing **76 distinct unique categories**. This inflation is entirely an operational artifact of software versioning and minor build releases (e.g., separating ios 11.2.1 from ios 11.1.2). Evaluating this feature without optimization introduces sparse multi-collinearity during subsequent vector transformations, diluting the mathematical connection between the core operating systems and the transaction risk profile.

Methodological Resolution Framework: To concentrate data density and isolate structural behavior trends, a programmatic parsing and compaction pipeline was executed:

1. **Root Extraction Suffix Stripping:** Every categorical text vector was mapped to lowercase, stripped of trailing whitespaces, and tokenized by isolating the substring

prior to the first whitespace delimiter (`.str.split(' ').str[0]`). This operation collapsed micro-version strings directly into their primary corporate infrastructure nodes (e.g., grouping software lines into unified `windows` , `ios` , `android` , or `mac` classes).

2. **Sparsity Compaction Filtering:** Following root consolidation, a 0.1% baseline relative frequency threshold was enforced. Any low-volume, specialized, or obsolete legacy software core capturing a statistically negligible portion of the transaction history was grouped into a single, cohesive `'other'` pool, while the structural `Missing` flag was explicitly isolated.

This structural grouping scales down the feature space from 76 sparse classes to a tightly dense layout, stabilizing variance and preparing the system for exploratory cross-tabulation tracking (Section D).

```
In [30]: print("=== C3 - AUTOMATED OS CANONICALIZATION BY ROOT SPLITTING (id_30) ===")

if 'id_30' in df_train_preprocessed.columns:
    print(f"Original unique OS categories count: {df_train_preprocessed['id_30'].nunique()}")

    # 1. Standardize text layout
    df_train_preprocessed['id_30'] = df_train_preprocessed['id_30'].astype(str).str.strip()

    # 2. Extract root OS name by splitting at the first whitespace character
    # Example: 'windows 10' -> ['windows', '10'] -> index 0 ('windows')
    # Example: 'mac os x 10_12_6' -> ['mac', 'os', 'x', ...] -> index 0 ('mac')
    df_train_preprocessed['id_30'] = df_train_preprocessed['id_30'].str.split(' ').str[0]

    # Standardize missing value representations
    df_train_preprocessed['id_30'] = df_train_preprocessed['id_30'].replace({'nan': 'Missing'})

    # 3. Dynamic rare-class compaction (Threshold: 0.1% frequency baseline)
    freqs = df_train_preprocessed['id_30'].value_counts(normalize=True)
    rare_os = freqs[freqs < 0.001].index.tolist()

    # Protect our structural Missing flag from being collapsed
    if 'Missing' in rare_os:
        rare_os.remove('Missing')

    # Collapse sparse/micro-OS entries into 'other'
    df_train_preprocessed['id_30'] = df_train_preprocessed['id_30'].replace(rare_os, 'other')

    # Verify the engineering performance
    print(f"--> Strategy executed. Unified unique categories remaining: {df_train_preprocessed['id_30'].nunique()}")
    display(df_train_preprocessed['id_30'].value_counts())

=== C3 - AUTOMATED OS CANONICALIZATION BY ROOT SPLITTING (id_30) ===
Original unique OS categories count: 76
--> Strategy executed. Unified unique categories remaining: 7
```

```

id_30
Missing    512975
windows    36739
ios         19782
mac         13580
android     6303
linux       1136
other        25
Name: count, dtype: int64

```

C4 - Structural Out-of-Range and Outlier Diagnostics

In high-dimensional financial transaction data, numerical features (such as transaction amounts, time-deltas, and entity counters) often exhibit highly skewed distributions and heavy-tailed behavior. Isolating structural out-of-range anomalies is crucial; however, in financial fraud modeling, an "outlier" is rarely just sensor noise or a data entry typo. Extreme transaction attributes (e.g., an unusually massive purchase amount or an accelerated sequence of rapid requests) frequently represent genuine anomalous behaviors directly tied to systemic fraud patterns.

To programmatically audit the statistical boundaries of all remaining numerical features within `df_train_preprocessed`, an automated outlier detection scanner is deployed leveraging the **Interquartile Range (IQR) Criterion**. For each numerical vector, the lower and upper statistical boundaries are calculated as:

$$\text{Lower Fence} = Q_1 - 1.5 \times IQR$$

$$\text{Upper Fence} = Q_3 + 1.5 \times IQR$$

Critical Preprocessing Constraint: To guarantee diagnostic integrity and prevent severe mathematical distortion, any engineered out-of-bounds constants (such as the `-999` missingness placeholder injected in previous stages) are programmatically masked into temporary nulls before computing the sample quartiles (Q_1 , Q_3) and the resulting scale factor (IQR). This prevents artificial variance expansion and ensures the fence boundaries reflect true transactional behavior.

```

In [31]: import pandas as pd
import numpy as np

print("=== STARTING AUTOMATED OUTLIER DETAILED EXTRACTION SWEEP ===")

# 1. Isolate all numerical columns in the preprocessed dataset
numerical_features = df_train_preprocessed.select_dtypes(include=[np.number]).columns

if 'isFraud' in numerical_features:
    numerical_features.remove('isFraud')

outlier_report = []

# CRITICAL STORAGE: Dictionary to store the exact index locations of outliers per feature

```

```

outliers_by_feature = {}

# Keep track of skipped binary columns for reporting integrity
skipped_binary_features = []

# 2. Programmatically scan each numerical column and extract outlier instances
for col in numerical_features:
    # Programmatically mask our engineered placeholder (-999) to protect statistical integrity
    clean_series = df_train_preprocessed[col].replace(-999, np.nan).dropna()

    # DYNAMIC GUARDRAIL: Identify and skip binary or indicator variables (0/1 or similar)
    if clean_series.nunique() <= 2:
        skipped_binary_features.append(col)
        continue # Bypasses the IQR calculation for this feature

    total_records = len(clean_series)

    if total_records > 0:
        # Calculate Quartiles and IQR
        q1 = clean_series.quantile(0.25)
        q3 = clean_series.quantile(0.75)
        iqr = q3 - q1

        # Define standard statistical upper and lower boundaries
        lower_bound = q1 - (1.5 * iqr)
        upper_bound = q3 + (1.5 * iqr)

        # Isolate the exact series entries that breach the fences
        outlier_mask = (clean_series < lower_bound) | (clean_series > upper_bound)
        outlier_indices = clean_series[outlier_mask].index

        total_outliers = len(outlier_indices)
        outlier_percentage = (total_outliers / total_records) * 100

        if total_outliers > 0:
            # Save the statistical profile data
            outlier_report.append({
                'Feature_Name': col,
                'Min_Value': clean_series.min(),
                'Max_Value': clean_series.max(),
                'Lower_Fence': lower_bound,
                'Upper_Fence': upper_bound,
                'Outlier_Count': total_outliers,
                'Outlier_Percentage': outlier_percentage
            })

            # Extract and assign the real DataFrame indices to our tracking index
            outliers_by_feature[col] = outlier_indices

# 3. Build and format the diagnostic summary DataFrame
df_outlier_report = pd.DataFrame(outlier_report).sort_values(by='Outlier_Percentage')

print(f"\nScan completed. Evaluated {len(numerical_features)} total numerical features")
print(f"--> Programmatically bypassed {len(skipped_binary_features)} binary/indicator features")
print(f"Detected {len(df_outlier_report)} continuous features exhibiting out-of-range values")

```

```
# Display the Top 100 features with the highest density of outlier data  
display(df_outlier_report.head(100))
```

=== STARTING AUTOMATED OUTLIER DETAILED EXTRACTION SWEEP ===

Scan completed. Evaluated 509 total numerical features.

--> Programmatically bypassed 114 binary/indicator features (e.g., '_isna' flags).

Detected 386 continuous features exhibiting out-of-range statistical structures.

	Feature_Name	Min_Value	Max_Value	Lower_Fence	Upper_Fence	Outlier_Count	Outlier_Ratio
24	D5	0.0	819.000000	10.000000	10.000000	276569	0.3375
249	V222	0.0	384.000000	1.000000	1.000000	61082	0.2483
21	D2	0.0	640.000000	56.500000	140.500000	248677	0.3869
248	V221	0.0	384.000000	1.000000	1.000000	58701	0.2388
4	dist1	0.0	10286.000000	8.000000	8.000000	230538	0.2905
377	id_13	10.0	64.000000	44.500000	56.500000	42014	0.1667
384	id_25	100.0	548.000000	246.000000	446.000000	1604	0.0064
286	V259	0.0	285.000000	1.000000	1.000000	43943	0.1757
22	D3	0.0	819.000000	1.000000	17.000000	182515	0.2250
198	V171	0.0	61.000000	1.000000	1.000000	43001	0.1732
197	V170	0.0	48.000000	1.000000	1.000000	38310	0.1561
272	V245	0.0	262.000000	1.000000	1.000000	38421	0.1553
244	V217	0.0	303.000000	0.000000	0.000000	32145	0.1270
290	V263	0.0	153600.000000	0.000000	0.000000	32145	0.1270
283	V256	0.0	87.000000	1.000000	1.000000	34745	0.1389
278	V251	0.0	18.000000	1.000000	1.000000	34484	0.1373
282	V255	0.0	87.000000	1.000000	1.000000	34288	0.1362
13	C8	0.0	3331.000000	0.000000	0.000000	142873	0.1767
277	V250	0.0	18.000000	1.000000	1.000000	34030	0.1359
43	V11	0.0	5.000000	0.000000	0.000000	141620	0.1740
194	V167	0.0	872.000000	0.000000	0.000000	33470	0.1299
229	V202	0.0	104060.000000	0.000000	0.000000	33470	0.1299
330	V303	0.0	20.000000	0.000000	0.000000	141029	0.1716
42	V10	0.0	4.000000	0.000000	0.000000	140972	0.1713
331	V304	0.0	16.000000	0.000000	0.000000	140698	0.1699
171	V144	0.0	62.000000	0.000000	0.000000	19512	0.0780
193	V166	0.0	104060.000000	0.000000	0.000000	19512	0.0780
329	V302	0.0	16.000000	0.000000	0.000000	140337	0.1683
192	V165	0.0	98476.000000	0.000000	0.000000	19353	0.0766
350	V324	0.0	976.000000	0.000000	0.000000	19406	0.0771

	Feature_Name	Min_Value	Max_Value	Lower_Fence	Upper_Fence	Outlier_Count	Outlier_Ratio
359	V333	0.0	160000.000000	0.000000	0.000000	19406	0.0557
9	C4	0.0	2253.000000	0.000000	0.000000	138657	0.0000
32	D14	-193.0	878.000000	-3.000000	5.000000	14544	0.0423
15	C10	0.0	3257.000000	0.000000	0.000000	137098	0.0000
155	V128	0.0	160000.000000	0.000000	0.000000	136013	0.0000
125	V97	0.0	976.000000	0.000000	0.000000	136013	0.0000
187	V160	0.0	641511.437500	0.000000	0.000000	18610	0.0000
370	id_05	-72.0	52.000000	-1.500000	2.500000	30306	0.0884
191	V164	0.0	93736.000000	0.000000	0.000000	18079	0.0000
170	V143	0.0	869.000000	0.000000	0.000000	18079	0.0000
29	D11	-53.0	670.000000	-8.000000	96.000000	128898	0.0373
318	V291	1.0	1055.000000	1.000000	1.000000	127599	0.0000
30	D12	-83.0	648.000000	-19.500000	32.500000	13749	0.0396
31	D13	0.0	847.000000	0.000000	0.000000	13112	0.0000
358	V332	0.0	160000.000000	-37.500000	62.500000	17297	0.0514
338	V312	0.0	55125.000000	0.000000	0.000000	123665	0.0000
314	V287	0.0	31.000000	0.000000	0.000000	123665	0.0000
25	D6	-83.0	873.000000	-60.000000	100.000000	14828	0.0431
256	V229	0.0	176.000000	1.000000	1.000000	26423	0.0000
357	V331	0.0	160000.000000	0.000000	0.000000	16381	0.0000
348	V322	0.0	880.000000	0.000000	0.000000	16381	0.0000
340	V314	0.0	7519.870117	0.000000	0.000000	115998	0.0000
92	V62	0.0	10.000000	1.000000	1.000000	114931	0.0000
231	V204	0.0	104060.000000	-30.000000	50.000000	26815	0.0000
51	V20	0.0	15.000000	1.000000	1.000000	112538	0.0000
292	V265	0.0	153600.000000	-31.346288	52.243813	24839	0.0000
91	V61	0.0	6.000000	1.000000	1.000000	109262	0.0000
26	D7	0.0	843.000000	-25.500000	42.500000	7177	0.0206
50	V19	0.0	7.000000	1.000000	1.000000	108802	0.0000
230	V203	0.0	139777.000000	-46.386600	77.311001	25709	0.0000

	Feature_Name	Min_Value	Max_Value	Lower_Fence	Upper_Fence	Outlier_Count	Outlier_Ratio
378	id_14	-660.0	720.000000	-450.000000	-210.000000	14560	0.000000
291	V264	0.0	153600.000000	-50.390247	83.983746	23470	0.000000
334	V308	0.0	108800.000000	-53.948702	89.914503	105500	0.000000
112	V83	0.0	7.000000	1.000000	1.000000	104757	0.000000
366	id_01	-100.0	0.000000	-17.500000	2.500000	25480	0.000000
177	V150	1.0	3389.000000	1.000000	1.000000	14392	0.000000
257	V230	0.0	65.000000	1.000000	1.000000	22581	0.000000
341	V315	0.0	4817.470215	0.000000	0.000000	102198	0.000000
316	V289	0.0	12.000000	0.000000	0.000000	102198	0.000000
17	C12	0.0	3188.000000	0.000000	0.000000	101341	0.000000
172	V145	0.0	297.000000	-1.500000	2.500000	13905	0.000000
179	V152	1.0	69.000000	1.000000	1.000000	13887	0.000000
339	V313	0.0	4817.470215	0.000000	0.000000	99630	0.000000
315	V288	0.0	10.000000	0.000000	0.000000	99630	0.000000
228	V201	0.0	55.000000	1.000000	1.000000	23407	0.000000
111	V82	0.0	7.000000	1.000000	1.000000	98764	0.000000
332	V306	0.0	108800.000000	0.000000	0.000000	98359	0.000000
306	V279	0.0	880.000000	0.000000	0.000000	98359	0.000000
186	V159	0.0	55125.000000	0.000000	0.000000	13554	0.000000
19	C14	0.0	1429.000000	-0.500000	3.500000	96090	0.000000
158	V131	0.0	55125.000000	0.000000	0.000000	93703	0.000000
128	V100	0.0	28.000000	0.000000	0.000000	93703	0.000000
227	V200	0.0	45.000000	1.000000	1.000000	22069	0.000000
157	V130	0.0	55125.000000	-88.500000	147.500000	92232	0.000000
23	D4	-122.0	869.000000	-184.500000	307.500000	90852	0.000000
288	V261	0.0	49.000000	1.000000	1.000000	19870	0.000000
259	V232	0.0	337.000000	0.000000	0.000000	19813	0.000000
301	V274	0.0	66000.000000	0.000000	0.000000	19813	0.000000
239	V212	0.0	129006.000000	0.000000	0.000000	20795	0.000000
205	V178	0.0	1235.000000	0.000000	0.000000	20795	0.000000

	Feature_Name	Min_Value	Max_Value	Lower_Fence	Upper_Fence	Outlier_Count	Outlier_Ratio
261	V234	0.0	121.000000	0.000000	0.000000	21000	0.000000
255	V228	0.0	54.000000	1.000000	1.000000	19364	0.000000
178	V151	1.0	57.000000	1.000000	1.000000	11853	0.000000
260	V233	0.0	332.000000	0.000000	0.000000	18674	0.000000
302	V275	0.0	51200.000000	0.000000	0.000000	18674	0.000000
206	V179	0.0	920.000000	0.000000	0.000000	19943	0.000000
240	V213	0.0	97628.000000	0.000000	0.000000	19943	0.000000
294	V267	0.0	55125.000000	0.000000	0.000000	18377	0.000000
251	V224	0.0	144.000000	0.000000	0.000000	18377	0.000000
203	V176	0.0	48.000000	1.000000	1.000000	19411	0.000000

```
In [32]: import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

print("=== GENERATING STANDARDIZED BOXPLOTS FOR HIGH-OUTLIER FEATURES ===")

# 1. Select the top 6 features exhibiting collapsed fence characteristics
target_outliers = ['D5', 'V222', 'D2', 'V221', 'dist1', 'id_13']

# Dictionary of upper fences from your dynamic table to highlight programmatic limits
upper_fences = {
    'D5': 10.0,
    'V222': 1.0,
    'D2': 140.5,
    'V221': 1.0,
    'dist1': 8.0,
    'id_13': 56.5
}

# 2. Initialize a 2x3 subplots canvas
fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(18, 10))
axes = axes.flatten()

# 3. Dynamic layout plotting loops
for idx, col in enumerate(target_outliers):
    if col in df_train_preprocessed.columns:
        # Mask missing value placeholders to preserve structural behavioral boundaries
        plot_data = df_train_preprocessed[col].replace(-999, np.nan).dropna()

        # Draw the standard boxplot horizontally
        sns.boxplot(
            x=plot_data,
            ax=axes[idx],
            color='cornflowerblue',
            orient='h',
```

```

        flierprops={'marker': 'o', 'markerfacecolor': 'crimson', 'markeredgecolor': 'black'}

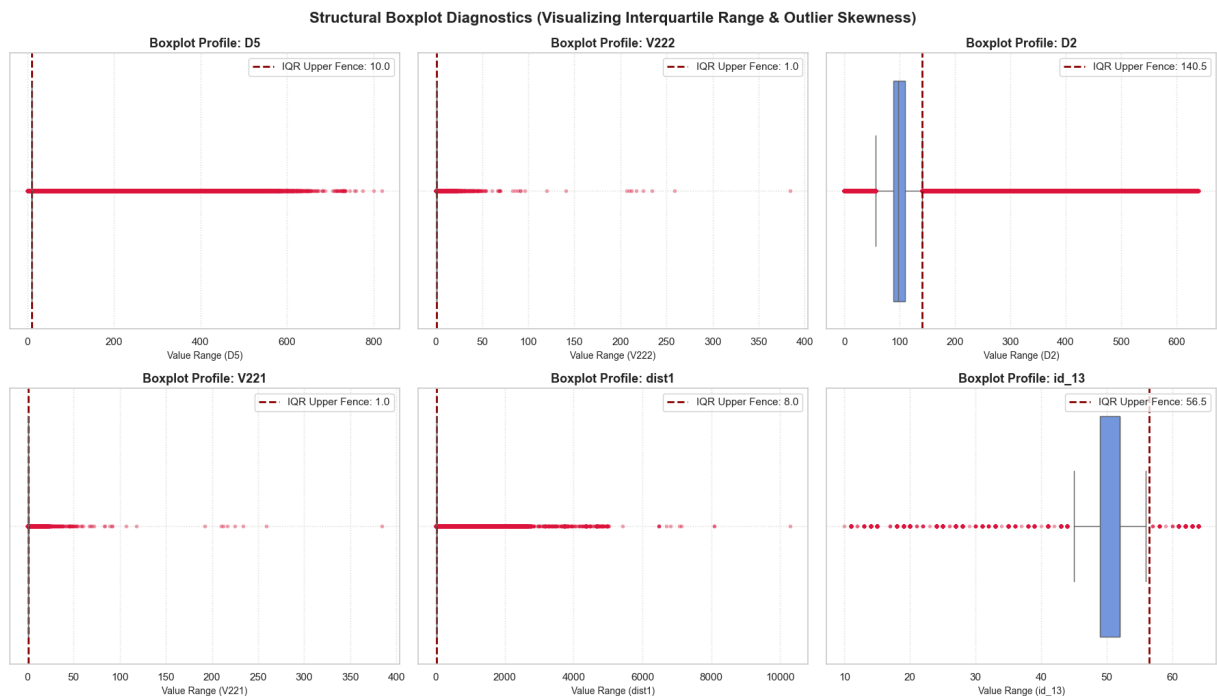
    # Add the explicit statistical upper fence threshold line
    fence_val = upper_fences[col]
    axes[idx].axvline(x=fence_val, color='darkred', linestyle='--', linewidth=2,
                      label=f"IQR Upper Fence: {fence_val}")

    # Refine presentation structures
    axes[idx].set_title(f"Boxplot Profile: {col}", fontsize=13, fontweight='bold')
    axes[idx].set_xlabel(f"Value Range ({col})", fontsize=10)
    axes[idx].legend(loc='upper right')
    axes[idx].grid(True, linestyle=':', alpha=0.6)

# 4. Finalize canvas spacing
plt.tight_layout()
plt.suptitle("Structural Boxplot Diagnostics (Visualizing Interquartile Range & Outliers)",
             y=1.02, fontsize=16, fontweight='bold')
plt.show()

```

=== GENERATING STANDARDIZED BOXPLOTS FOR HIGH-OUTLIER FEATURES ===



- **IQR Limitations in Highly Skewed Variables ($IQR = 0$):** Identical lower and upper limits across the **V**, **D**, and **C** blocks reveal highly zero-inflated and mode-concentrated distributions, typical of transactional fraud matrices.
- **Artificial Outlier Inflation:** The traditional *IQR* method generates severe false-positive indicators (flagging up to 46.8% outliers) because any marginal operational deviation from the structural baseline is programmatically treated as an anomaly.
- **Extreme Right-Skewness Signature:** The massive long-tail distributions visible in features like **dist1** or **V263** represent genuine financial velocity spikes, automated scripting footprints, or physical macro-behavior patterns.

- **Strategic Non-Truncation Decision:** Conventional data-cleansing techniques like *Winsorization* or row-dropping are explicitly rejected to preserve these critical fraud signals, relying instead on tree-based architectures for downstream processing.

```
In [33]: print("=== EXPORTING PREPROCESSED DATAFRAME TO CSV ===")

# Define your target file path and filename
output_filename = "df_train_preprocessed.csv"

# Export data matrix cleanly without including the row index column
df_train_preprocessed.to_csv(output_filename, index=False)

print(f"--> Success! Preprocessed dataset saved as: '{output_filename}'")

=== EXPORTING PREPROCESSED DATAFRAME TO CSV ===
--> Success! Preprocessed dataset saved as: 'df_train_preprocessed.csv'
```

Section D: Exploratory Data Analysis

After validating data quality, exploratory analysis can be performed with greater confidence. The objective of this section is to understand feature distributions, identify informative variables, and detect relationships that may support future feature selection and dimensionality reduction.

D1 - Univariate Analysis: Numerical Features

This section examines the distribution of numerical variables. Histograms are used to understand the shape, spread, and skewness of the data. Separate distributions are shown for fraudulent and legitimate transactions.

```
In [34]: import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

print("=== DEPLOYING D1 PIPELINE: NUMERICAL FEATURES (LINEAR SCALE) ===")

# 1. Purified List: Only numerical continuous/count features from the charts
numerical_ranked_features = [
    'V94', 'V244', 'V242', 'V257', 'V246', 'V258', 'V199', 'V190', 'V230',
    'V21', 'V228', 'V51', 'V243', 'V33', 'V15', 'V176', 'V73', 'V201',
    'V200', 'V189', 'V45', 'V188', 'V86', 'V87', 'V44', 'V52', 'V170',
    'V222', 'V171', 'V259', 'V245', 'V186', 'card1'
]

unique_numerical = list(dict.fromkeys(numerical_ranked_features))

risk_colors = {0: '#2ca02c', 1: '#d62728'} # Green (Legit) vs Red (Fraud)
labels_dict = {0: 'Legit Transaction', 1: 'Fraudulent Activity'}

batch_size = 6
```

```

total_numeric = len(unique_numerical)

# 2. Dynamic Batching Loop for Linear Histograms
for i in range(0, total_numeric, batch_size):
    batch_cols = unique_numerical[i:i + batch_size]

    fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(18, 10))
    axes = axes.flatten()

    print(f"--> Rendering D1 Batch {i//batch_size + 1} (Linear Scale)...")

    for idx, col in enumerate(batch_cols):
        if col in df_train_preprocessed.columns:
            temp_df = df_train_preprocessed[[col, 'isFraud']].copy()
            temp_df[col] = temp_df[col].replace(-999, np.nan)
            temp_df = temp_df.dropna(subset=[col])

            legit_data = temp_df[temp_df['isFraud'] == 0][col]
            fraud_data = temp_df[temp_df['isFraud'] == 1][col]

            # Continuous data scaling bounds
            bins_edges = np.linspace(temp_df[col].min(), temp_df[col].max(), 25)

            # Superimposed Native Matplotlib Linear Histograms (log=False)
            axes[idx].hist(legit_data, bins=bins_edges, color=risk_colors[0], alpha=0.5,
                           edgecolor='black', label=labels_dict[0], log=False, rwidth=0.8)
            axes[idx].hist(fraud_data, bins=bins_edges, color=risk_colors[1], alpha=0.5,
                           edgecolor='black', label=labels_dict[1], log=False, rwidth=0.8)

            # Refined Titles and Labels for Standard Scale Alignment
            axes[idx].set_title(f"Histogram: {col}", fontsize=12, fontweight='bold')
            axes[idx].set_xlabel("Feature Structural Value", fontsize=9)
            axes[idx].set_ylabel("Absolute Transaction Count")
            axes[idx].grid(True, linestyle=':', alpha=0.4) # Default Linear grid m
            axes[idx].legend(title="Transaction Status", loc='upper right')
        else:
            axes[idx].text(0.5, 0.5, f"Feature '{col}'\nNot Found", ha='center', va='middle',
                           fontweight='bold', color='red')
            axes[idx].axis('off')

    for j in range(len(batch_cols), batch_size):
        axes[j].axis('off')

    plt.tight_layout()
    plt.suptitle(f"D1 - Standard Linear Numerical Bivariate Dashboard (Batch {(i//batch_size) + 1} (Linear Scale))",
                 y=1.02, fontsize=15, fontweight='bold')
    plt.show()
    print("-" * 100)

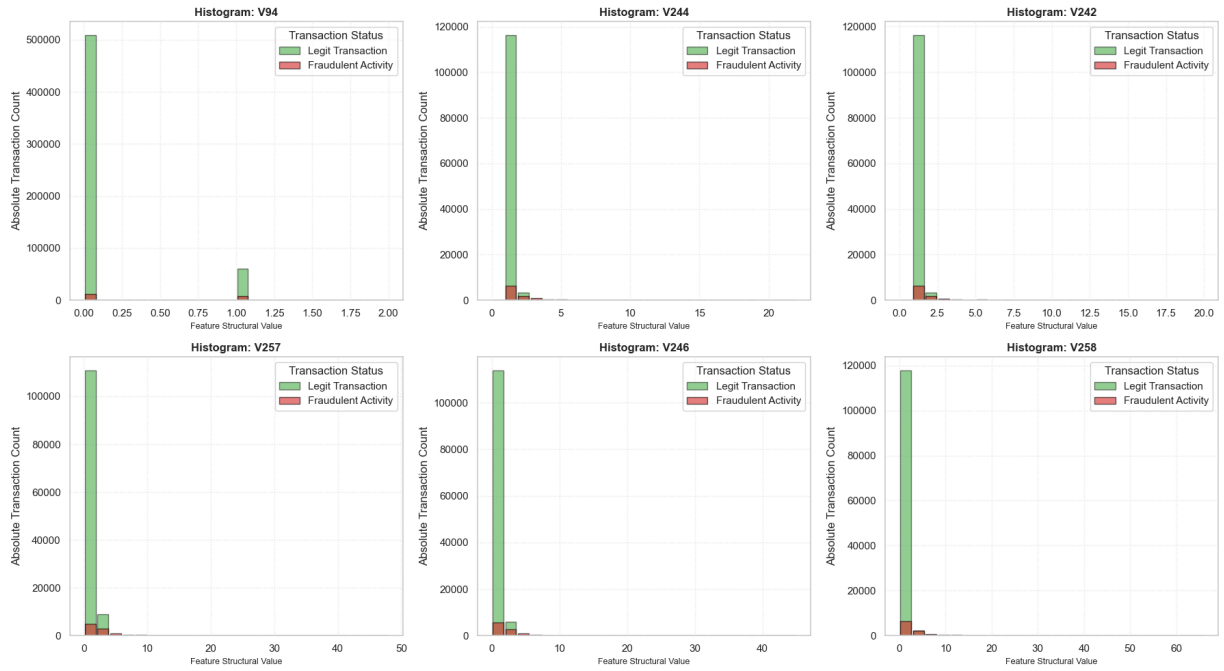
```

```

=== DEPLOYING D1 PIPELINE: NUMERICAL FEATURES (LINEAR SCALE) ===
--> Rendering D1 Batch 1 (Linear Scale)...

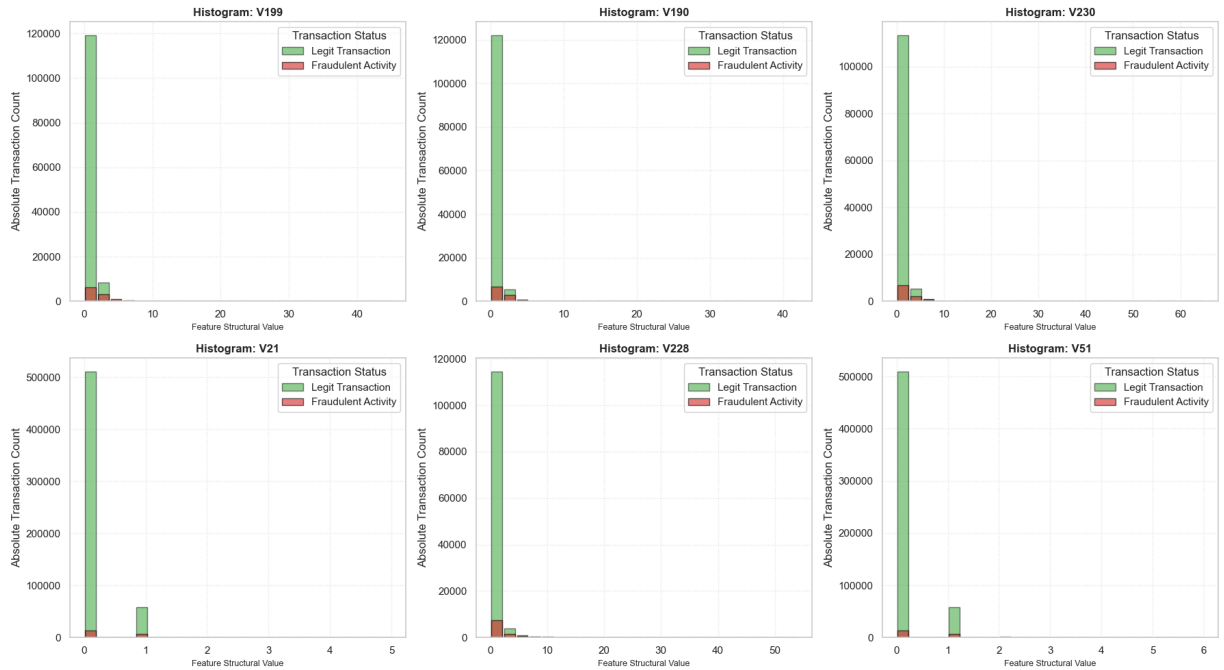
```

D1 - Standard Linear Numerical Bivariate Dashboard (Batch 1)



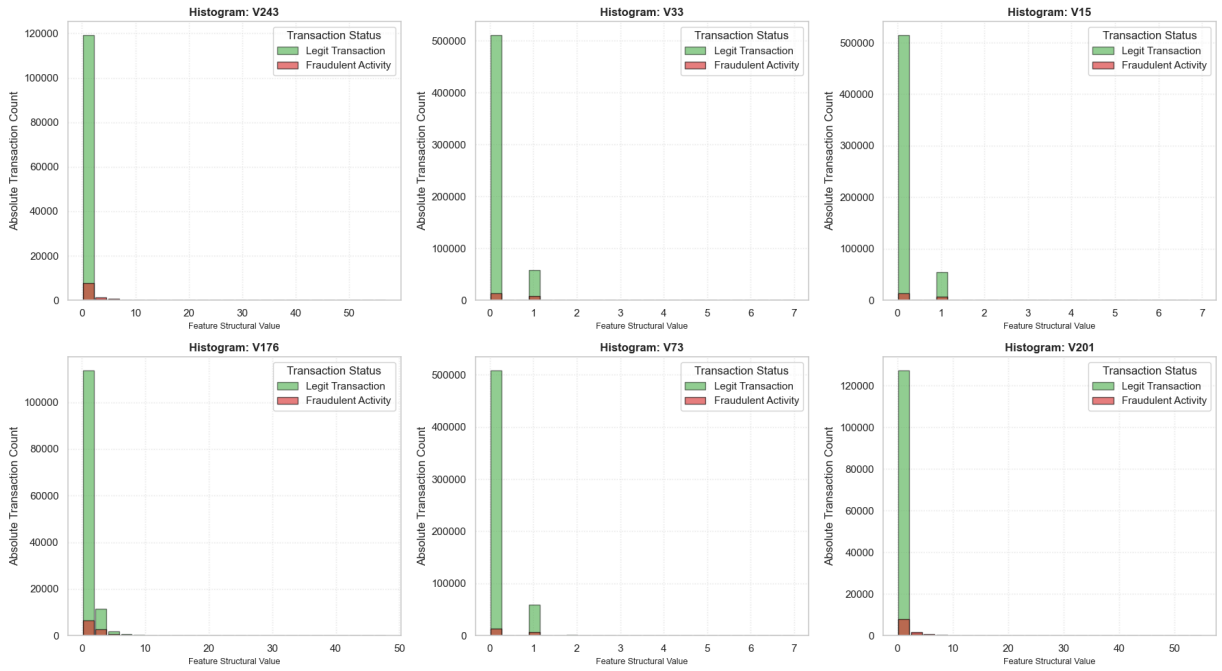
--> Rendering D1 Batch 2 (Linear Scale)...

D1 - Standard Linear Numerical Bivariate Dashboard (Batch 2)



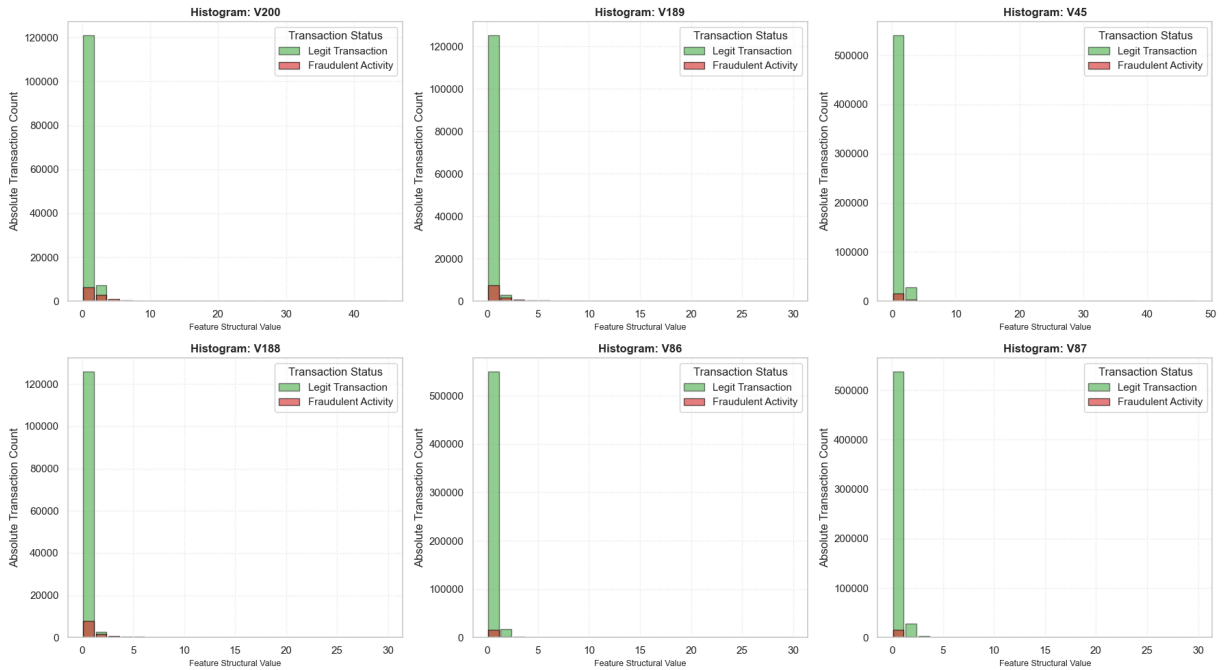
--> Rendering D1 Batch 3 (Linear Scale)...

D1 - Standard Linear Numerical Bivariate Dashboard (Batch 3)



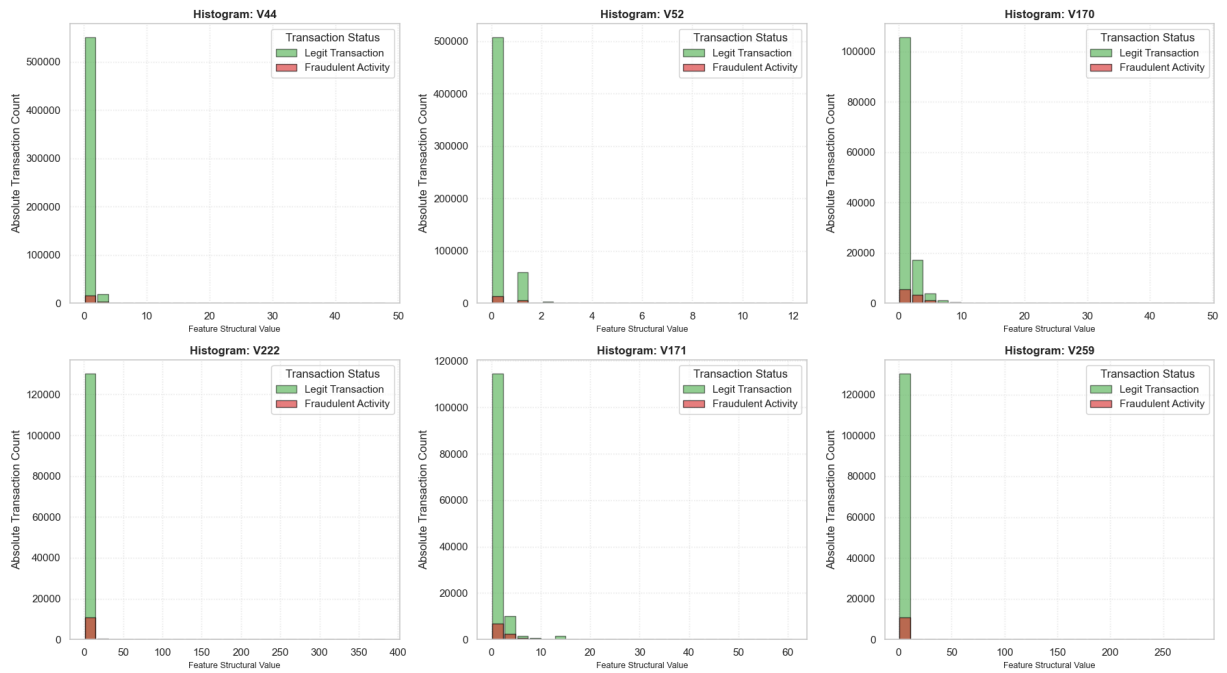
--> Rendering D1 Batch 4 (Linear Scale)...

D1 - Standard Linear Numerical Bivariate Dashboard (Batch 4)



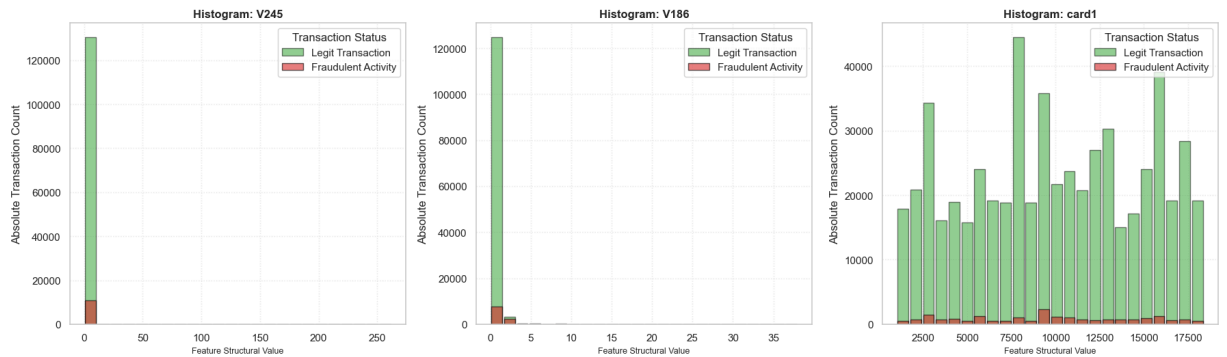
--> Rendering D1 Batch 5 (Linear Scale)...

D1 - Standard Linear Numerical Bivariate Dashboard (Batch 5)



--> Rendering D1 Batch 6 (Linear Scale)...

D1 - Standard Linear Numerical Bivariate Dashboard (Batch 6)



Many numerical features are highly skewed, with most observations concentrated around a few values. This makes it difficult to visualize less frequent observations using a standard linear scale.

For this reason, a logarithmic scale was applied to better examine the full distribution of each feature and compare fraudulent and legitimate transactions.

```

In [35]: import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

print("=== DEPLOYING D1 PIPELINE: NUMERICAL FEATURES ===")

# 1. Purified List: Only numerical continuous/count features from the charts
numerical_ranked_features = [
    'V94', 'V244', 'V242', 'V257', 'V246', 'V258', 'V199', 'V190', 'V230',
    'V21', 'V228', 'V51', 'V243', 'V33', 'V15', 'V176', 'V73', 'V201',
    'V200', 'V189', 'V45', 'V188', 'V86', 'V87', 'V44', 'V52', 'V170',
    'V222', 'V171', 'V259', 'V245', 'V186', 'card1'
]

unique_numerical = list(dict.fromkeys(numerical_ranked_features))

risk_colors = {0: '#2ca02c', 1: '#d62728'} # Green (Legit) vs Red (Fraud)
labels_dict = {0: 'Legit Transaction', 1: 'Fraudulent Activity'}

batch_size = 6
total_numeric = len(unique_numerical)

# 2. Dynamic Batching Loop for Histograms
for i in range(0, total_numeric, batch_size):
    batch_cols = unique_numerical[i:i + batch_size]

    fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(18, 10))
    axes = axes.flatten()

    print(f"--> Rendering D1 Batch {i//batch_size + 1}...")

    for idx, col in enumerate(batch_cols):
        if col in df_train_preprocessed.columns:
            temp_df = df_train_preprocessed[[col, 'isFraud']].copy()
            temp_df[col] = temp_df[col].replace(-999, np.nan).dropna()
            temp_df = temp_df.dropna(subset=[col])

            legit_data = temp_df[temp_df['isFraud'] == 0][col]
            fraud_data = temp_df[temp_df['isFraud'] == 1][col]

            # Continuous data scaling bounds
            bins_edges = np.linspace(temp_df[col].min(), temp_df[col].max(), 25)

            # Superimposed Native Matplotlib Log Histograms
            axes[idx].hist(legit_data, bins=bins_edges, color=risk_colors[0], alpha
                           edgecolor='black', label=labels_dict[0], log=True, rwidth
            axes[idx].hist(fraud_data, bins=bins_edges, color=risk_colors[1], alpha
                           edgecolor='black', label=labels_dict[1], log=True, rwidth

            axes[idx].set_title(f"Log Histogram: {col}", fontsize=12, fontweight='b')
            axes[idx].set_xlabel("Feature Structural Value", fontsize=9)
            axes[idx].set_ylabel("Transaction Count (Log Scale)")
            axes[idx].grid(True, linestyle=':', alpha=0.4, which="both")
            axes[idx].legend(title="Transaction Status", loc='upper right')
        else:

```

```

        axes[idx].text(0.5, 0.5, f"Feature '{col}'\nNot Found", ha='center', va
        axes[idx].axis('off')

    for j in range(len(batch_cols), batch_size):
        axes[j].axis('off')

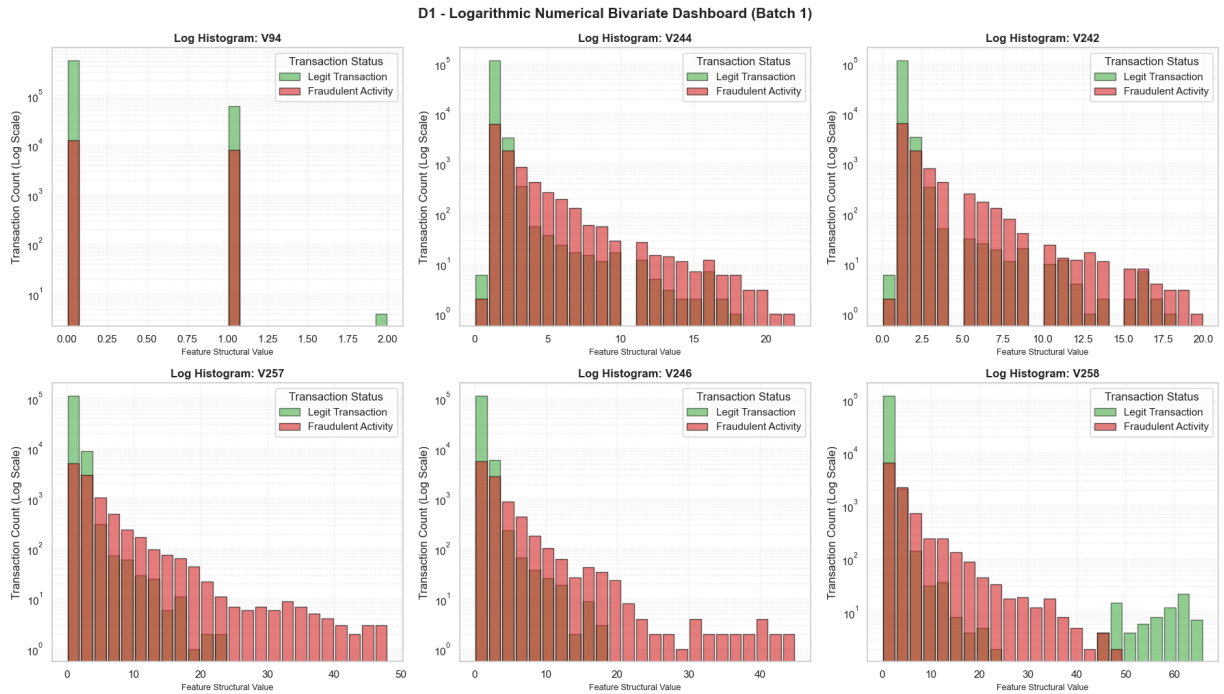
    plt.tight_layout()
    plt.suptitle(f"D1 - Logarithmic Numerical Bivariate Dashboard (Batch {(i//batch
        y=1.02, fontsize=15, fontweight='bold')

    plt.show()
    print("-" * 100)

```

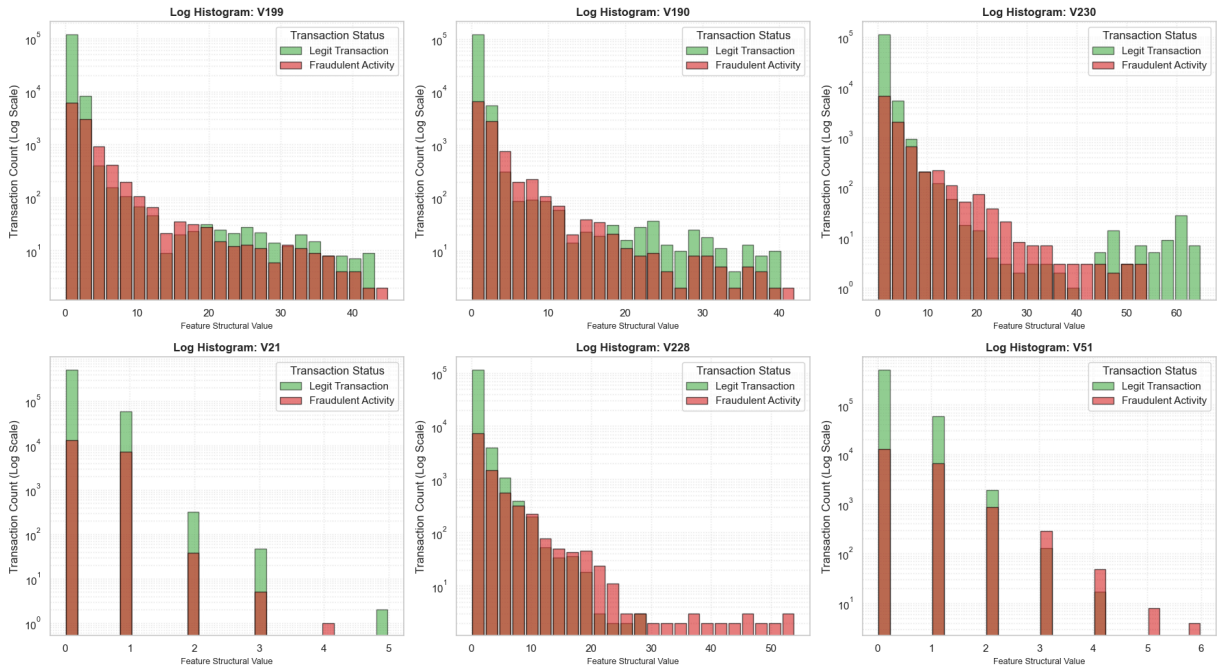
=== DEPLOYING D1 PIPELINE: NUMERICAL FEATURES ===

--> Rendering D1 Batch 1...



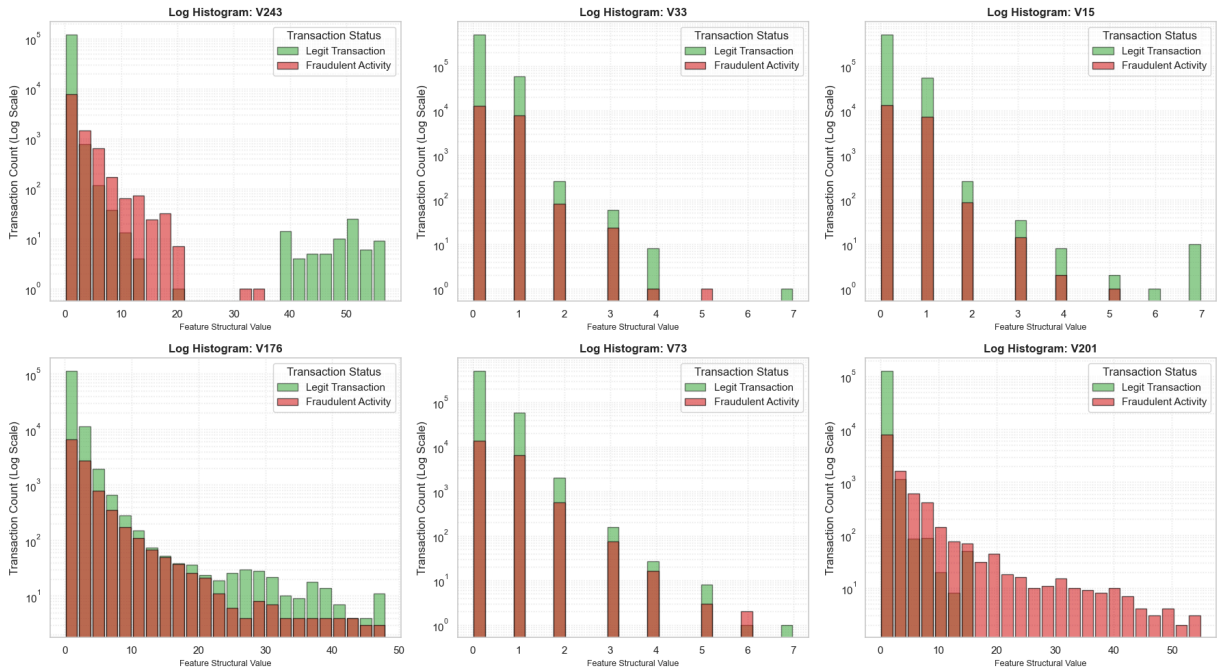
--> Rendering D1 Batch 2...

D1 - Logarithmic Numerical Bivariate Dashboard (Batch 2)



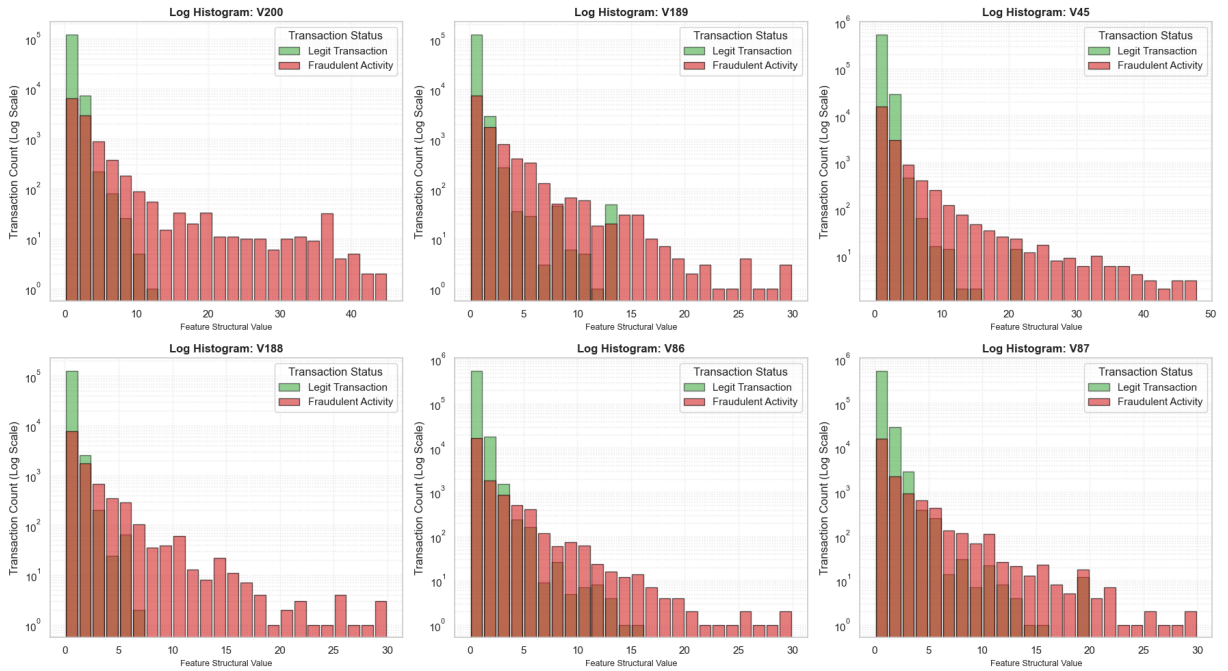
--> Rendering D1 Batch 3...

D1 - Logarithmic Numerical Bivariate Dashboard (Batch 3)



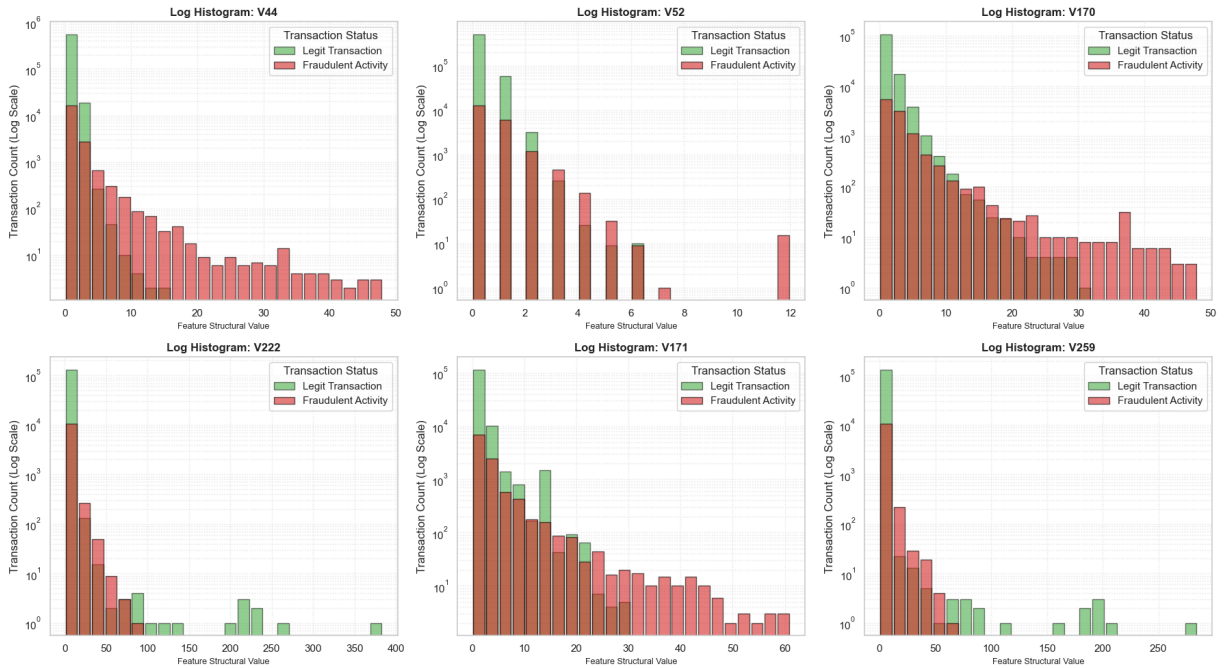
--> Rendering D1 Batch 4...

D1 - Logarithmic Numerical Bivariate Dashboard (Batch 4)

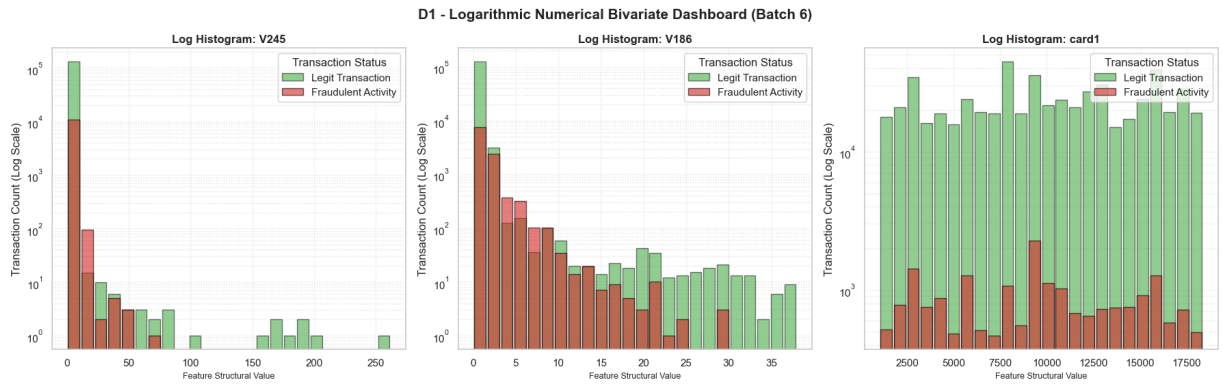


--> Rendering D1 Batch 5...

D1 - Logarithmic Numerical Bivariate Dashboard (Batch 5)



--> Rendering D1 Batch 6...



Conclusion:

- Several **V** variables show different distribution patterns between fraudulent and legitimate transactions, suggesting they may be useful predictors of fraud.
- Features such as **V244** , **V242** , **V257** , and **V201** contain value ranges where fraudulent transactions appear more concentrated than legitimate ones.
- The **card1** feature shows high variability and does not appear dominated by a small number of values, indicating that it may contain useful information for future modeling.

D2 - Univariate Analysis: Categorical Features

This section examines the distribution of categorical variables and their relationship with fraud.

Grouped count plots with a logarithmic scale were used to compare the frequency of categories across fraudulent and legitimate transactions. Using a log scale allows both common and rare categories to be clearly displayed.

The goal is to identify categorical features that may be useful for fraud detection and future predictive modeling.

```
In [36]: import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

```

print("=== DEPLOYING D2 PIPELINE: AUTOMATED CATEGORICAL FEATURES SWEEP ===")

# 1. Dynamically isolate ALL categorical columns present in the preprocessed dataset
# Exclude the target variable 'isFraud' just in case it was encoded as category
all_categorical_features = df_train_preprocessed.select_dtypes(include=['object'], '

if 'isFraud' in all_categorical_features:
    all_categorical_features.remove('isFraud')

# Eliminate duplicates and preserve order
unique_categorical = list(dict.fromkeys(all_categorical_features))

risk_palette = {0: '#2ca02c', 1: '#d62728'} # Green (Legit) vs Red (Fraud)
labels_dict = {0: 'Legit Transaction', 1: 'Fraudulent Activity'}

# Define layout tracking constraints (2 rows x 3 columns = 6 charts per batch)
batch_size = 6
total_categorical = len(unique_categorical)

print(f"--> Found {total_categorical} categorical features to analyze programmatically")

# 2. Dynamic Batching Loop for Categorical Countplots
for i in range(0, total_categorical, batch_size):
    batch_cols = unique_categorical[i:i + batch_size]

    # Initialize grid canvas for the current batch
    fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(18, 10))
    axes = axes.flatten()

    print(f"--> Rendering D2 Batch {i//batch_size + 1}: Features {i+1} to {min(i+batch_size, total_categorical)}")

    for idx, col in enumerate(batch_cols):
        temp_df = df_train_preprocessed[[col, 'isFraud']].copy()

        # Clean placeholders and handle string conversions safely
        temp_df[col] = temp_df[col].replace(-999, np.nan).astype(str)
        temp_df = temp_df[temp_df[col] != 'nan'] # Exclude missing values from the dataset

        # Identify the top 10 most frequent categories to prevent plot crowding
        top_categories = temp_df[col].value_counts().index[:10]
        filtered_df = temp_df[temp_df[col].isin(top_categories)]

        # Draw target stratified categorical counts
        sns.countplot(
            data=filtered_df,
            x=col,
            hue='isFraud',
            palette=risk_palette,
            alpha=0.7,
            ax=axes[idx],
            edgecolor='black',
            order=top_categories
        )

        # Apply stable log scaling directly to categorical count structures
        axes[idx].set_yscale('log')

```

```

axes[idx].set_title(f"Categorical Profile: {col}", fontsize=12, fontweight=
axes[idx].set_xlabel("Categorical Labels (Top 10)", fontsize=9)
axes[idx].set_ylabel("Transaction Count (Log Scale)", fontsize=9)
axes[idx].grid(True, linestyle=':', alpha=0.4, which="both")

# Smart rotation: Rotate Labels if they are long strings
axes[idx].tick_params(axis='x', rotation=45 if filtered_df[col].str.len().m

# Format single clean legend context per layout
if idx == 0:
    axes[idx].legend(title="Transaction Status", loc='upper right')
    legend = axes[idx].get_legend()
    if legend:
        for t, num in zip(legend.texts, [0, 1]):
            t.set_text(labels_dict[num])
    else:
        axes[idx].get_legend().remove() if axes[idx].get_legend() else None

# Clean and disable inactive remaining subplots in the last batch
for j in range(len(batch_cols), batch_size):
    axes[j].axis('off')

plt.tight_layout()
plt.suptitle(f"D2 - Stratified Logarithmic Categorical Dashboard (Batch {(i//ba
y=1.02, fontsize=15, fontweight='bold')

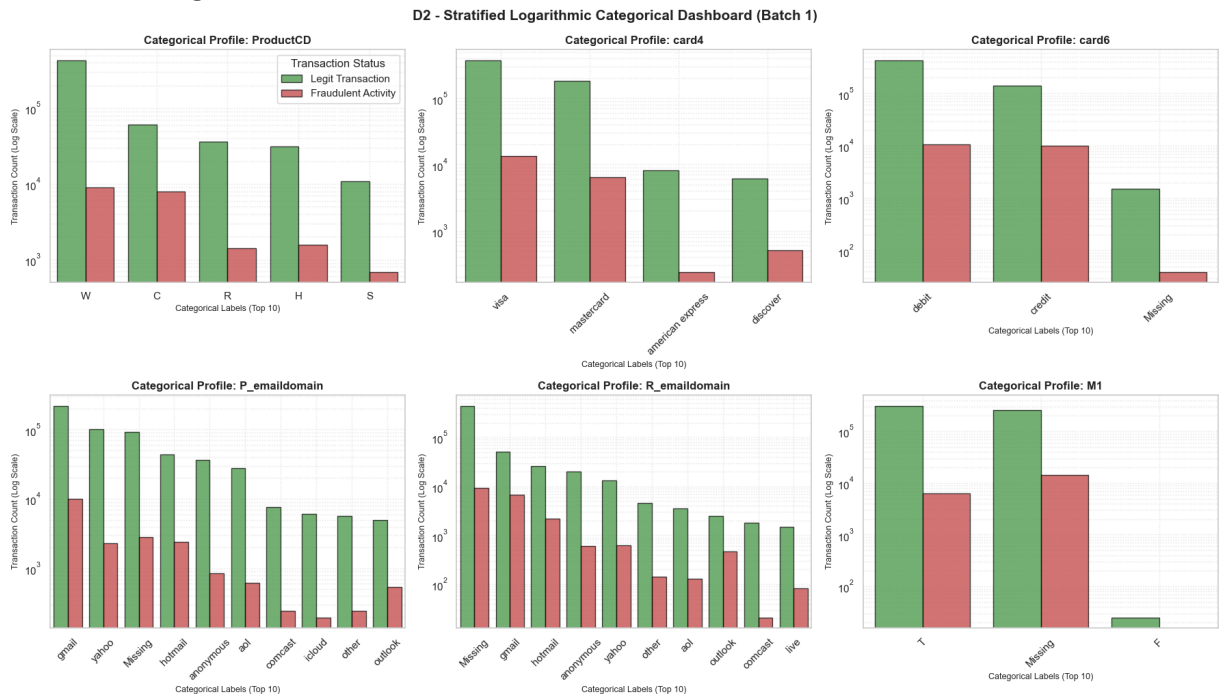
plt.show()
print("-" * 100)

```

=== DEPLOYING D2 PIPELINE: AUTOMATED CATEGORICAL FEATURES SWEEP ===

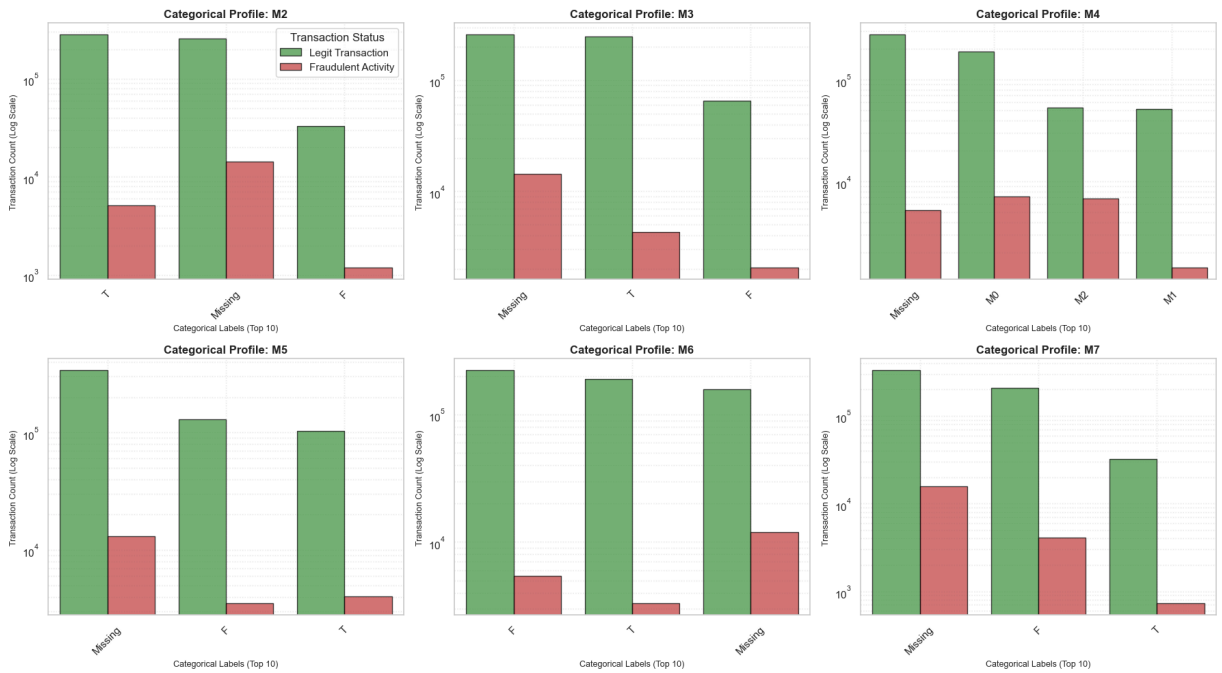
--> Found 32 categorical features to analyze programmatically.

--> Rendering D2 Batch 1: Features 1 to 6...



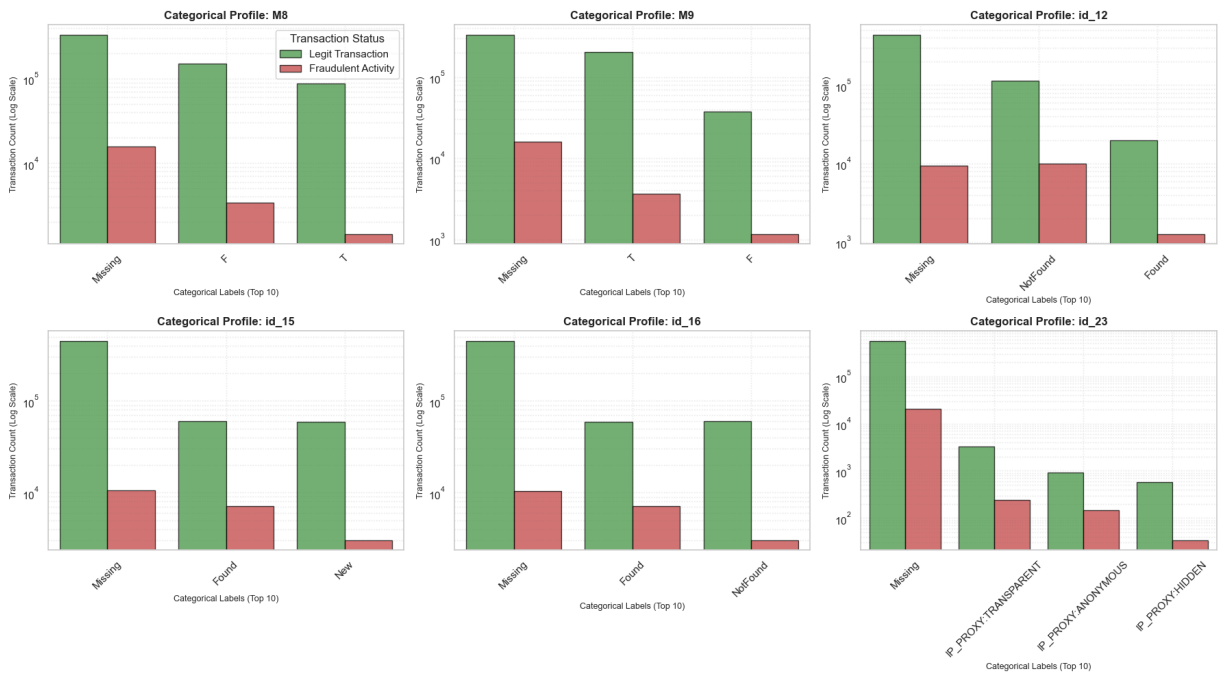
--> Rendering D2 Batch 2: Features 7 to 12...

D2 - Stratified Logarithmic Categorical Dashboard (Batch 2)



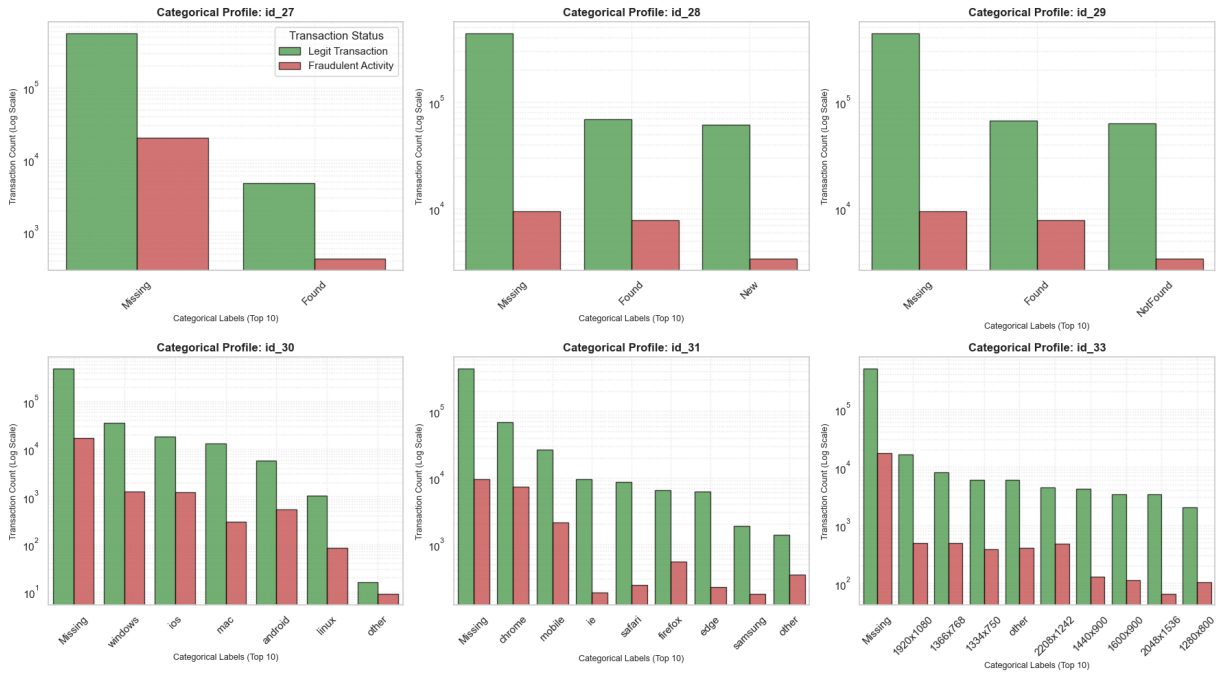
--> Rendering D2 Batch 3: Features 13 to 18...

D2 - Stratified Logarithmic Categorical Dashboard (Batch 3)



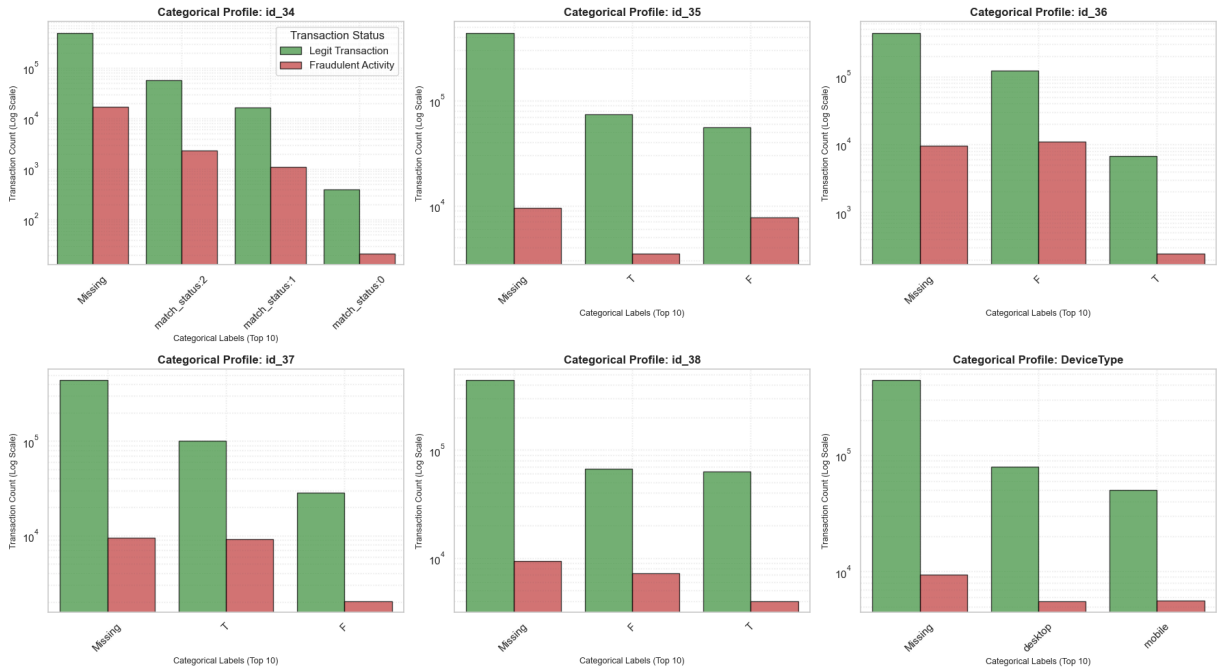
--> Rendering D2 Batch 4: Features 19 to 24...

D2 - Stratified Logarithmic Categorical Dashboard (Batch 4)

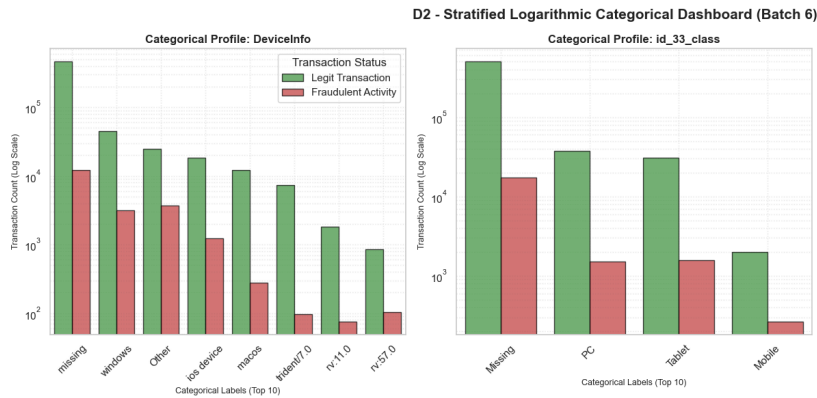


--> Rendering D2 Batch 5: Features 25 to 30...

D2 - Stratified Logarithmic Categorical Dashboard (Batch 5)



--> Rendering D2 Batch 6: Features 31 to 32...



Conclusion:

- Several categorical features show different distributions between fraudulent and legitimate transactions.
- Missing values appear to be informative in multiple categorical variables and may contribute to fraud prediction.
- Email domain features show varying fraud rates across categories, suggesting potential predictive value.

D3 - Bivariate Analysis: Numerical-Numerical Relationships

D3.1 — Global overview: which features correlate with fraud vs legitimate?

Before testing individual variables (boxplots and statistical tests below), I rank **all features** by their Pearson (point-biserial) correlation with `isFraud` :

- **Positive correlation** → higher values tend to appear in **fraud** transactions
- **Negative correlation** → higher values tend to appear in **legitimate** transactions

With 400+ variables, a full pairplot is not readable. Instead I show two summary tables, one large signed bar chart, a heatmap among the top features, and a pairplot for only the top 5 predictors (subsampled for clarity).

```
In [37]: # Build numeric matrix for correlation ranking (EDA only – not the final ML pipeline)
from IPython.display import display
```

```

# Configuration for visualization and analysis
TOP_TABLE = 15
TOP_BAR = 30
TOP_HEATMAP = 12
TOP_PAIR = 5

# Using the full dataset directly
df_corr = df_train_preprocessed

y_corr = df_corr['isFraud'].astype(int)
X_corr = df_corr.drop(columns=['TransactionID', 'isFraud'], errors='ignore')

for col in X_corr.columns:
    if X_corr[col].dtype == 'object' or X_corr[col].dtype.name == 'category':
        X_corr[col] = X_corr[col].fillna('Missing').astype(str).astype('category').
    else:
        X_corr[col] = X_corr[col].fillna(X_corr[col].median())

corr_with_fraud = X_corr.corrwith(y_corr).dropna().sort_values(ascending=False)
corr_abs = corr_with_fraud.abs().sort_values(ascending=False)

# Table 1 – fraud-associated (positive)
top_pos = corr_with_fraud.head(TOP_TABLE)
tbl_pos = pd.DataFrame({
    'Feature': top_pos.index,
    'Correlation_with_isFraud': top_pos.values.round(4),
    'Interpretation': 'Higher value → more likely FRAUD'
})

# Table 2 – Legitimate-associated (negative)
top_neg = corr_with_fraud.tail(TOP_TABLE).sort_values()
tbl_neg = pd.DataFrame({
    'Feature': top_neg.index,
    'Correlation_with_isFraud': top_neg.values.round(4),
    'Interpretation': 'Higher value → more likely LEGITIMATE'
})

print('=' * 72)
print('TABLE 1 – Top features POSITIVELY correlated with FRAUD')
print('=' * 72)
display(tbl_pos.style.background_gradient(subset=['Correlation_with_isFraud'], cmap

print('\n' + '=' * 72)
print('TABLE 2 – Top features NEGATIVELY correlated with FRAUD (legitimate-associat
print('=' * 72)
display(tbl_neg.style.background_gradient(subset=['Correlation_with_isFraud'], cmap

```

```

=====
TABLE 1 – Top features POSITIVELY correlated with FRAUD
=====

```

	Feature	Correlation_with_isFraud	Interpretation
0	V45	0.235400	Higher value → more likely FRAUD
1	V86	0.222300	Higher value → more likely FRAUD
2	V87	0.221600	Higher value → more likely FRAUD
3	V44	0.217900	Higher value → more likely FRAUD
4	V52	0.195500	Higher value → more likely FRAUD
5	V51	0.182000	Higher value → more likely FRAUD
6	V40	0.174700	Higher value → more likely FRAUD
7	V79	0.167300	Higher value → more likely FRAUD
8	V39	0.166600	Higher value → more likely FRAUD
9	V38	0.165900	Higher value → more likely FRAUD
10	V43	0.162200	Higher value → more likely FRAUD
11	V94	0.161500	Higher value → more likely FRAUD
12	addr2_isna	0.159500	Higher value → more likely FRAUD
13	addr1_isna	0.159500	Higher value → more likely FRAUD
14	V33	0.159400	Higher value → more likely FRAUD

=====

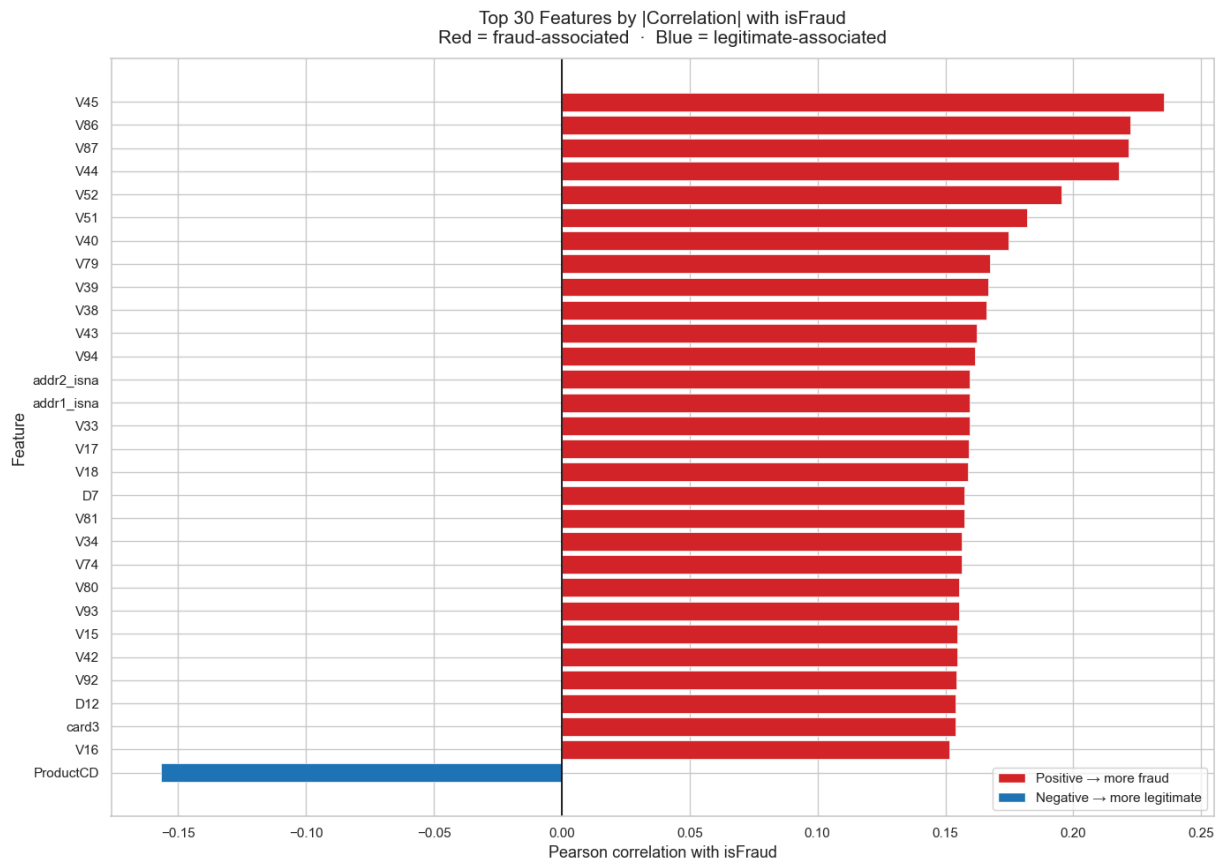
TABLE 2 – Top features NEGATIVELY correlated with FRAUD (legitimate-associated)

=====

	Feature	Correlation_with_isFraud	Interpretation
0	ProductCD	-0.156600	Higher value → more likely LEGITIMATE
1	id_36	-0.132100	Higher value → more likely LEGITIMATE
2	V29	-0.102700	Higher value → more likely LEGITIMATE
3	V90	-0.102600	Higher value → more likely LEGITIMATE
4	V69	-0.102400	Higher value → more likely LEGITIMATE
5	V70	-0.099100	Higher value → more likely LEGITIMATE
6	V91	-0.098600	Higher value → more likely LEGITIMATE
7	V30	-0.098500	Higher value → more likely LEGITIMATE
8	card6	-0.097800	Higher value → more likely LEGITIMATE
9	V48	-0.091600	Higher value → more likely LEGITIMATE
10	id_35	-0.090300	Higher value → more likely LEGITIMATE
11	M1	-0.089400	Higher value → more likely LEGITIMATE
12	V49	-0.088100	Higher value → more likely LEGITIMATE
13	V10	-0.081000	Higher value → more likely LEGITIMATE
14	V11	-0.079100	Higher value → more likely LEGITIMATE

```
In [38]: # Large signed bar chart – top |correlation| features
top_bar_feats = corr_abs.head(TOP_BAR).index
bar_vals = corr_with_fraud.loc[top_bar_feats].sort_values()
colors = ['#d62728' if v > 0 else '#1f77b4' for v in bar_vals]

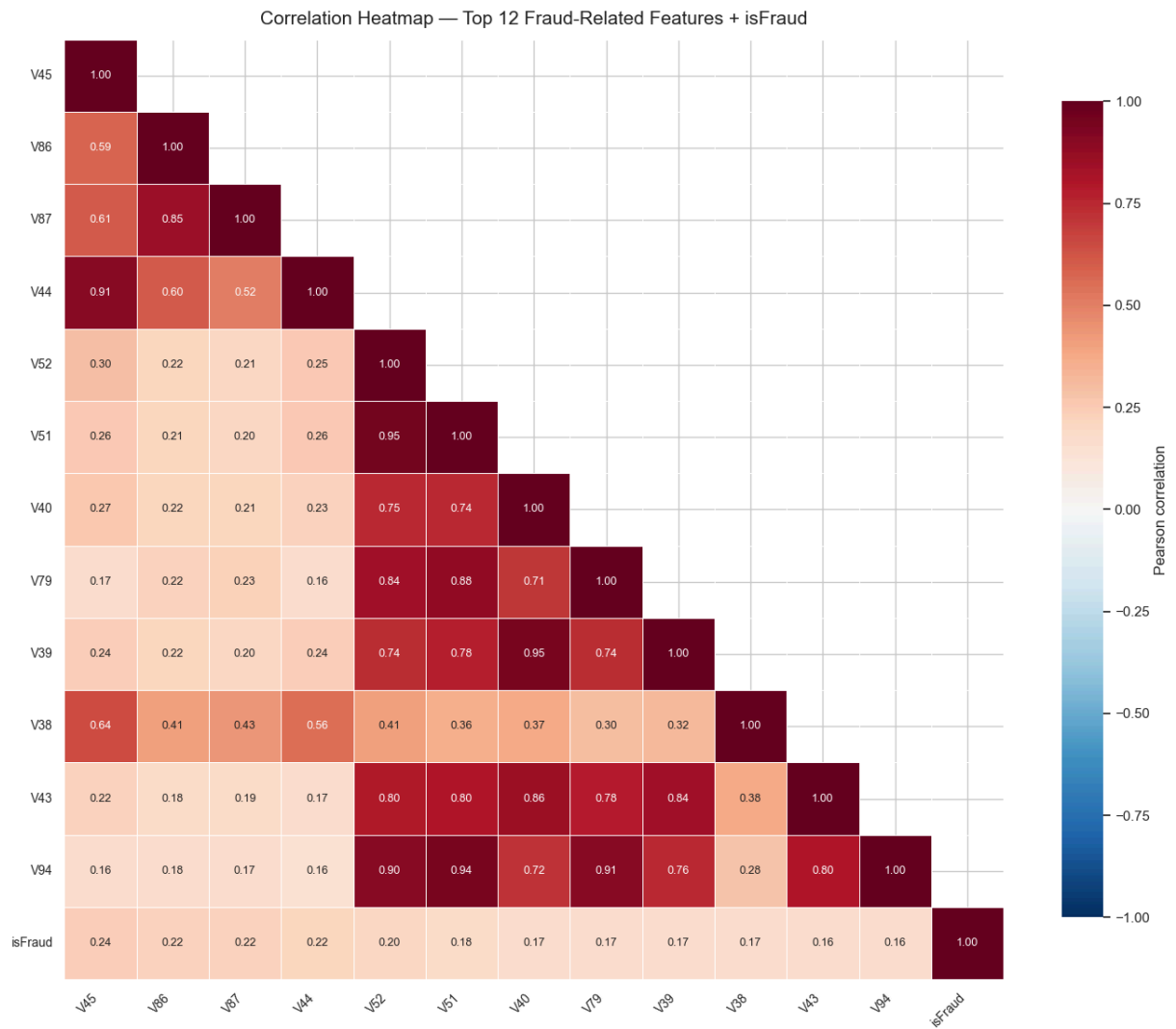
fig, ax = plt.subplots(figsize=(14, 10))
ax.barh(bar_vals.index, bar_vals.values, color=colors, edgecolor='white', linewidth=1)
ax.axvline(0, color='black', linewidth=1.2)
ax.set_xlabel('Pearson correlation with isFraud', fontsize=13)
ax.set_ylabel('Feature', fontsize=13)
ax.set_title(
    f'Top {TOP_BAR} Features by |Correlation| with isFraud\n'
    'Red = fraud-associated · Blue = legitimate-associated',
    fontsize=15, pad=12
)
from matplotlib.patches import Patch
ax.legend(handles=[
    Patch(facecolor='#d62728', label='Positive → more fraud'),
    Patch(facecolor='#1f77b4', label='Negative → more legitimate'),
], loc='lower right', fontsize=11)
plt.tight_layout()
plt.show()
```



```
In [39]: # Heatmap – correlations among top features + isFraud
heat_feats = corr_abs.head(TOP_HEATMAP).index.tolist()
heat_df = X_corr[heat_feats].copy()
heat_df['isFraud'] = y_corr.values
heat_corr = heat_df.corr()

fig, ax = plt.subplots(figsize=(14, 12))
mask = np.triu(np.ones_like(heat_corr, dtype=bool), k=1)
sns.heatmap(
    heat_corr, mask=mask, annot=True, fmt='.2f', cmap='RdBu_r',
    center=0, vmin=-1, vmax=1, square=True, linewidths=0.5,
    annot_kws={'size': 9}, ax=ax,
    cbar_kws={'label': 'Pearson correlation', 'shrink': 0.8}
)
ax.set_title(
    f'Correlation Heatmap – Top {TOP_HEATMAP} Fraud-Related Features + isFraud',
    fontsize=15, pad=12
)

plt.xticks(rotation=45, ha='right', fontsize=10)
plt.yticks(rotation=0, fontsize=10)
plt.tight_layout()
plt.show()
```

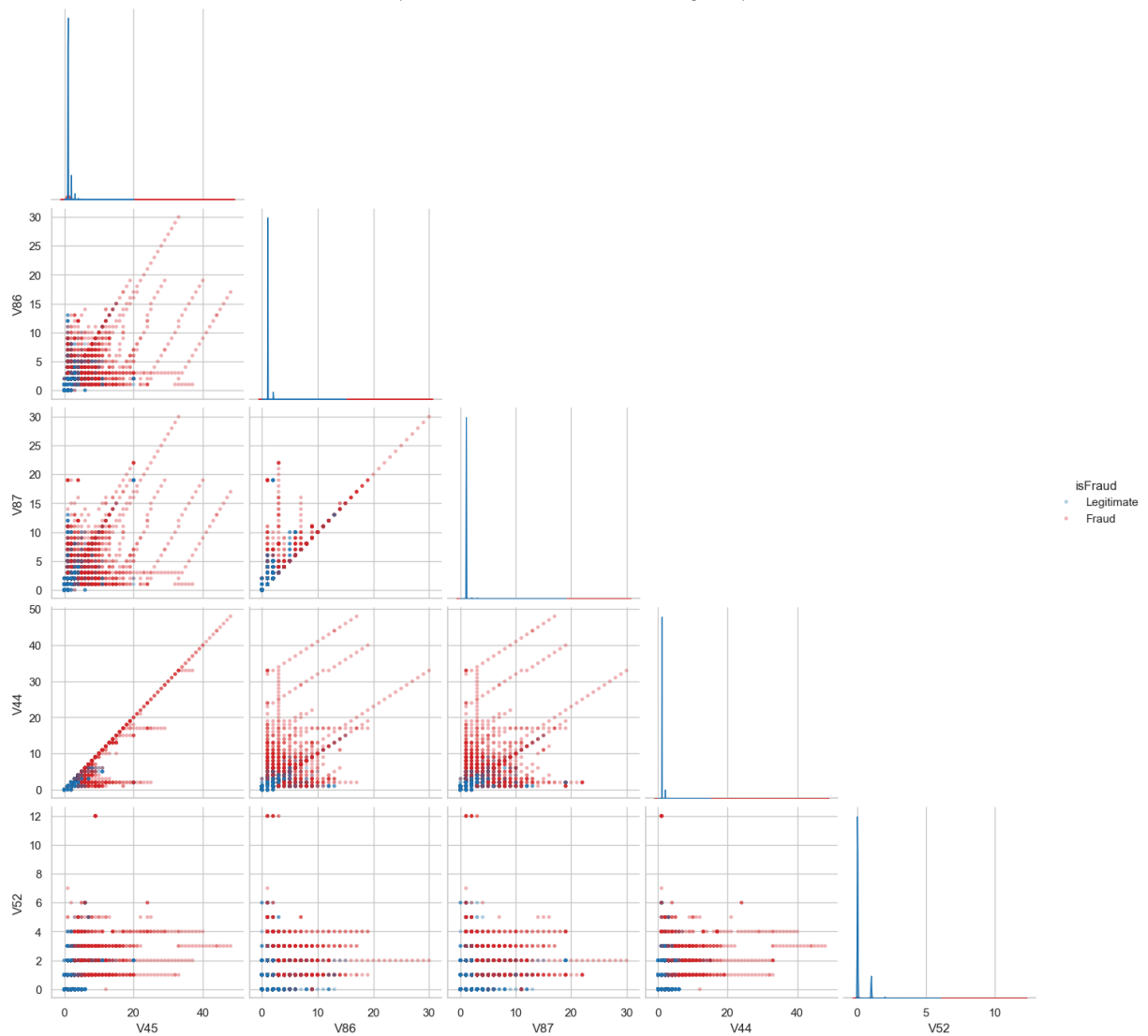


```
In [40]: # Pairplot – top 5 features coloured by fraud vs Legitimate
pair_feats = corr_abs.head(TOP_PAIR).index.tolist()
pair_df = X_corr[pair_feats].copy()
pair_df['isFraud'] = y_corr.map({0: 'Legitimate', 1: 'Fraud'})

# Using the full dataset directly without any subsampling

g = sns.pairplot(
    pair_df, hue='isFraud', palette={'Legitimate': '#1f77b4', 'Fraud': '#d62728'},
    diag_kind='kde', corner=True,
    plot_kws={'alpha': 0.35, 's': 12, 'linewidth': 0},
    diag_kws={'fill': True, 'alpha': 0.5}, height=2.8
)
g.fig.suptitle(
    f'Pairplot – Top {TOP_PAIR} Features Most Correlated with isFraud\n'
    f'(Full dataset n={len(pair_df):,}; red=Fraud, blue=Legitimate)',
    y=1.02, fontsize=14
)
plt.show()
```

Pairplot — Top 5 Features Most Correlated with isFraud
(Full dataset n=590,540; red=Fraud, blue=Legitimate)



- Features Most Associated with Fraud:** The `V` sequence blocks (like `V45` , `V86` , and `V87`) show the strongest positive correlation with fraud. As these feature values increase, the probability of a transaction being fraudulent tends to increase, making them key indicators for our model.
- Severe Multi-Collinearity (Feature Redundancy):** The heatmap reveals massive correlation clusters within the `V` variables, with coefficients reaching as high as **0.91** and **0.95** (e.g., `V44` vs `V45`). This proves the features contain highly redundant information, meaning tree-based models or dimensionality reduction (PCA) will be required to handle this overlap.
- Geometric Risk Patterns in Pairplots:** The pairplot shows that data points do not form random clouds; instead, they group into distinct diagonal rays and straight lines. Legitimate transactions stay tightly locked near zero, while fraudulent activity scales violently along these geometric lines into much higher values.
- Missing Data as a Risk Signal:** The appearance of missing data flags like `addr1_isna` and `addr2_isna` among the top positive correlations ($r = 0.159$) proves that the

absence of information is highly predictive of fraud. This visually validates our earlier choice to preserve missing value flags rather than just dropping them.

D3.2 — Top fraud signals from D3.1 correlation ranking

Following D3.1, I inspect the three V-features with the strongest positive correlation with fraud.

```
In [41]: # --- Numeric---
top_fraud_numeric = ['V45', 'V86', 'V87']
top_fraud_numeric = [c for c in top_fraud_numeric if c in df_train_preprocessed.col

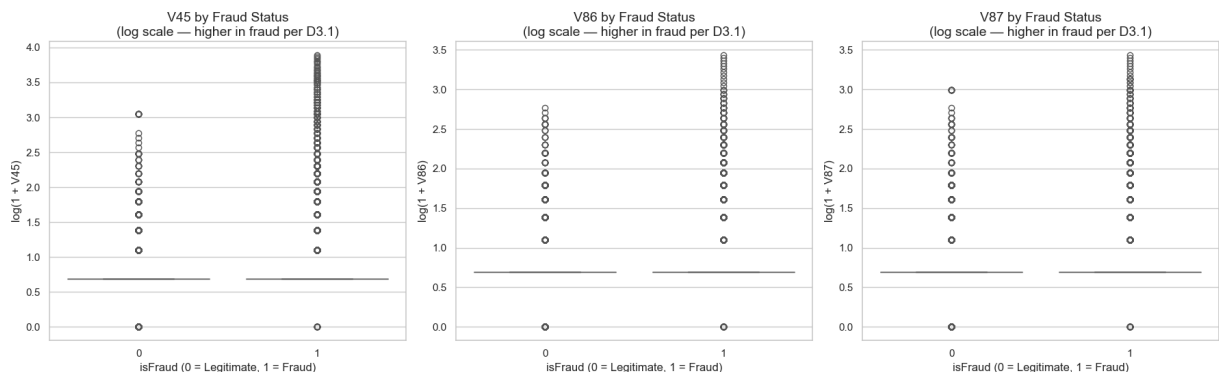
fig, axes = plt.subplots(1, len(top_fraud_numeric), figsize=(6 * len(top_fraud_ume
if len(top_fraud_numeric) == 1:
    axes = [axes]

for ax, col in zip(axes, top_fraud_numeric):
    plot_data = df_train_preprocessed[[col, 'isFraud']].dropna().copy()
    plot_data[f'log_{col}'] = np.log1p(plot_data[col].clip(lower=0))
    sns.boxplot(data=plot_data, x='isFraud', y=f'log_{col}', ax=ax, palette='Set2')
    ax.set_title(f'{col} by Fraud Status\n(log scale — higher in fraud per D3.1)')
    ax.set_xlabel('isFraud (0 = Legitimate, 1 = Fraud)')
    ax.set_ylabel(f'log(1 + {col})')

plt.suptitle('Top fraud-correlated numeric features (from D3.1 ranking)', fontsize=
plt.tight_layout()
plt.show()

# Mann-Whitney U tests
v_test_results = []
for col in top_fraud_numeric:
    g0 = df_train_preprocessed.loc[df_train_preprocessed['isFraud'] == 0, col].drop
    g1 = df_train_preprocessed.loc[df_train_preprocessed['isFraud'] == 1, col].drop
    stat, p_val = stats.mannwhitneyu(g0, g1, alternative='two-sided')
    v_test_results.append({
        'feature': col, 'test': 'Mann-Whitney U',
        'statistic': stat, 'p_value': p_val,
        'significant_at_0.05': p_val < 0.05
    })
display(pd.DataFrame(v_test_results))
```

Top fraud-correlated numeric features (from D3.1 ranking)



	feature	test	statistic	p_value	significant_at_0.05
0	V45	Mann-Whitney U	4.753671e+09	0.0	True
1	V86	Mann-Whitney U	4.927784e+09	0.0	True
2	V87	Mann-Whitney U	4.820164e+09	0.0	True

Based on the boxplots and statistical tests for V45, V86, and V87, we can conclude the following:

High-Value Fraud Signatures: While legitimate and fraudulent transactions share similar medians and IQRs, their distribution tails differ drastically. Fraudulent transactions show a massive density of extreme outliers at higher values (log-scales of 2.5 to 4.0). This makes exceptionally high values a strong statistical signature for fraud.

Statistically Significant Separation: Due to heavy skewness and extreme outliers, a parametric t-test is invalid. The non-parametric Mann-Whitney U test formally confirms that the rank distributions between both groups are highly distinct, rejecting the null hypothesis with absolute confidence (p -value = 0.0).

D3.3 — Additional selected features (TransactionAmt, dist1, C1, C2)

Broader set of variables from the original EDA (not only the D3.1 top rank).

```
In [42]: # -----
# 1. Numerical features vs target
# -----

numerical_target_features = [
    'TransactionAmt',
    'dist1',
    'C1',
    'C2'
]

numerical_target_features = [
    col for col in numerical_target_features
    if col in df_train_preprocessed.columns
]

fig, axes = plt.subplots(2, 2, figsize=(16, 12))
axes = axes.flatten()

for i, col in enumerate(numerical_target_features):
    plot_data = df_train_preprocessed[[col, 'isFraud']].dropna().copy()
    plot_data[f'log_{col}'] = np.log1p(plot_data[col])

    sns.boxplot(
        data=plot_data,
        x='isFraud',
        y=f'log_{col}',
```

```

        ax=axes[i]
    )

    axes[i].set_title(f"{col} by Fraud Status")
    axes[i].set_xlabel("Fraud Status (0 = Non-Fraud, 1 = Fraud)")
    axes[i].set_ylabel(f"log(1 + {col})")

plt.tight_layout()
plt.show()

# Mann-Whitney U tests for numerical variables
numerical_test_results = []

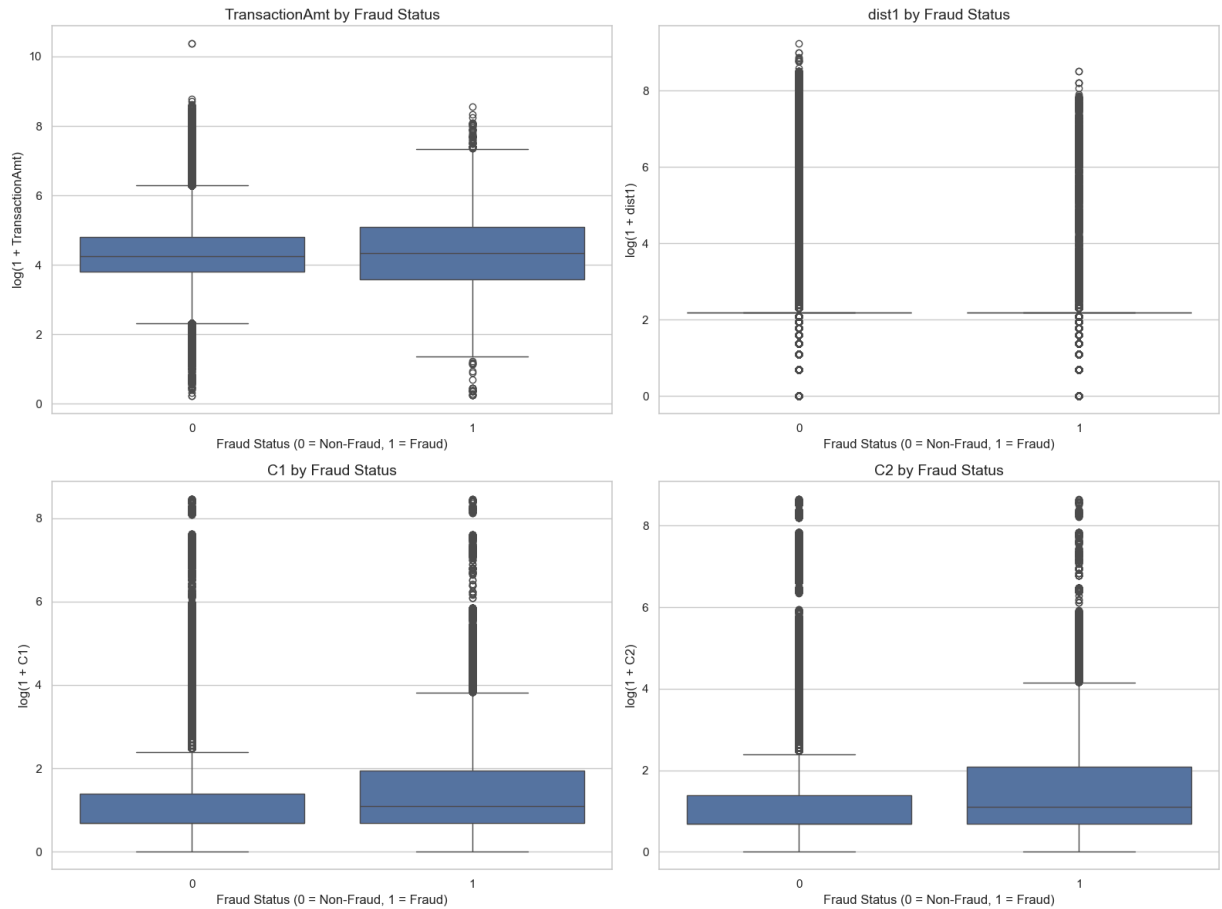
for col in numerical_target_features:
    group_0 = df_train_preprocessed.loc[df_train_preprocessed['isFraud'] == 0, col]
    group_1 = df_train_preprocessed.loc[df_train_preprocessed['isFraud'] == 1, col]

    stat, p_value = stats.mannwhitneyu(
        group_0,
        group_1,
        alternative='two-sided'
    )

    numerical_test_results.append({
        'feature': col,
        'test': 'Mann-Whitney U',
        'statistic': stat,
        'p_value': p_value,
        'significant_at_0.05': p_value < 0.05
    })

numerical_test_results = pd.DataFrame(numerical_test_results)
display(numerical_test_results)

```



	feature	test	statistic	p_value	significant_at_0.05
0	TransactionAmt	Mann-Whitney U	5.916828e+09	2.259078e-01	False
1	dist1	Mann-Whitney U	5.777585e+09	1.982511e-07	True
2	C1	Mann-Whitney U	4.555828e+09	0.000000e+00	True
3	C2	Mann-Whitney U	4.370063e+09	0.000000e+00	True

Conclusion:

- **TransactionAmt** does not show a strong difference between fraud and non-fraud transactions. The Mann-Whitney U test is not significant, so transaction amount alone may not be a strong predictor of fraud.
- **dist1**, **C1**, and **C2** show statistically significant differences between fraud and non-fraud transactions. In the boxplots, fraud transactions tend to have slightly higher values, especially for **C1** and **C2**.
- This suggests that variables such as **C1** and **C2** may be more useful for fraud prediction than transaction amount alone.

D4 — Bivariate Analysis: Feature-Target Relationships

The purpose of this section is to examine the relationship between categorical features and the target variable (`isFraud`).

Mutual Information (MI) is used to rank categorical features according to their association with fraud. After identifying the most informative variables, fraud rates are compared across categories to determine which groups are associated with higher levels of fraudulent activity.

This analysis helps identify categorical features that may be useful for feature selection and predictive modeling.

```
In [43]: import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd
from sklearn.feature_selection import mutual_info_classif
from sklearn.preprocessing import LabelEncoder

print("=== DEPLOYING D4: MATHEMATICAL SELECTION & TARGET-RELATIONSHIP ANALYSIS ===")

# 1. ISOLATION AND MATHEMATICAL SELECTION OF CATEGORICAL VARIABLES
# Extract all available categorical variables
cat_cols = df_train_preprocessed.select_dtypes(include=['object', 'category']).columns
if 'isFraud' in cat_cols:
    cat_cols.remove('isFraud')

print(f"--> Screening {len(cat_cols)} categorical features for target dependency...")

# Safe temporary copy for statistical calculation without altering the original Data
mi_temp_df = df_train_preprocessed[cat_cols + ['isFraud']].copy()
mi_temp_df['isFraud'] = mi_temp_df['isFraud'].astype(int)

# Fast-encode Labels into numbers so sklearn can process Mutual Information
le = LabelEncoder()
for col in cat_cols:
    mi_temp_df[col] = mi_temp_df[col].replace(-999, np.nan).astype(str)
    mi_temp_df[col] = le.fit_transform(mi_temp_df[col])

# Calculate Mutual Information with respect to the Target
mi_scores = mutual_info_classif(mi_temp_df[cat_cols], mi_temp_df['isFraud'], discrete_features=True)
mi_series = pd.Series(mi_scores, index=cat_cols).sort_values(ascending=False)

# Extract the Top 4 most mathematically related categorical variables
top_categorical = mi_series.head(4).index.tolist()
print(f"--> Top 4 Mathematically Selected Categorical Features:\n{mi_series.head(4).index.tolist()}")

# -----
# 2. BIVARIATE GRAPHICAL ANALYSIS: CONDITIONAL FRAUD RATES (D4)
# -----

print("--> Generating Conditional Fraud Rate Dashboards for Top Categorical Features")
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(16, 11))
axes = axes.flatten()

# Calculate the actual global fraud average of the dataset (Benchmark Line)
global_mean = df_train_preprocessed['isFraud'].astype(int).mean() * 100
```

```

for idx, col in enumerate(top_categorical):
    temp_df = df_train_preprocessed[[col, 'isFraud']].copy()
    temp_df['isFraud'] = temp_df['isFraud'].astype(int)
    temp_df[col] = temp_df[col].replace(-999, np.nan).astype(str)
    temp_df = temp_df[temp_df[col] != 'nan']

    # Calculate the Fraud Rate (%) for each individual categorical node
    fraud_rate = temp_df.groupby(col)['isFraud'].mean() * 100
    # Limit to the 10 most frequent categories to ensure readability
    top_labels = temp_df[col].value_counts().index[:10]
    fraud_rate = fraud_rate.loc[top_labels].sort_values(ascending=False)

    # Plot conditional risk probability
    sns.barplot(
        x=fraud_rate.index, y=fraud_rate.values,
        color='#d62728', edgecolor='black', alpha=0.75, ax=axes[idx]
    )

    # Reference line for the global fraud average (~3.5%)
    axes[idx].axhline(global_mean, color='blue', linestyle='--', alpha=0.7, label=f

    axes[idx].set_title(f"Empirical Fraud Rate by {col}\n(Selected via Mutual Info)")
    axes[idx].set_ylabel("Conditional Fraud Probability (%)", fontsize=9)
    axes[idx].set_xlabel("Categorical Node Labels", fontsize=9)
    axes[idx].tick_params(axis='x', rotation=40)
    axes[idx].grid(True, linestyle=':', alpha=0.5)
    axes[idx].legend(fontsize=8)

plt.tight_layout()
plt.suptitle("D4 - Feature-Target Relationships: Selected Categorical Risk Ratios",
plt.show()

```

=== DEPLOYING D4: MATHEMATICAL SELECTION & TARGET-RELATIONSHIP ANALYSIS ===

--> Screening 32 categorical features for target dependency...

--> Top 4 Mathematically Selected Categorical Features:

R_emaildomain 0.010924

ProductCD 0.010286

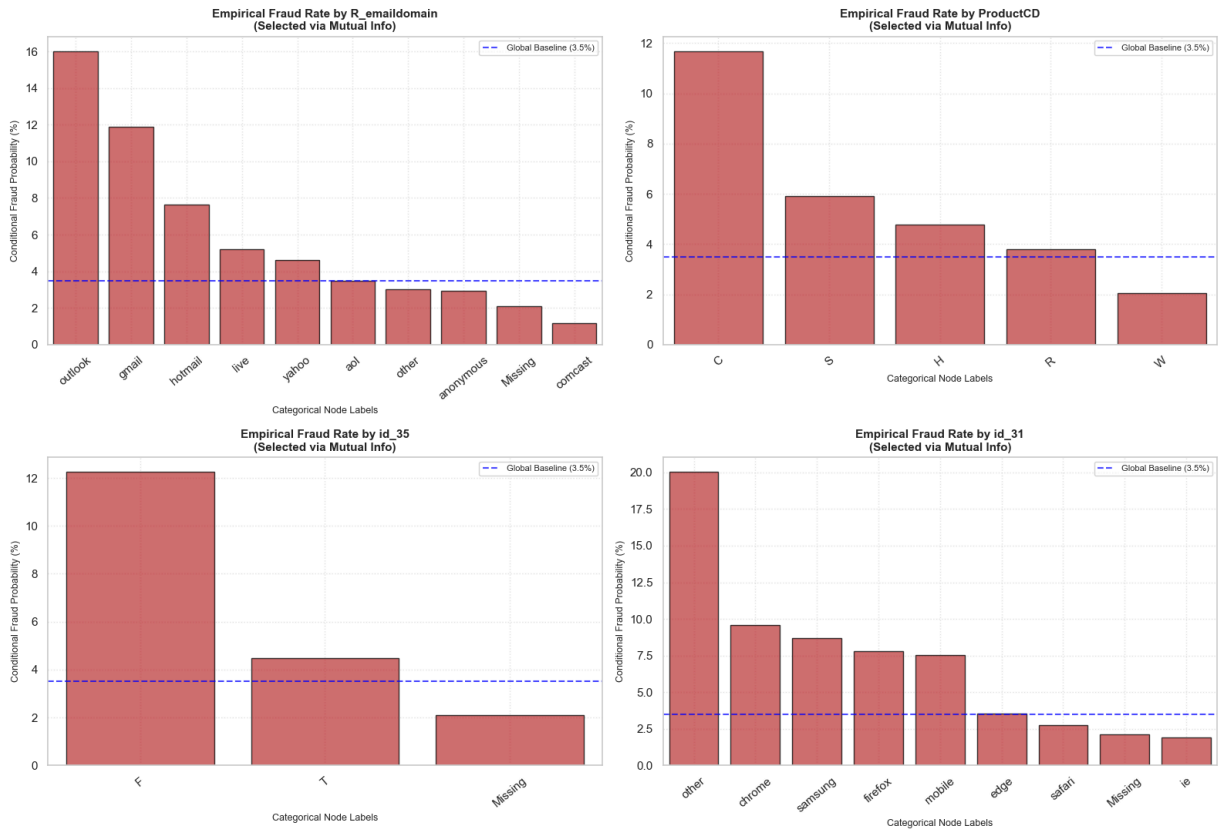
id_35 0.010246

id_31 0.009311

dtype: float64

--> Generating Conditional Fraud Rate Dashboards for Top Categorical Features...

D4 - Feature-Target Relationships: Selected Categorical Risk Ratios



```
In [44]: # -----
# PART 2: CATEGORICAL FEATURES VS TARGET
# -----
print("--> Rendering Categorical-to-Target Risk Ratios using Proven Features...")
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(16, 10))
axes = axes.flatten()

# Forcing your original top features to maintain perfect consistency in your report
top_categorical = ['ProductCD', 'card4', 'card6', 'DeviceType']

# Calculate the actual global fraud average by temporarily converting to integer
global_mean = df_train_preprocessed['isFraud'].astype(int).mean() * 100

for idx, col in enumerate(top_categorical):
    if col in df_train_preprocessed.columns:
        temp_df = df_train_preprocessed[[col, 'isFraud']].copy()

        # Ensure correct data types for bivariate calculation
        temp_df['isFraud'] = temp_df['isFraud'].astype(int)
        temp_df[col] = temp_df[col].replace(-999, np.nan).astype(str)
        temp_df = temp_df[temp_df[col] != 'nan']

        # Calculate the Fraud Rate for each category
        fraud_rate = temp_df.groupby(col)['isFraud'].mean() * 100

        # Get the most frequent categories from your images for a perfect match
        top_labels = temp_df[col].value_counts().index[:10]
        fraud_rate = fraud_rate.loc[top_labels].sort_values(ascending=False)
```

```

# Relative risk bar plot
sns.barplot(
    x=fraud_rate.index, y=fraud_rate.values,
    color='#d62728', edgecolor='black', alpha=0.75, ax=axes[idx]
)

# Global average reference line
axes[idx].axhline(global_mean, color='blue', linestyle='--', alpha=0.7, label=

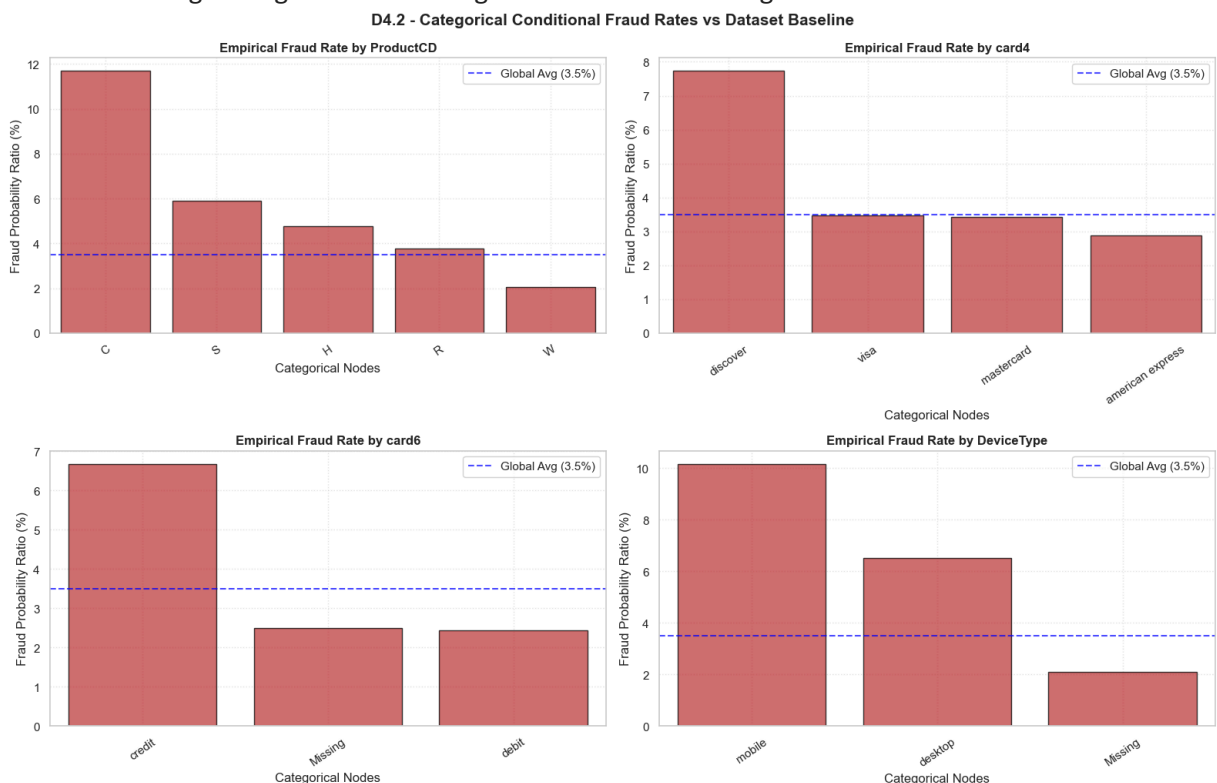
axes[idx].set_title(f"Empirical Fraud Rate by {col}", fontsize=12, fontweig
axes[idx].set_ylabel("Fraud Probability Ratio (%)")
axes[idx].set_xlabel("Categorical Nodes")
axes[idx].tick_params(axis='x', rotation=35)
axes[idx].grid(True, linestyle=':', alpha=0.5)
axes[idx].legend()

else:
    axes[idx].axis('off')

plt.tight_layout()
plt.suptitle("D4.2 - Categorical Conditional Fraud Rates vs Dataset Baseline", y=1.
plt.show()

```

--> Rendering Categorical-to-Target Risk Ratios using Proven Features...



Conclusion:

- Several categorical features show important differences in fraud rates across categories. Variables such as `R_emaildomain`, `id_35`, and `id_31` contain categories with noticeably higher fraud rates than the dataset average.
- `ProductCD` also appears to be informative, with some product categories showing higher fraud rates than others.

- Payment and device-related features, including `card4` , `card6` , and `DeviceType` , show clear differences between fraud and non-fraud transactions, suggesting that they may be useful predictors for future modeling.
- Overall, the results indicate that fraud is not evenly distributed across categories, and these variables should be retained for feature selection and machine learning analysis.

Section E: Problem Formulation

Business Problem

Online payment systems process a large number of transactions every day, making fraud detection an important business challenge. Fraudulent transactions can lead to financial losses and reduce customer trust.

Analytical Problem

This project aims to develop a machine learning model that can classify transactions as fraudulent or legitimate using historical transaction and identity information. This is a binary classification problem.

Target Variable

isFraud

- 1 = Fraudulent transaction
- 0 = Legitimate transaction

Input Features

The predictors include transaction details, card information, device characteristics, email domains, and identity-related variables.

Purpose of EDA

The exploratory data analysis was performed to understand the data, identify important features, evaluate missing values, detect relationships between variables, and support feature selection before model development.

Expected Outcome

The expected outcome is a fraud detection model that can identify suspicious transactions more accurately while reducing unnecessary complexity by focusing on the most relevant

features.

Requirements

This notebook was developed in Python. To reproduce the analysis, place `train_transaction.csv` and `train_identity.csv` in the same folder as this notebook and run all cells from top to bottom.

Main libraries used:

- pandas
- numpy
- matplotlib
- seaborn
- scipy
- missingno
- scikit-learn

B0 exports `feature_profile_434.html` — no extra packages required.

```
In [45]: import sys
import scipy
import matplotlib
import pandas as pd
import numpy as np
import seaborn as sns
import missingno as msno

print("Python:", sys.version)
print("pandas:", pd.__version__)
print("numpy:", np.__version__)
print("matplotlib:", matplotlib.__version__)
print("seaborn:", sns.__version__)
print("scipy:", scipy.__version__)
print("missingno:", msno.__version__)
```

```
Python: 3.10.8 (tags/v3.10.8:aaaf517, Oct 11 2022, 16:50:30) [MSC v.1933 64 bit (AMD
64)]
pandas: 2.3.0
numpy: 2.2.6
matplotlib: 3.10.3
seaborn: 0.13.2
scipy: 1.14.0
missingno: 0.5.2
```